

人工智能课程组
系列课程

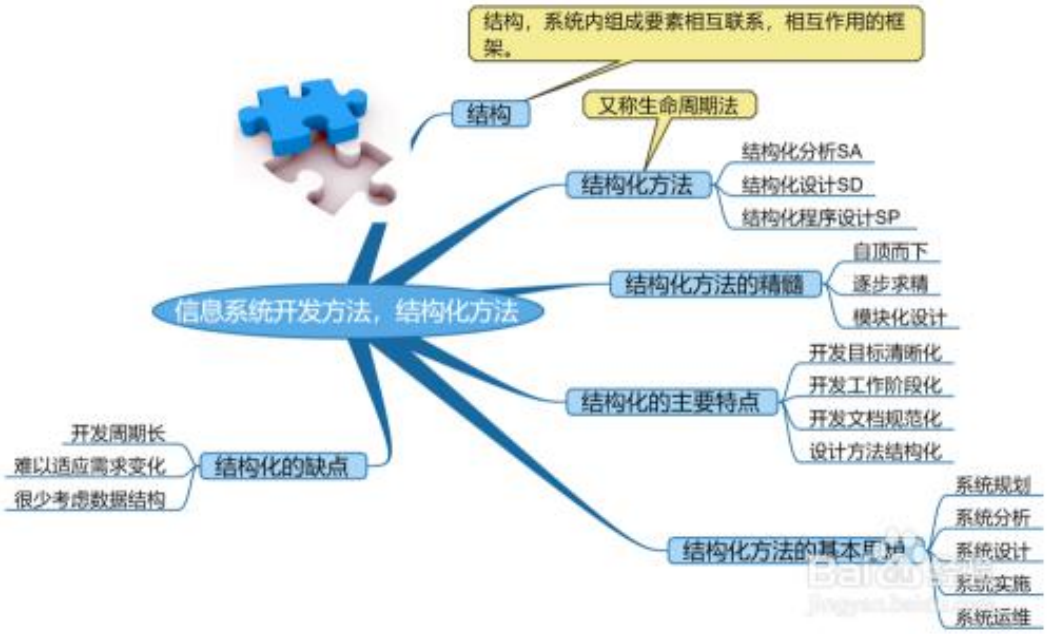
软件工程
软件设计

陈龙彪
厦门大学
2025年秋学期

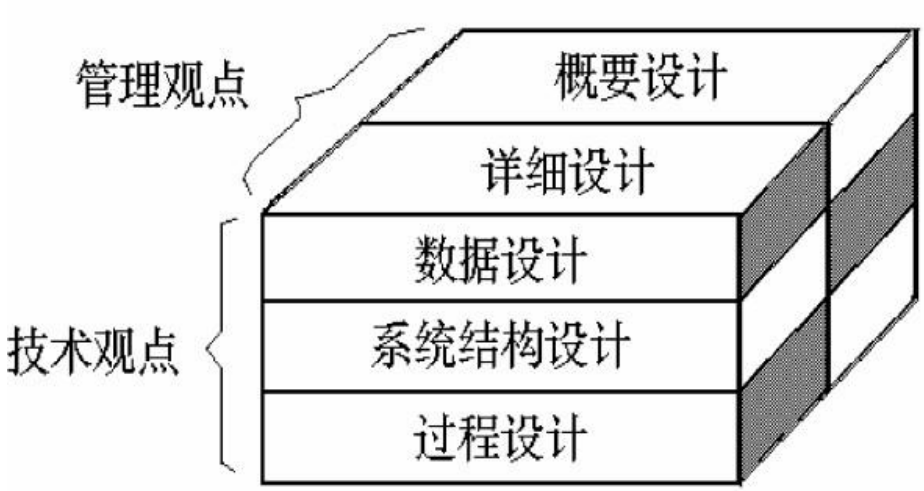
第13章：软件设计概述

软件设计

- 软件结构化设计（Structured Design）分为两个阶段
  - 总体设计：通过仔细分析软件规格说明，适当地对软件进行功能分解，从而把软件划分为模块，并且设计出对应的模块结构
  - 详细设计：详细设计每个模块，确定每个模块所需要的算法和数据结构



从不同角度对软件设计的认识



我壓力太大了

做工程師壓力很大嗎？

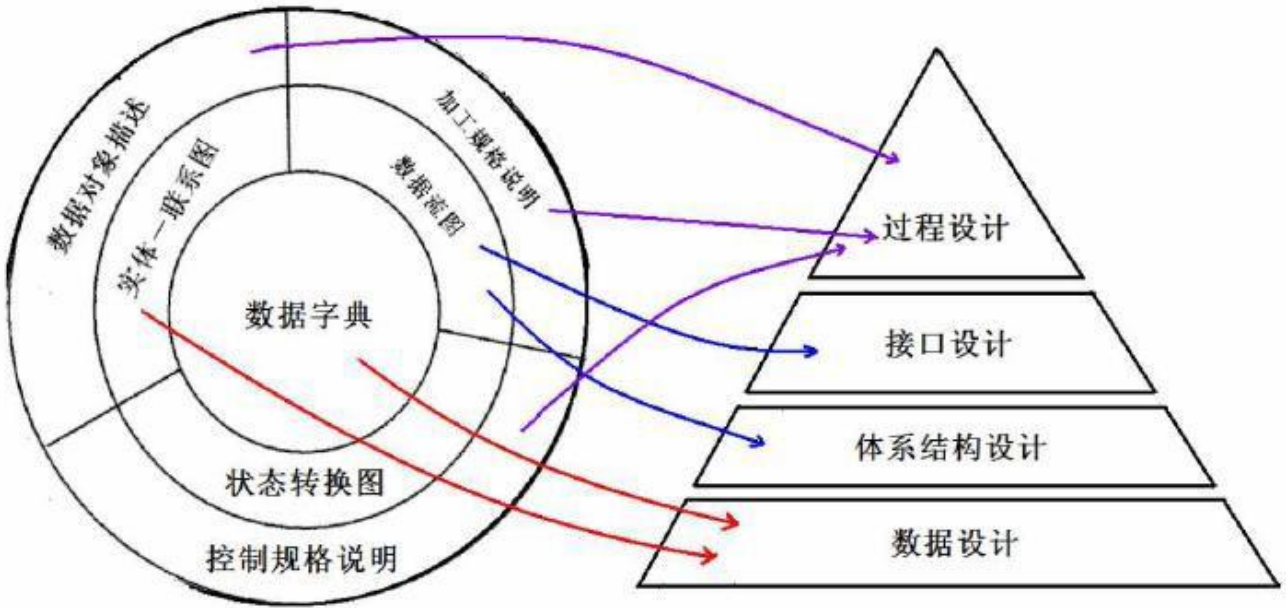
我昨天夢到我死了

到了地獄

閻羅王要我把生死簿做成後台管理系統

软件设计

- 在软件分析模型中的需求信息（数据、功能和行为）基础上，产生软件系统各个层次上的设计方案



软件设计的内容

- **数据/类设计**：将分析-类模型变换成类的实现和软件实现所需要的数据结构
- **体系结构设计**：体系结构设计定义了软件的主要 结构元素以及相互之间的关系
- **接口设计**：接口设计描述了软件内部、软件和协 作系统之间以及软件同人之间如何通信
- **部件级设计**：部件级设计将软件体系结构的结构 性元素变换为对软件部件的过程性描述

软件设计的步骤

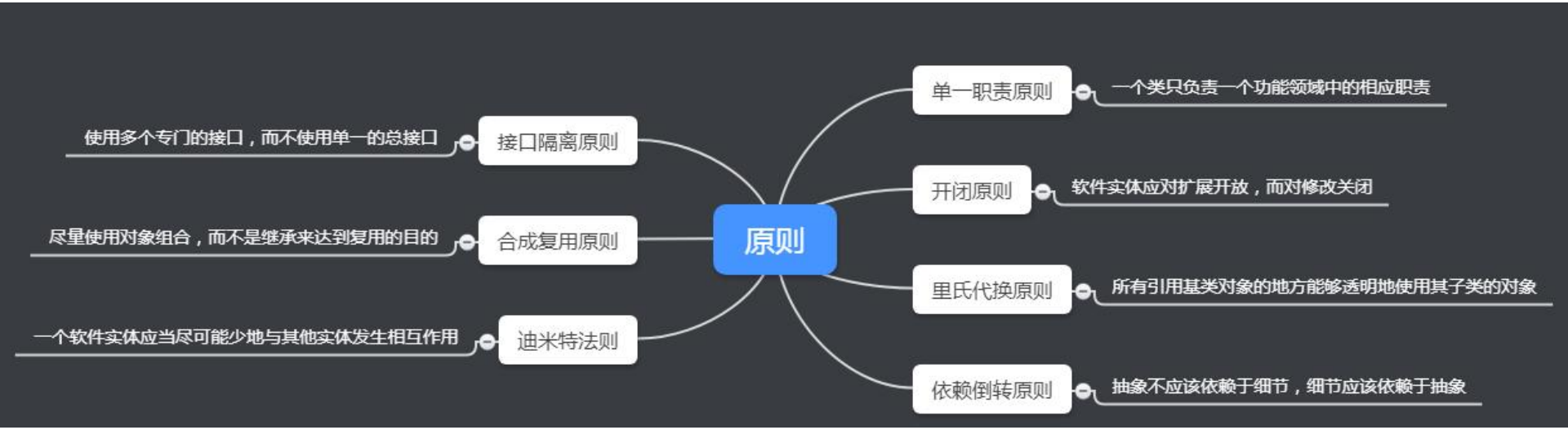
1. **制定设计规范**
   - 通过阅读系统需求说明书确定设计目标以及这些目标的优先顺序
   - 设计方法：结构化、面向对象...
   - 设计文档的编制标准
   - 基本实现规范：代码的信息形式、与硬件及操作系统的接口规约、命名规则
2. **系统结构设计**
   - 划分层次结构
   - 确定功能模块
   - 确定模块调用关系
   - 确定模块的接口
   - 评估模块划分的质量、导出模块结构规则

软件设计的步骤

3. **数据结构设计**
   - 确定输入、输出文件的结构
   - 数据库设计
4. **可靠性设计（可靠性、质量、易于维护）**
5. **编写设计文档**
   - 概要设计说明书、数据库设计说明书、用户手册、初步测试计划等
6. **设计评审**
   - 可追溯性、接口、风险、实用性、技术清晰度、可维护性、质量、可选方案等方面

软件设计的基本原则

- **抽象**：考虑高层问题时忽略低层细节
- **逐步求精**：不断接近问题的完整解
- **模块化**：将系统划分为相对独立但又有所关联的多个部分
- **信息隐藏**：对其它模块隐藏内部细节
- **模块独立**：模块功能相对独立—高内聚/低耦合



抽象

- 人们在实践中认识到，在现实世界中一定事物/状态或过程之间总存在着某些相似的方面（共性）
- **抽象就是抽出事物的本质特性而暂时不考虑它们的细节**
- 抽象是控制复杂性的基本策略
- **抽象过程**：从特殊到一般的过程，上层概念是下层概念的抽象，下层概念是上层概念的精化和细化
- 软件开发过程的每一步都是对较高一级抽象的解作一次具体化的描述

过程抽象与数据抽象

- **过程抽象**
  - 使用者将功能体作为单个功能看待
  - 这些功能实际上是由一系列更低级的功能或代码来实现的
- **数据抽象**
  - 定义数据类型和施加于该类型对象的操作
  - 限定了对象的取值范围，只能通过这些操作修改和观察数据

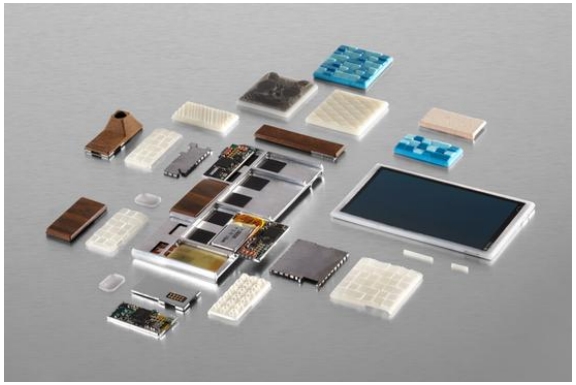
逐步求精

- 把问题的求解过程分解成若干步骤或阶段，每步都比上步更精化，更接近问题的解法
- 常与分层抽象的思想相结合
  - 抽象使得设计者能够描述过程和数据而忽略低层的细节
  - 求精有助于设计者在设计过程中揭示低层的细节
  - 高层抽象将在下层不断精化，最终得到软件实现

模块化分解

- 按照设计原则将系统划分为若干个较小的模块
  - 相互独立但又相互关联
  - 实际上是系统分解和抽象的过程
- 模块是相对独立的程序体
  - 是数据说明、可执行语句等程序对象的集合
  - 单独命名的，并且可以通过名字来访问
    - 例如：过程、函数、子程序、宏等





通过模块化降低开发复杂度

- C(x)：问题x的复杂性
- E(x)：解决问题x所需工作量
- 对于两个问题p1和p2：
  1. 如果C(p1)>C(p2)，那么E(p1)>E(p2)
     - 问题越复杂解决，问题所需要的花费更多
  2. C(p1+p2)>C(p1)+C(p2)，因此E(p1＋p2)>E(p1)+E(p2)
     - 将复杂问题分解成可以多个子问题分别解决会更加容易

模块化实例：智能灌溉

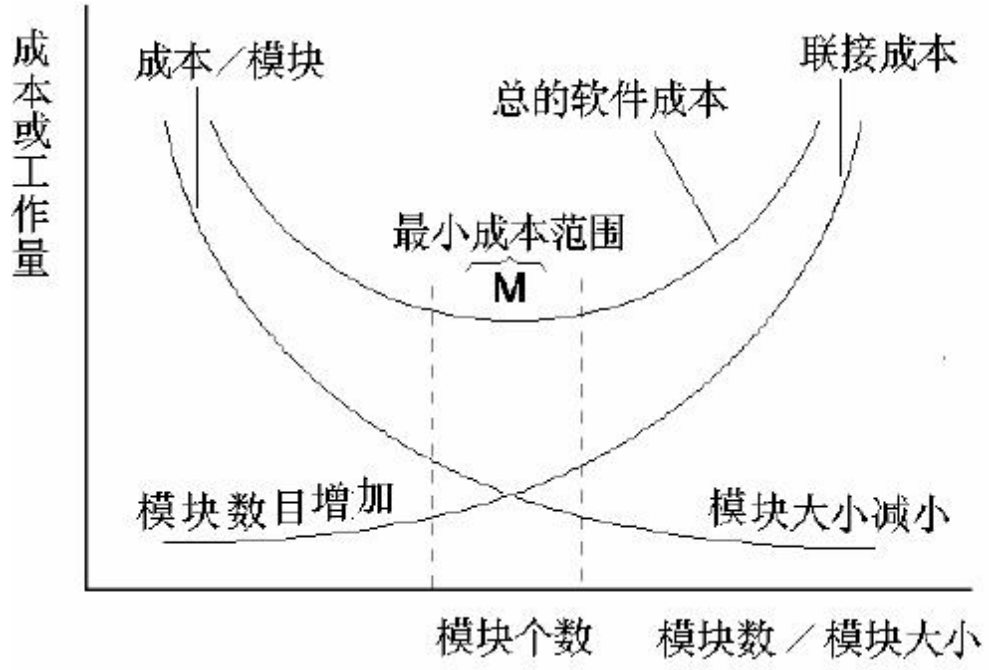
- 总体设计：将系统划分为传感器、灌溉决策和灌溉设备控制三个模块，定义好相互之间的数据和控制接口
  - 传感器模块：接受传感器收集的温度、湿度、土壤等各种数据
  - 灌溉决策：根据种植经验和收集到的数据判断灌溉决策
  - 灌溉设备控制：根据决策控制灌溉设备进行灌溉
- 降低整体复杂度：可以分而治之，而且可以由具备不同专业知识的开发者分别承担

误区：无限制地进行模块化分？

- 如果我们无限制地划分软件，开发它所需的工作量会变得小到可以忽略？
- 结论：**NO**
  - 模块数量增加时，只是使各个子模块的工作量之和有所减小
  - 然而开发工作量还有很大一部分来自于模块间的接口和集成
    - 除了技术上的接口和集成，还包括人与人之间的沟通
    - 集成和沟通的开销到了一定程度就会成为开发工作量的主要部分

结论：适度的模块化

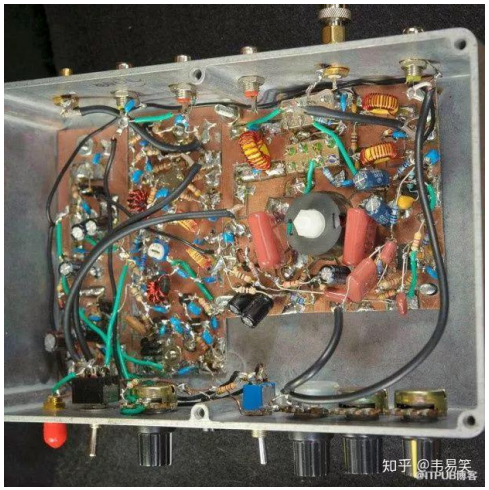
- 模块数增加时，模块间的关系也随之增加，接口和集成的工作量也随之增加
- 结论：寻找最佳模块化程度平衡点



模块独立

- 模块独立是模块化的根本要求
  - 模块完成独立的功能：明确可辨识
  - **高内聚**：内部结构紧密
  - **低耦合**：模块间关联和依赖程度尽可能小，与其他模块接口简单
  - 符合信息隐藏和信息局部化原则





模块独立的重要性

- 每个模块都尽量对其他模块隐藏自己的内部实现细节
  - 模块内部的数据和过程不允许其它不需要这些信息的模块使用
  - 定义和实施对模块的过程细节和局部数据结构的存取限制
- 模块的开发者专注于某一个相对独立的部分
- 不用过多关心其他模块
- 修改和bug影响的范围被局部化
- 单个模块更容易复用
- 独立的模块更易于维护和测试

内聚度的七个层次

- **巧合内聚**：将几个模块中的相同程序代码段独立出来建立的模块（无明显独立性）
- **逻辑内聚**：完成一组逻辑相关任务的模块，由控制型参数来确定执行哪一种功能
- **时间内聚**：模块中的多个任务必须在一段时间内先后执行



内聚度的七个层次

- **过程内聚**：模块内的多个任务必须按指定的过程执行
- **通信内聚**：模块内所有处理元素都集中在某个数据结构的一块区域中（例如对课程进行选、退课和查询）
- **顺序内聚**：指一个模块完成多个功能，这些功能又必须顺序执行（更加单一的过程内聚）
- **功能内聚**：指一个模块中各个部分都是为完成一项具体功能而协同工作，紧密联系，不可分割的（单个功能）



耦合度的七个层次

- **内容耦合**：一个模块可以直接访问另一个模块的内部数据或内部功能
- **公共耦合**：多个模块共同访问某些公共数据元素
- **外部耦合**：多个模块间需要遵循同样的外部约束，例如通信协议、数据格式等



耦合度的七个层次

- **控制耦合**：模块间的交互参数包含控制信息，可 影响另一个模块的执行逻辑
- **标记耦合**：模块间传递特定的数据结构
- **数据耦合**：模块间仅传递简单数据
- **非直接耦合**：两个模块可以相对独立工作



模块独立性：高内聚低耦合

- 设计目标：高内聚低耦合的模块
  - 一个模块内部各个元素之间的联系越紧密，则它的内聚性就越高
  - 模块之间的连接越紧密，联系越多，耦合性就越高，而其模块独立性就越弱

软件设计的（质量）目标

- **满足用户需求**
  - 必须实现分析模型中所有的显式需求
  - 必须满足用户希望的所有隐式需求
- **可读、可理解**：设计必须是可读、可理解的，使 得将来易于编程、易于测试、易于维护
- **全面性**：应从实现角度出发，给出与数据、功能、 行为相关的软件全貌

设计评审

- 软件设计的最终目标是**取得最佳方案**
  - 在所有候选方案中，在节省开发费用、降低资源消耗、缩短开发时间的条件下，选择能够赢得较高的生产率、较高的可靠性和可维护性的方案

Ⅰ. 工作范围	A. 系统目标
	B. 运行环境
	C. 主要软件需求
	D. 设计约束 / 限制
Ⅱ. 体系结构设计	A. 数据流与控制流复审
	B. 导出的程序结构
	C. 功能与程序交叉索引
Ⅲ. 数据设计	A. 数据对象与形成的数据结构
	B. 文件和数据库结构 i 文件的逻辑结构 ii 文件逻辑记录描述 iii 访问方式
	C. 全局数据
	D. 文件 / 数据与程序交叉索引
Ⅳ. 接口设计	A. 人机界面规格说明
	B. 人机界面设计规则
	C. 外部接口设计 i 外部数据接口 ii 外部系统或设备接口
	D. 内部接口设计规则
Ⅴ. 各部件的过程设计	A. 处理与算法描述
	B. 接口描述
	C. 设计语言(或其它)描述
	D. 使用的部件
	E. 内部程序逻辑描述
	F. 注释 / 约束 / 限制
Ⅵ. 运行设计	A. 运行部件组合
	B. 运行控制规则
	C. 运行时间安排
Ⅶ. 出错处理设计	A. 出错处理信息
	B. 出错处理对策 i 设置后备 ii 性能降级 iii 恢复和再启动
Ⅷ. 安全保密设计	
Ⅸ. 需求 / 设计交叉索引	
Ⅹ. 测试部分	A. 测试方针
	B. 集成策略
	C. 特殊考虑
Ⅺ. 特殊注解	
Ⅻ. 附录	

设计评审

- 评审分**正式评审**和**非正式评审**两种
  - 正式评审除软件开发人员外，还邀请用户代表和领域专家参加，通常采用**答辩**形式
  - 非正式评审有些同行切磋的性质，不拘泥于时间和形式

设计评审的内容

- **可追溯性**：即分析该软件的系统结构、子系统结构，确认该软件设计是否覆盖了所有已确定的软件需求，软件每一成分是否可追溯到某一项需求
- **接口**：即分析软件各部分之间的联系，确认该软件的内部接口与外部接口是否已经明确定义。部件是否满足高内聚和低耦合的要求。部件作用范 围是否在其控制范围之内
- **风险**：即确认该软件设计在现有技术条件下和预算范围内是否能按时实现
- **实用性**：即确认该软件设计对于需求的解决方案是否实用

设计评审的内容

- **技术清晰度**：即确认该软件设计是否以一种易于翻译 成代码的形式表达
- **可维护性**：从软件维护的角度出发，确认该软件设计 是否考虑了方便未来的维护
- **质量**：即确认该软件设计是否表现出良好的质量特征。
- 各种选择方案：看是否考虑过其它方案，比较各种选 择方案的标准是什么
- **限制**：评估对该软件的限制是否现实，是否与需求一 致
- **其它具体问题**：对于文档、可测试性、设计过程等等 进行评估

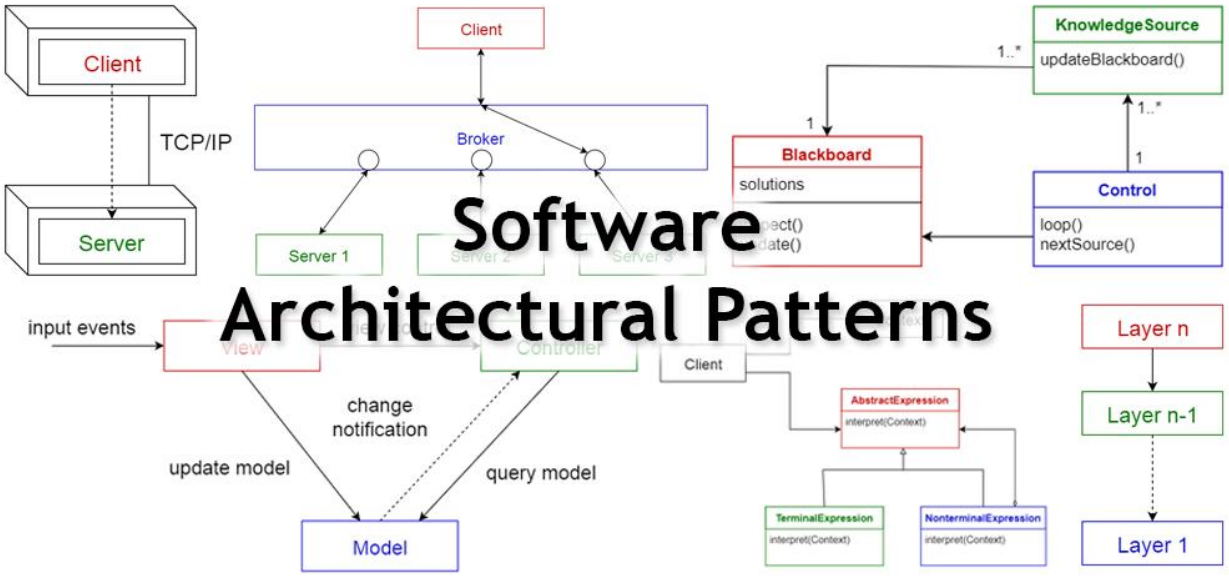
总结：软件设计的技术原则

- **分层设计**：建立软件系统的分层控制结构
- **模块化**：从逻辑上将软件划分为完成特定功能的部件
- **数据与过程**：设计既包含数据抽象，也包含过程 抽象
- **高内聚**：建立具有独立功能特征的模块
- **低耦合**：尽量降低模块与外部环境之间的接口复 杂度
- **系统化/方法化**：设计应能根据软件需求分析获取的信息，建立可驱动、可重复的方法

第14章：体系结构设计

软件体系结构设计

- 软件体系结构关注系统的结构，包含软件**部件**、部件的**属性**以及部件间的**关系**
- 体系结构的作用
  - 方便利益相关人员的交流
  - 有利于系统设计的前期决策
  - 可传递、易于理解的系统级抽象



为什么要进行软件系统设计

- 软件系统实现的高层方案
  - 软件系统越来越复杂，需要将其划分为若干**部分**，分而治之
  - 负责不同部分的开发者对于其它模块需要了解的**信息**越少越好
  - 这些部分之间还需要定义清晰、明确的**接口**
- 开发者之间的分工和合作的基础
- 决定系统质量的主要因素
  - 设计方案可以借鉴成熟的设计经验
  - 设计方案完成后需要经过质量分析和评审

- **好的设计方案不仅能很好地支持当前需求的实现，而且能减小未来的系统维护（理解、修改、扩展）成本**

体系结构发展过程

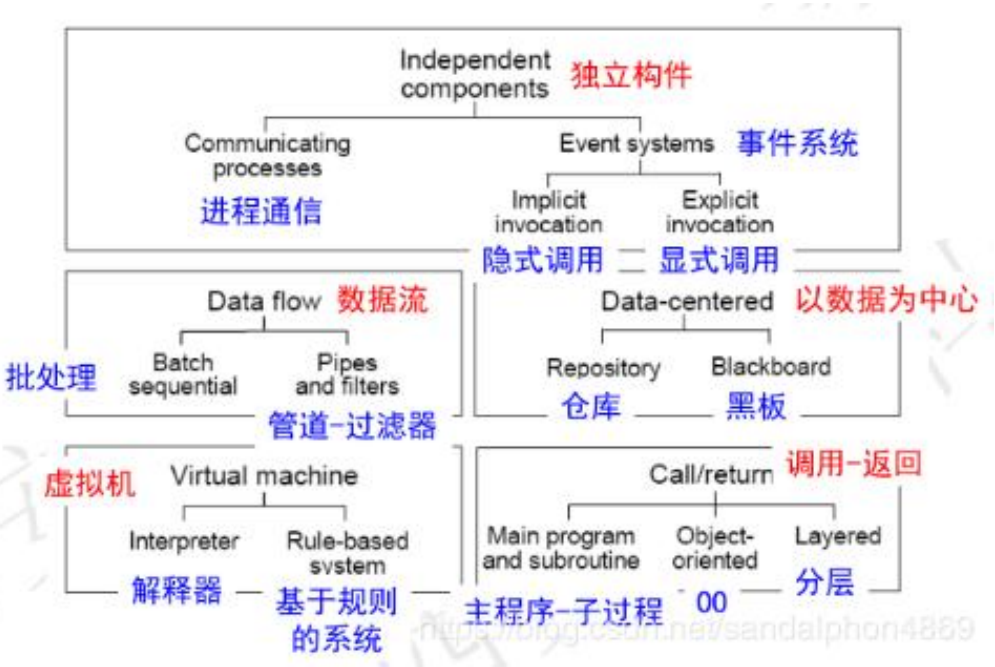
- 单主机结构
  - 界面、数据和程序集中在单台主机上
  - 不需要考虑多用户并发操作的问题
- C/S（Client/Server）结构
  - 常见的服务器端体现为关系数据库
  - 客户端负责显示和业务逻辑处理
  - 部署和扩展性存在不足：系统升级需要更新所有客户端
- B/S（Browser/Server）结构
  - 浏览器或客户端程序（Applet等）





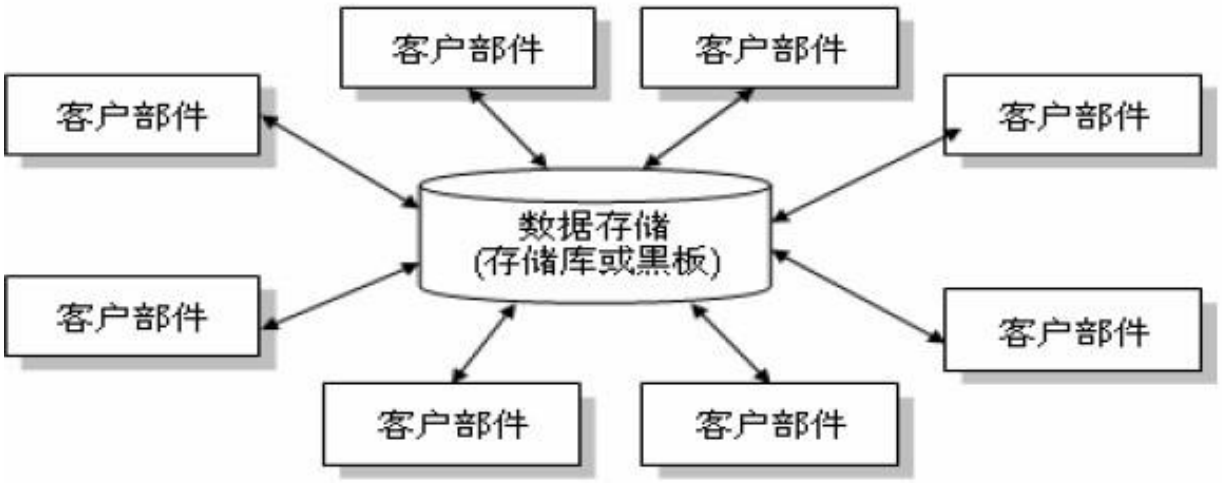
软件体系结构风格

- **体系结构风格**定义了一系列系统的结构组织的模式，它是对一类具有相似结构的系统体系结构的抽象
  - 一些实现系统所需的功能的部件
  - 连接各个部件，负责部件间通信和协作的连接器
  - 定义部件之间怎样整合的系统约束
  - 使设计者能够理解整个系统属性并分析已知属性的语义模型



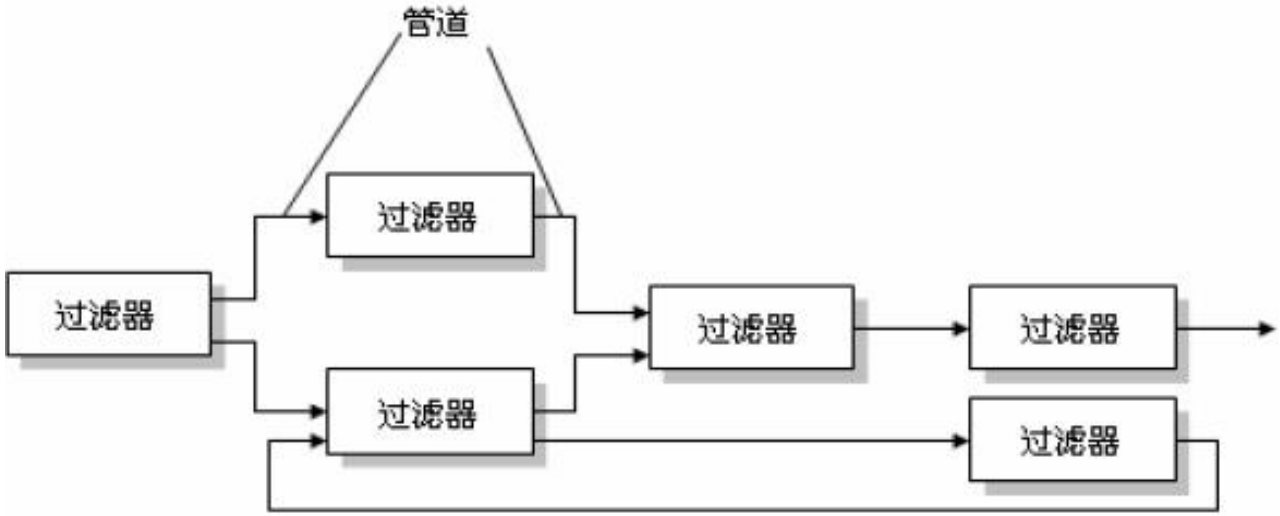
数据中心风格的体系结构

- 一些数据（比如一个文件或者数据库）保存在整个结构的中心，并且被其他部件频繁地使用、添加、删除、或修改



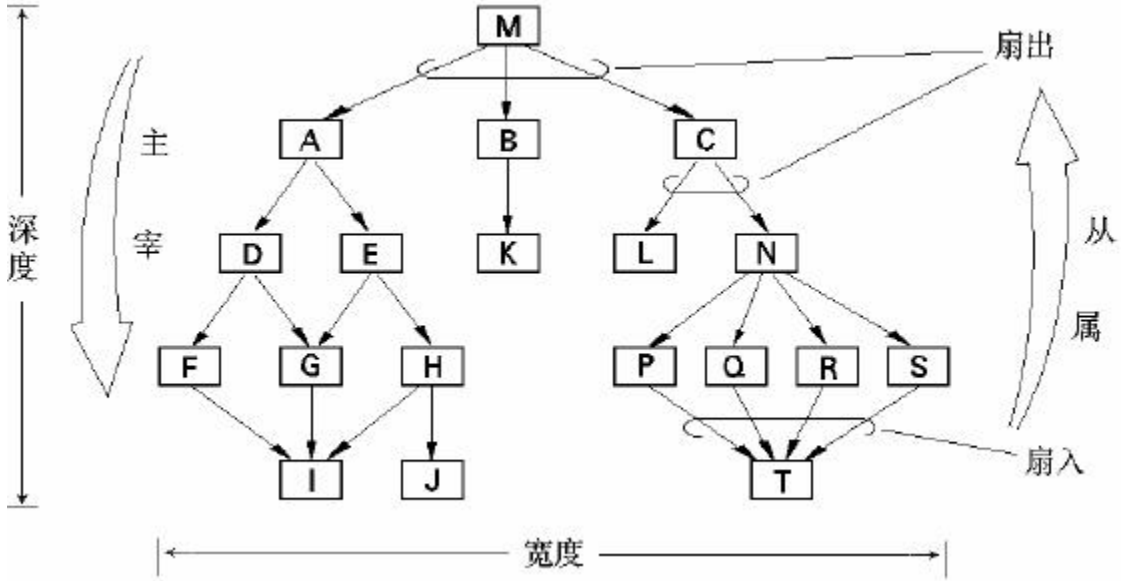
数据流风格的体系结构

- 又称**管道/过滤器**体系结构风格
- 这种结构适用于输入数据被一系列的计算或者处理部件变换成输出数据



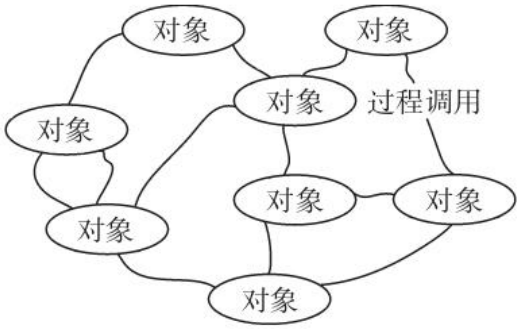
调用和返回风格的体系结构

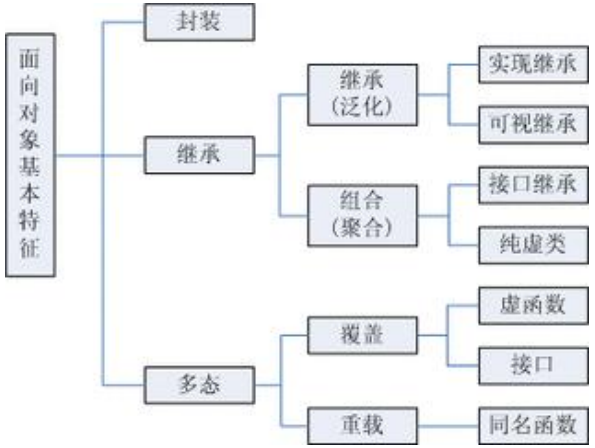
- 这种风格使一个软件设计者设计出非常容易修改和扩充的体系结构
  - 主程序/子程序风格体系结构
  - 远程过程调用风格的体系结构



面向对象风格的体系结构

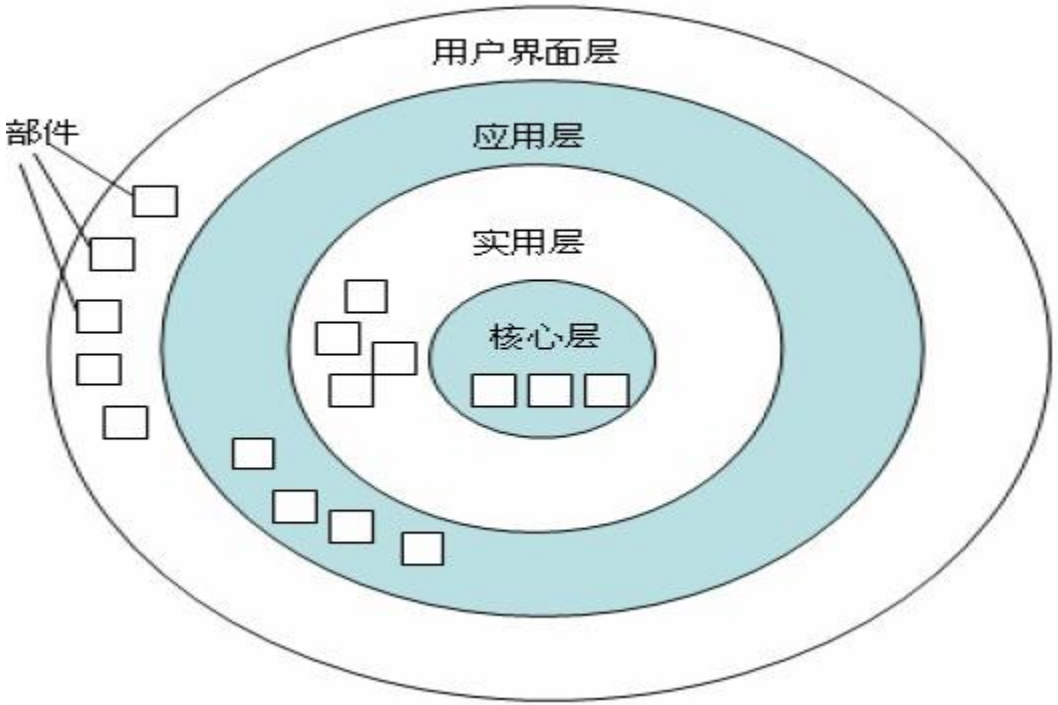
- 系统部件封装数据和操作数据的方法
- 部件之间的交互和协调通过消息来传递





层次式风格的体系结构

- 在这种结构中，定义不同的层次，每层都完成了 相对外层更靠近机器指令的操作



体系结构设计规则

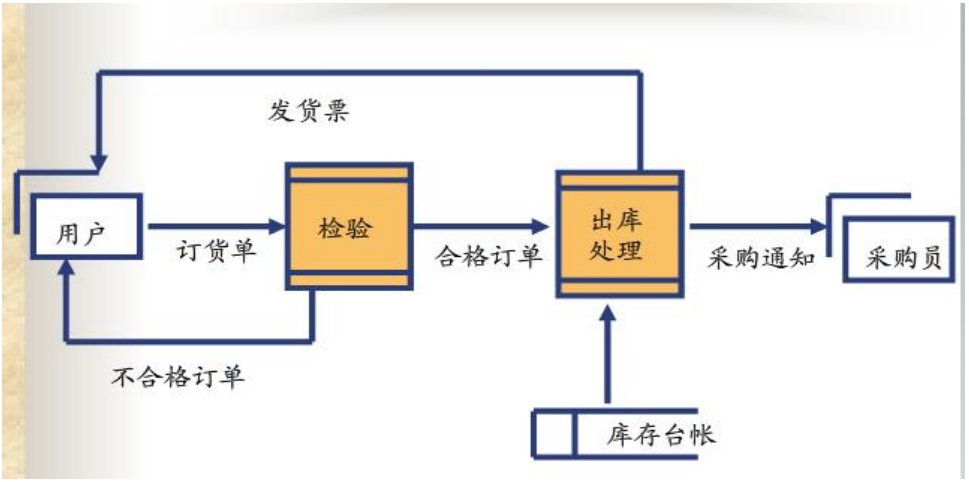
在进行软件体系结构设计时，可参考如下规则：

1. 改进软件结构提高**模块独立性**
2. 控制模块的**作用范围**
3. 降低模块**接口的复杂度**
4. 设计**单入口单出口**的模块
5. **模块功能**应该是可以预测
6. 一般一个模块的语句在30～50条左右较好



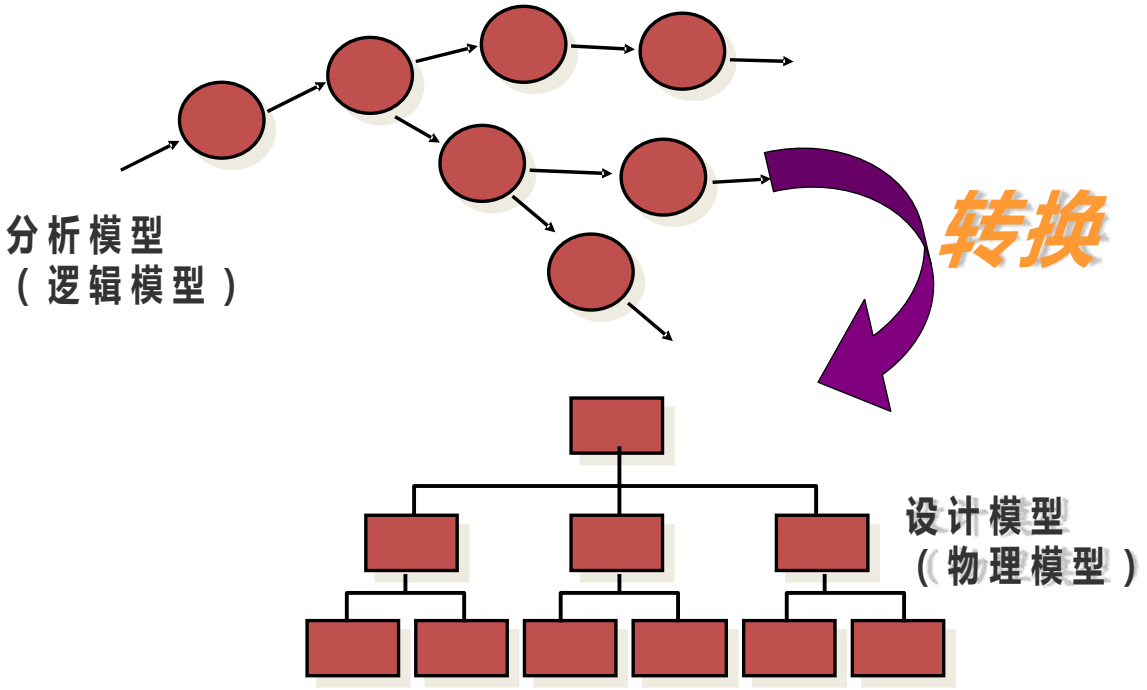
数据流图

- 数据流图（**DFD**：Data Flow Diagram）是组织中信息运动的抽象，是信息逻辑系统模型的主要形式
- 这个模型不涉及硬件、软件、数据结构与文件组织，它与对系统的物理描述无关，只是**用图形及与此相关的注释来表示系统的逻辑功能**
- 数据流图是系统分析人员与用户之间进行交流的有效手段，也是系统设计的主要依据之一



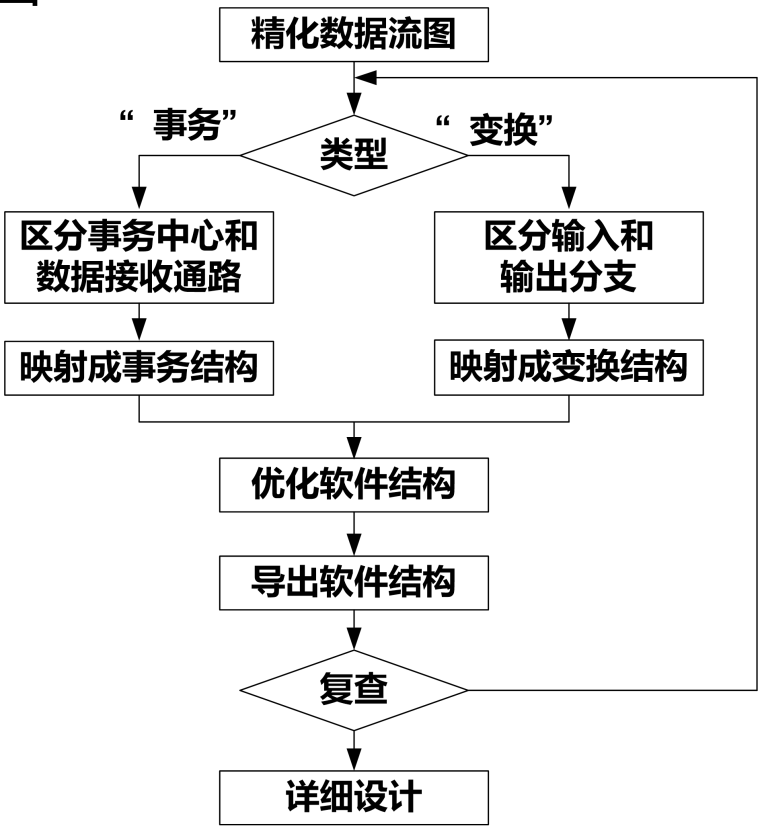
数据流图映射到体系结构图

- 结构化设计是将结构化分析的结果（数据流图，DFD）映射成软件的体系结构（结构图），以系统化的途径实现软件结构的设计
- 将数据流图分为**变换型**数据流图和**事务型**数据流图，对应的映射分别称为**变换分析**和**事务分析**



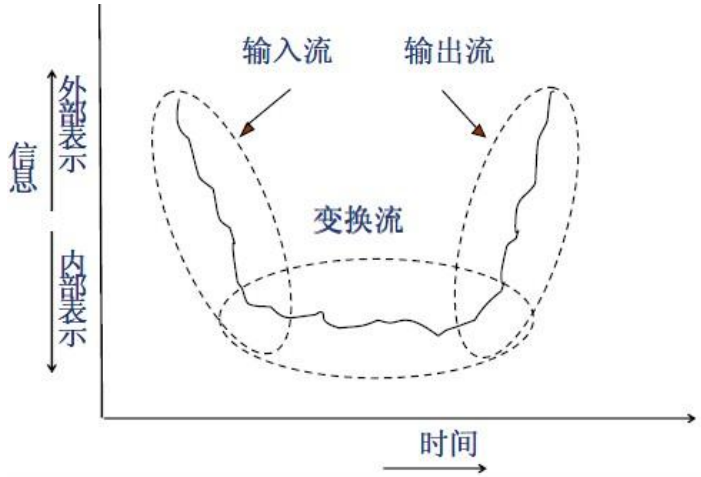
数据流图映射到体系结构图

- 复审和精化数据流图
- 确定数据流图的类型（变换型、事务型）
- 将数据流图映射成初始结构图：变换分析或事务分析
- 改进初始结构图



变换分析

- 特征：数据流图可明显地分成**输入、变换、输出**三部分
- 外部信息沿着输入路径进入系统，经过编辑、格式转换、合法性检查、预处理等辅助性加工后变成内部形式
- 内部形式的信息由变换中心进行处理
- 沿着输出路径经过格式转换、组成物理块、缓冲处理等辅助性加工后变成输出信息送到系统外





变换分析

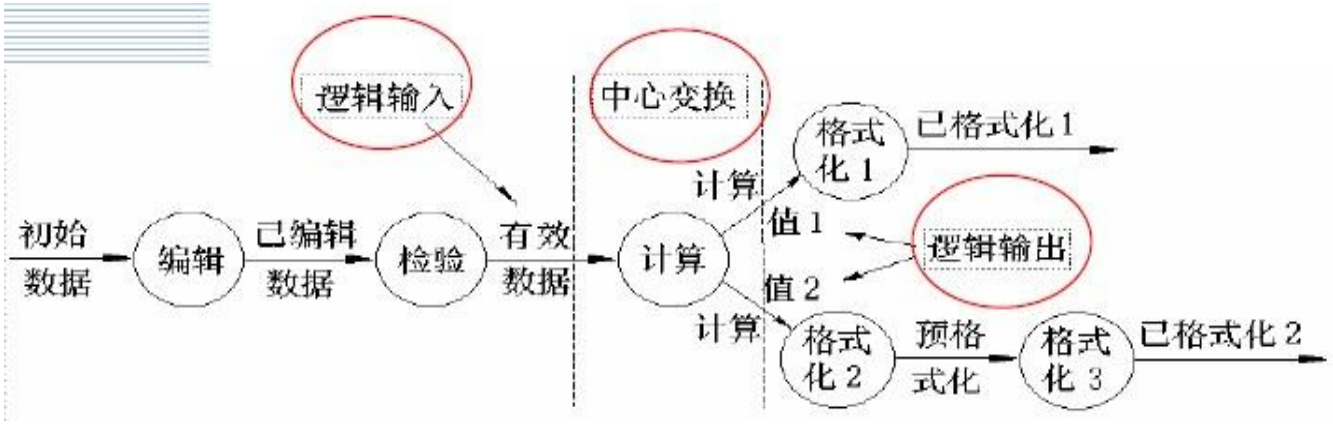
- 变换分析的任务是将变换型的数据流图映射成初始结构图，步骤如下：
  - 划定输入流和输出流的边界，确定变换中心
  - 进行第一级分解，设计上层模块：将DFD映射成变换型的程序结构
  - 进行第二级分解，设计中下层模块：将DFD中的加工映射成结构图中的一个适当的模块
  - 标注输入输出信息：根据DFD，在初始结构图上标注模块之间传递的输入信息和输出信息

确定输入输出流边界和变换中心

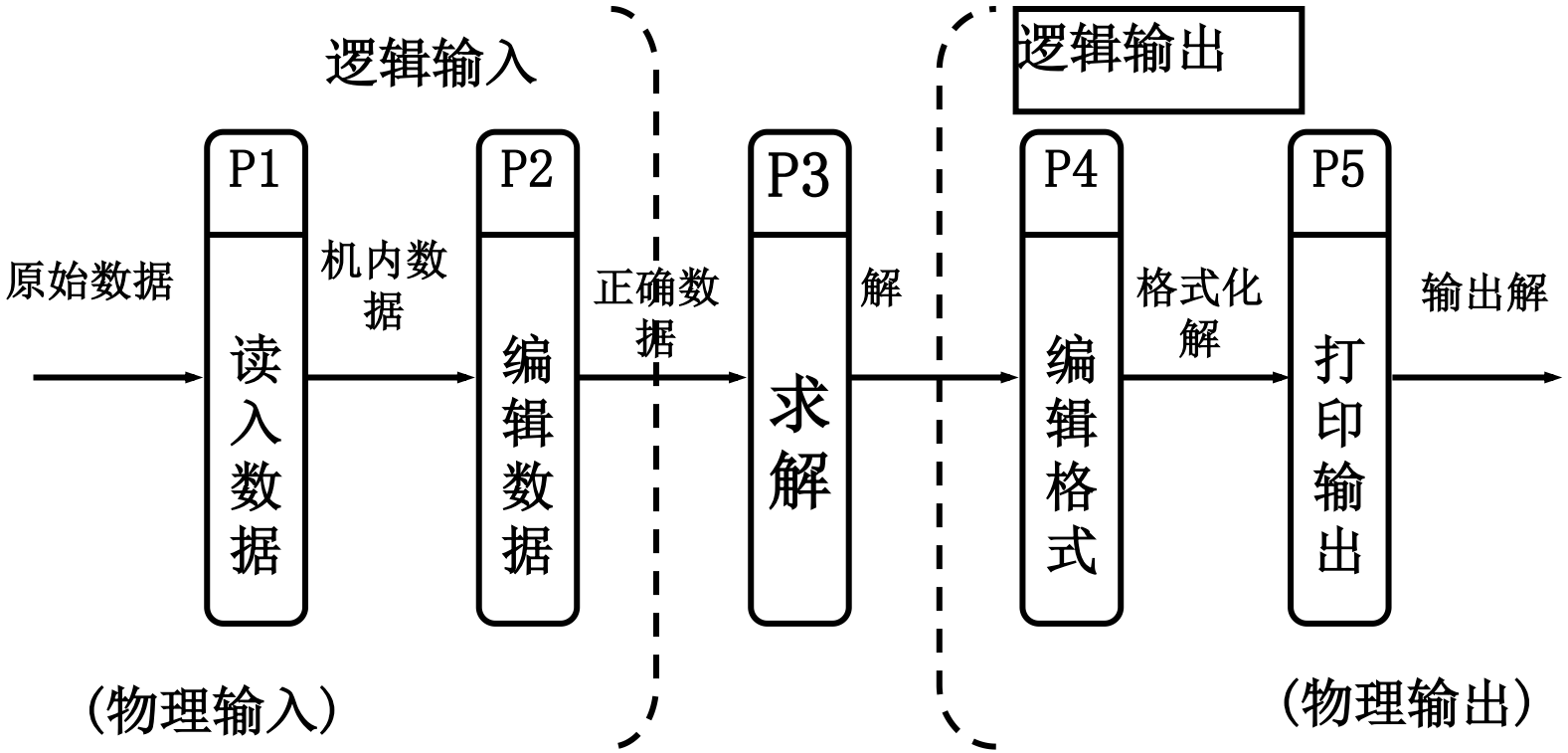
- 相关概念：
  - 物理输入：指系统输入端的数据流
  - 物理输出：指系统输出端的数据流
  - 逻辑输入：指变换中心的输入数据流
  - 逻辑输出：指变换中心的输出数据流
- 划分可能因人而异，但差别不会太大并可通过以后的结构图改进进行调整
- 有时物理输入无须预处理而直接用于系统的加工处理，此时物理输入就是逻辑输入（同样也存在物理输出就是逻辑输出的情况）

确定输入输出流边界和变换中心

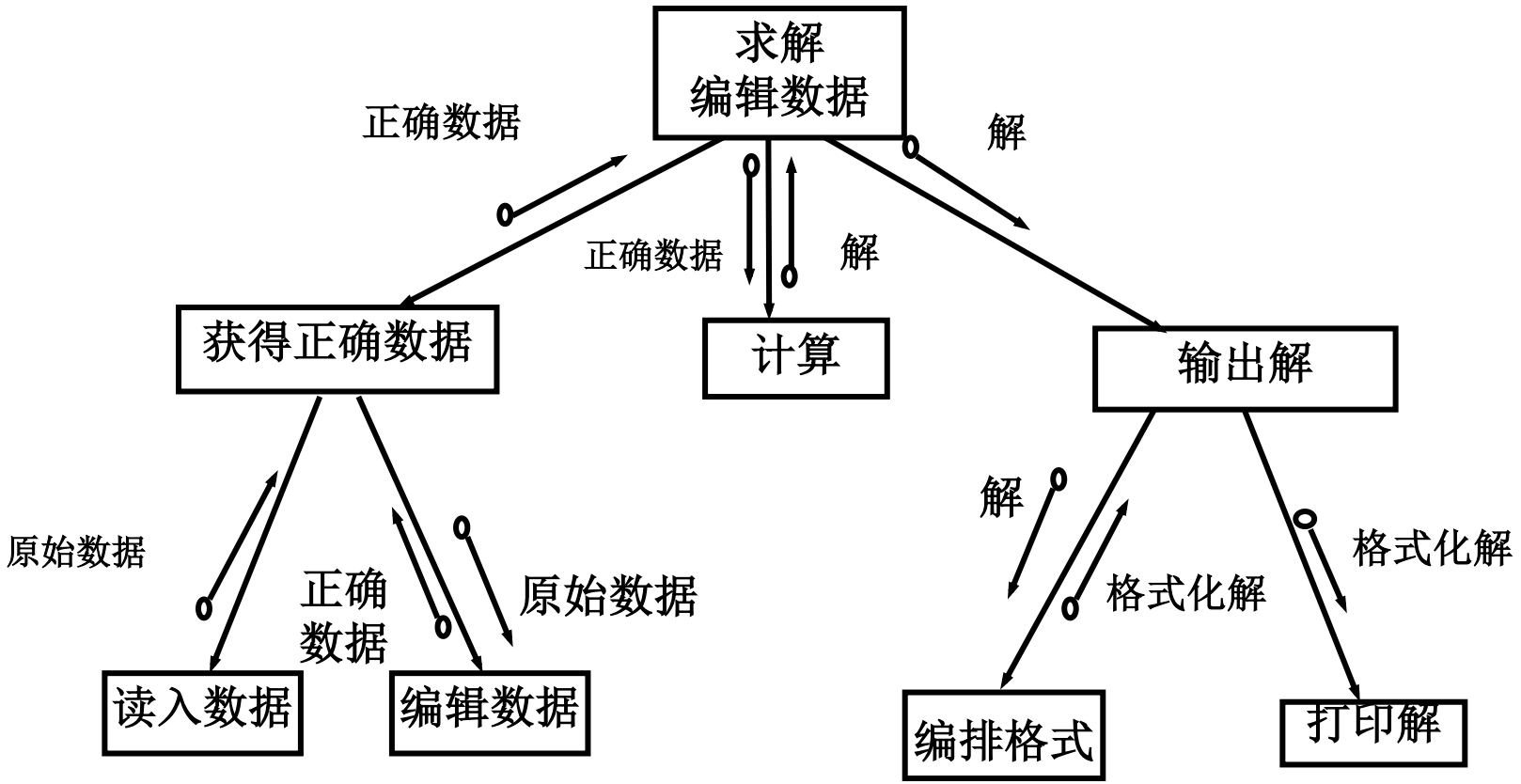
- 基本步骤
  - **确定逻辑输入**：根据DFD从物理输入端开始，一步步向系统的中间移动，可找到离物理输入端最远的，但仍可被看作系统输入数据流，这就是逻辑输入
  - **确定逻辑输出**：根据DFD从物理输出端开始，一步步向系统的中间移动，可找到离物理输出端最远的，但仍可被看作系统输出的数据流，这就是逻辑输出
  - **确定变换中心**：确定了所有的逻辑输入和逻辑输出后，位于逻辑输入和逻辑输出之间的部分就是变换中心



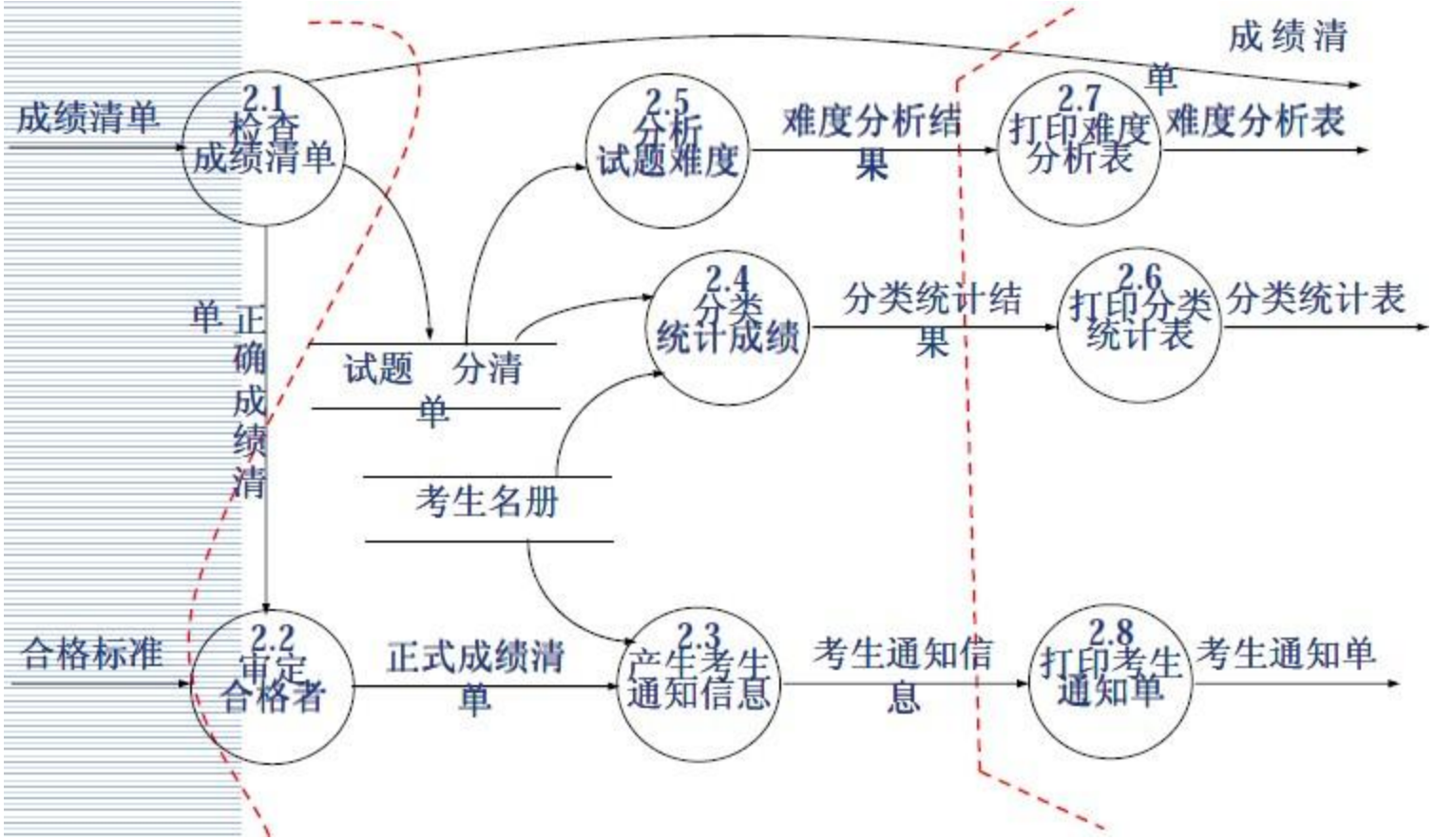
第1步：划分输入、加工、输出



第2步：构造第1、2层模块

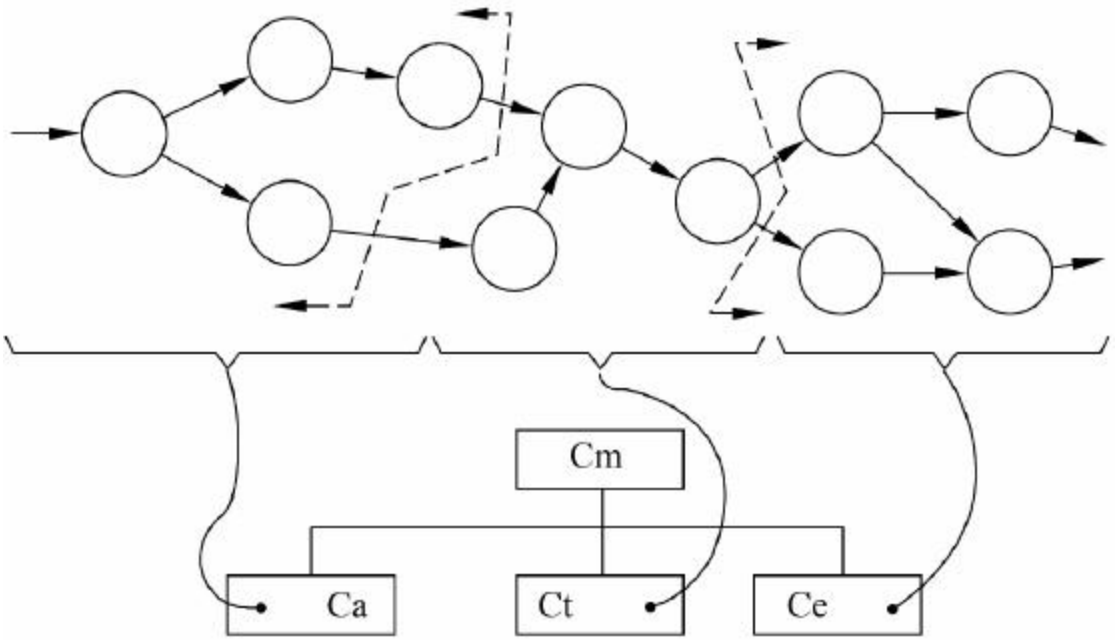


示例：统计成绩子图的输入、输出流边界

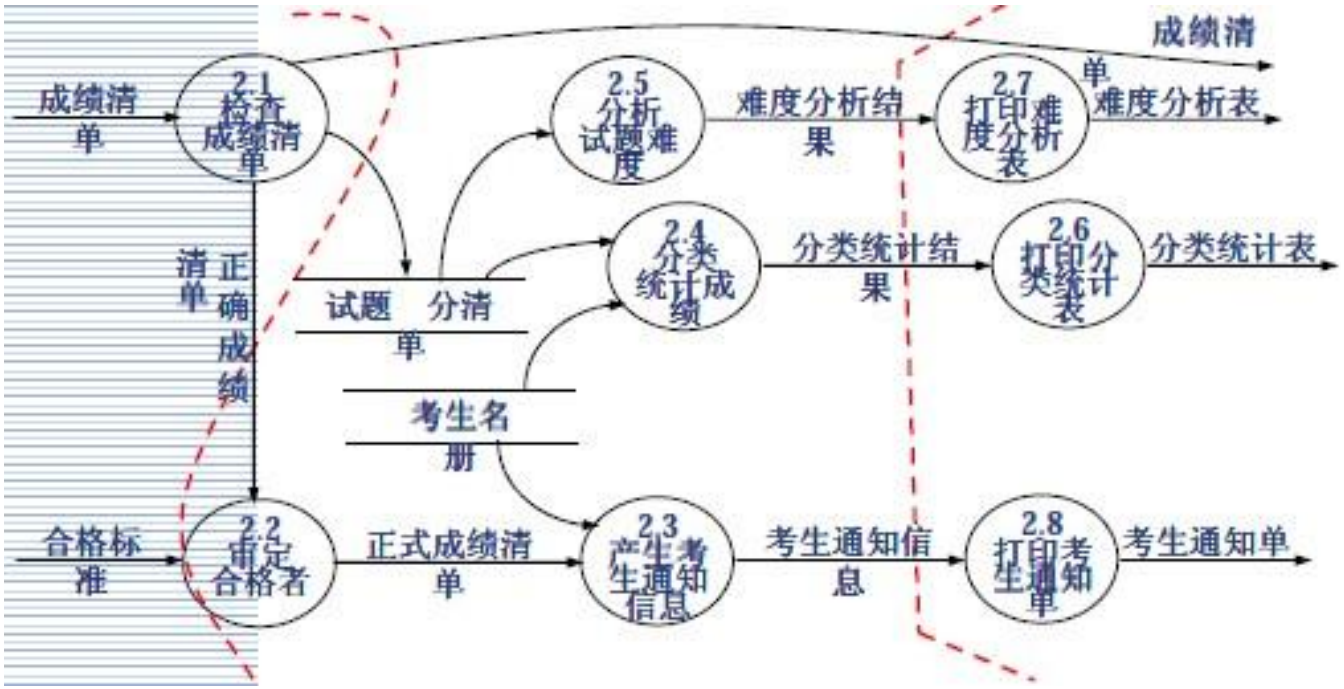


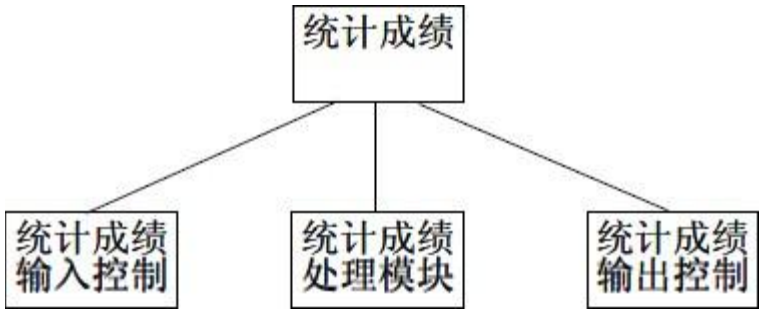
进行第一级分解

- 将DFD映射成变换型程序结构：**输入控制，变换控制，输出控制**
- 大型的软件系统第一级分解时可多分解几个模块，以减少最终结构图的层次数



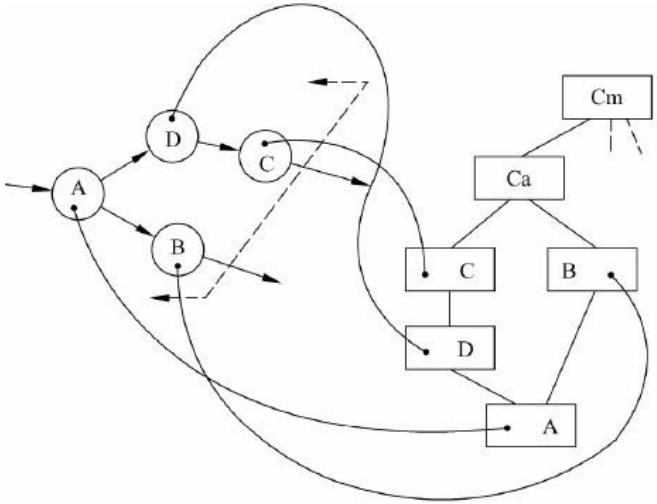
示例：统计成绩子图的第一级分解



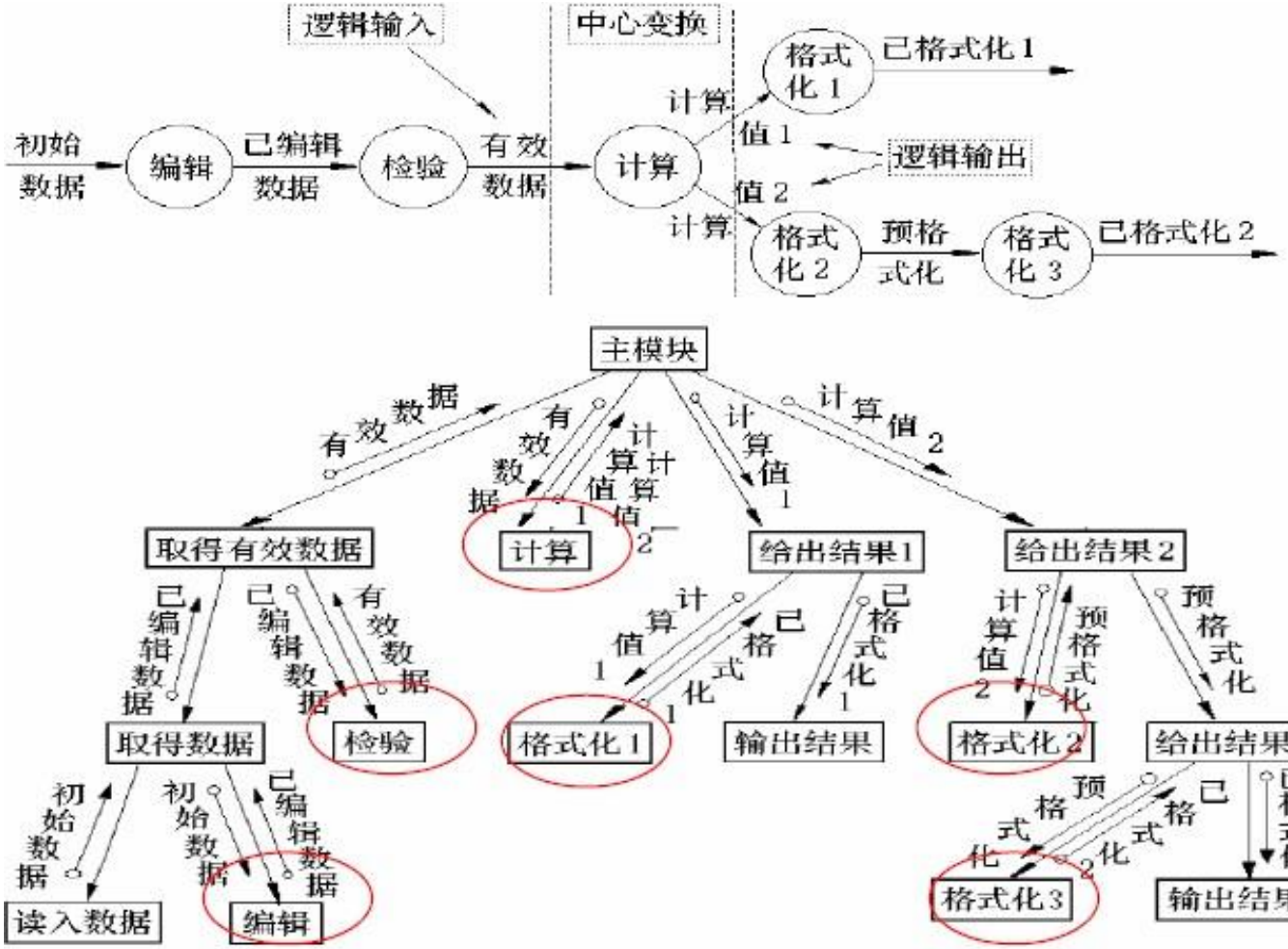


进行第二级分解

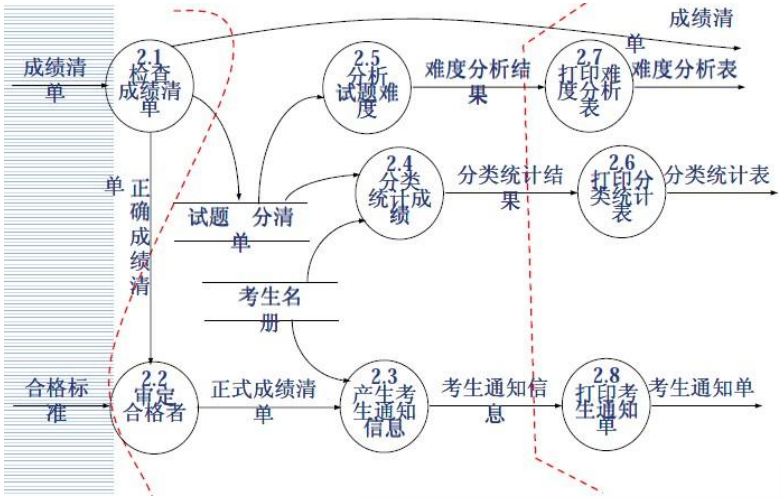
- 自顶向下，逐层细化，设计第一层模块的从属模块
  - **输入控制模块的分解**：从变换中心的边界开始，沿着输入路径向外移动，把输入路径上的每个加工映射成结构图中受输入控制模块控制的一个低层模块
  - **输出控制模块的分解**：从变换中心的边界开始，沿着输出路径向外移动，把输出路径上的每个加工映射成结构图中受输出控制模块控制的一个低层模块
  - **变换控制模块的分解**：把变换中心的每个加工映射成结构图中受变换控制模块控制的一个低层模块

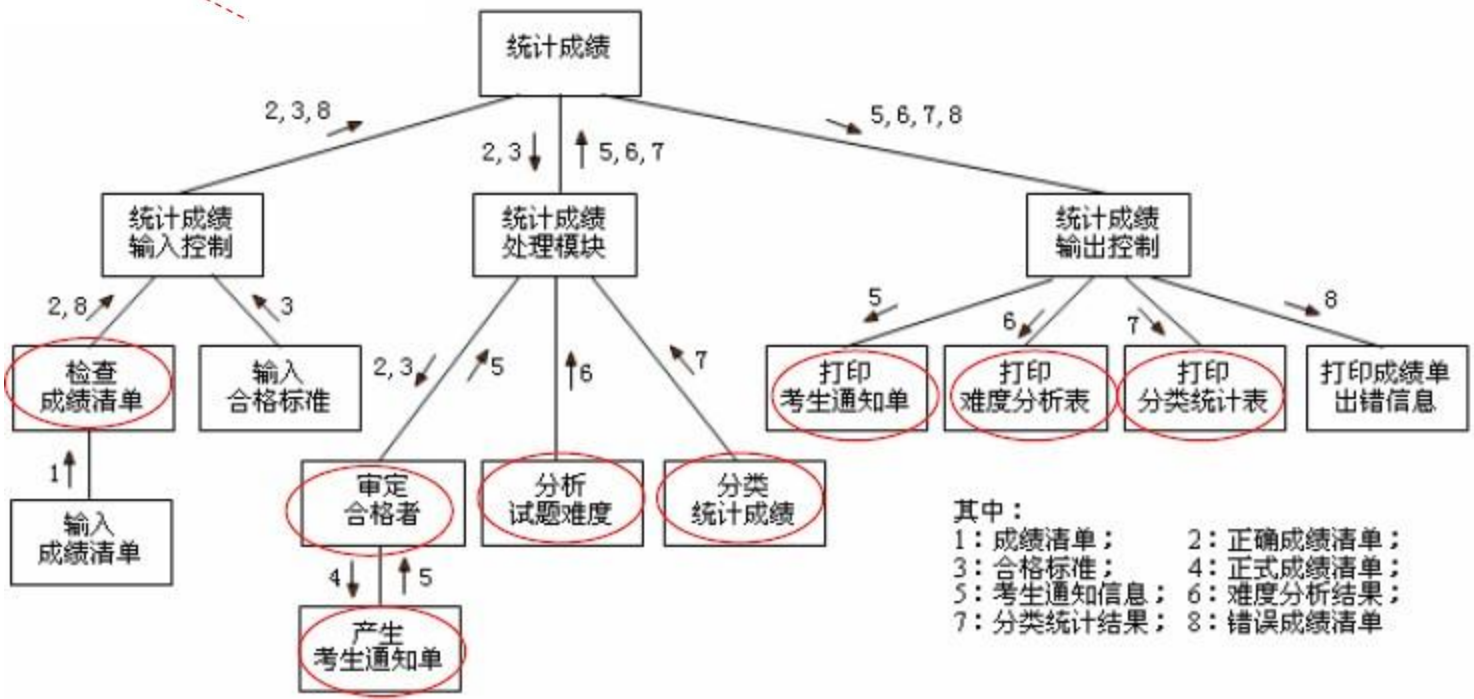


示例：统计成绩子图第二级分解



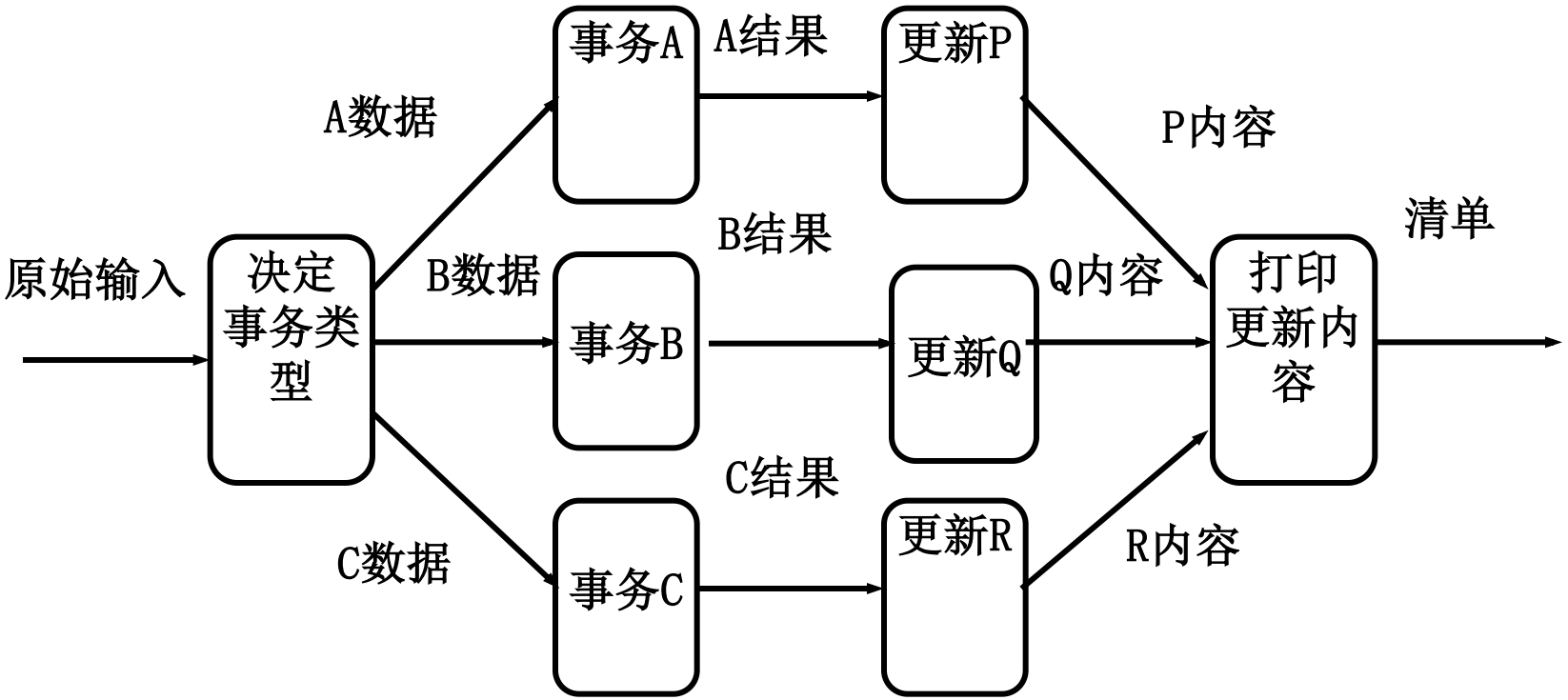
示例：统计成绩子图第二级分解





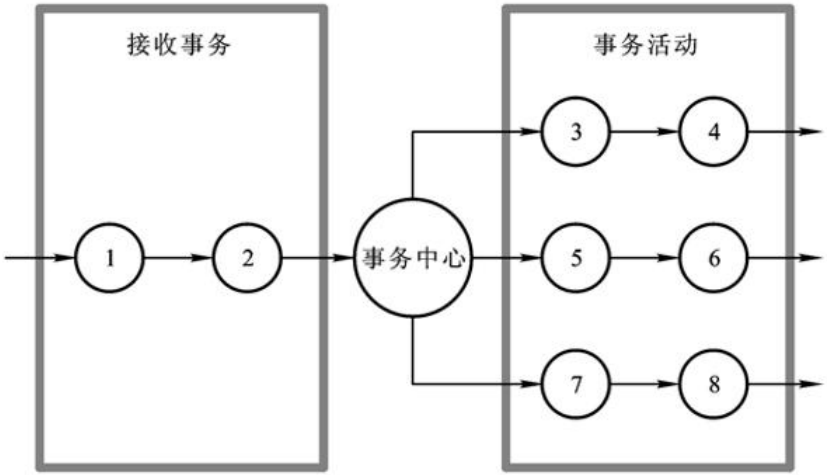
事务分析

对并行结构的DFD进行分析，如：根据输入数据判断业务类型，不同业务的具体处理过程有区别。



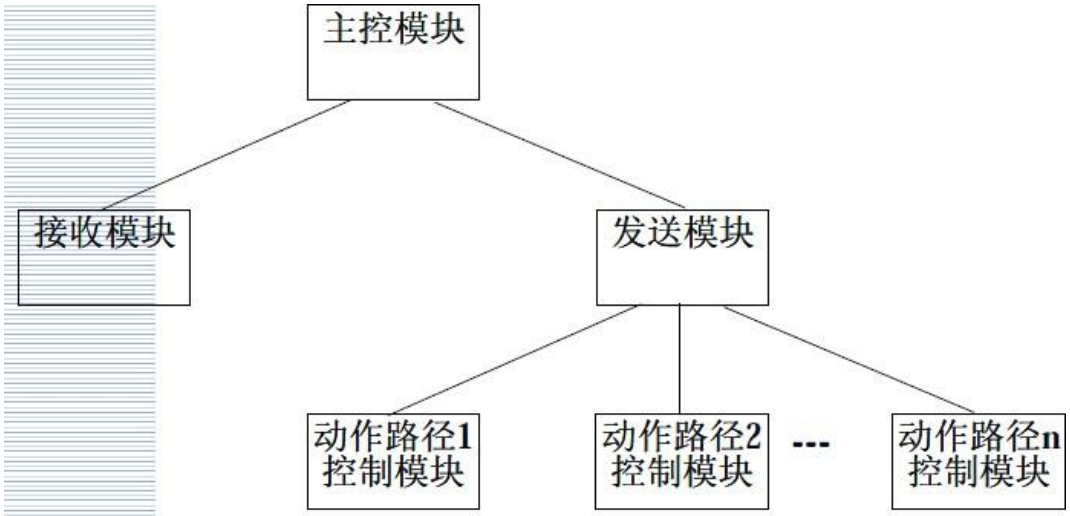
事务分析

- 特征：数据流沿着输入路径到达一个事务中心，事务中心根据输入数据的类型在若干条活动通路中选择一条来执行
- 事务中心的任务：**决策**
  - 接收输入数据（即事务）
  - 分析每个事务的类型
  - 根据事务类型选择执行一条活动通路



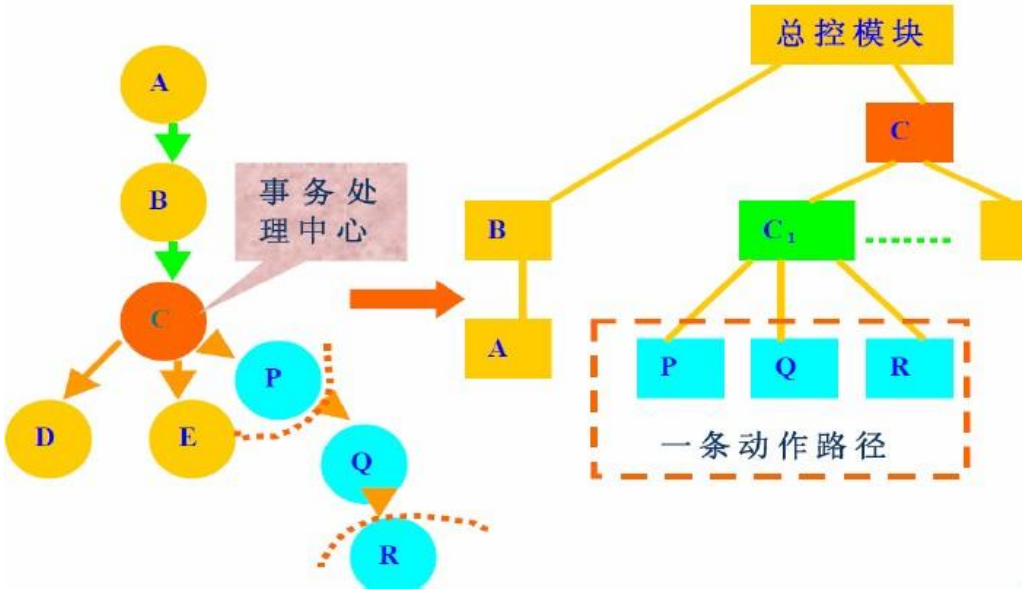
事务分析

- 将事务型DFD映射成初始的结构图
  - 银行业务中有存款、取款、查询余额、开户、转帐等多种事务，这种软件通常是接收一个事务，然后根据事务的类型执行一个处理功能
- 事务型的结构图如下图所示，包括：
  - **主控模块**：完成整个系统的功能
  - **接收模块**：接收输入数据（事务）
  - **发送模块**：根据输入事务的类型，选择一个动作路径控制模块
  - **动作路径控制模块**：完成相应的动作路径所执行的子功能

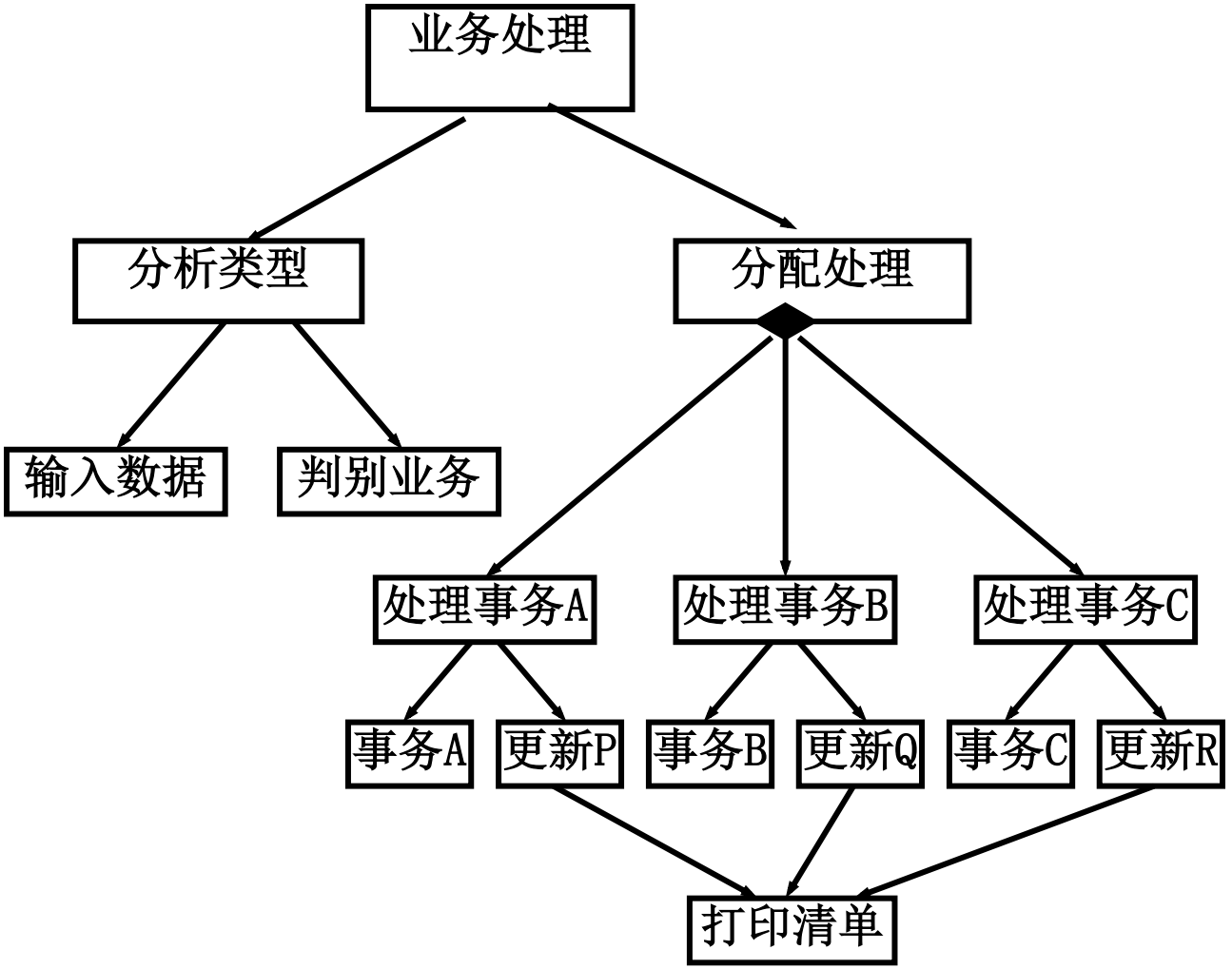


事务分析的步骤

- **确定事务中心**
  - 事务中心位于动作路径的起点，这些动作路径呈幅射状从该点流出
- **分解动作路径对应的结构图**
  - 接收模块的分解：从事务中心开始，沿着输入路径向外移动，把输入路径上的每个加工映射成结构图中受接收模块控制的一个低层模块
  - 动作路径控制模块的分解：首先确定每条动作路径的流类型（变换流或事务流）；然后运用变换分析或事务分析，将每条动作路径映射成与其流特性相对应的、以动作路径控制模块为根模块的结构图

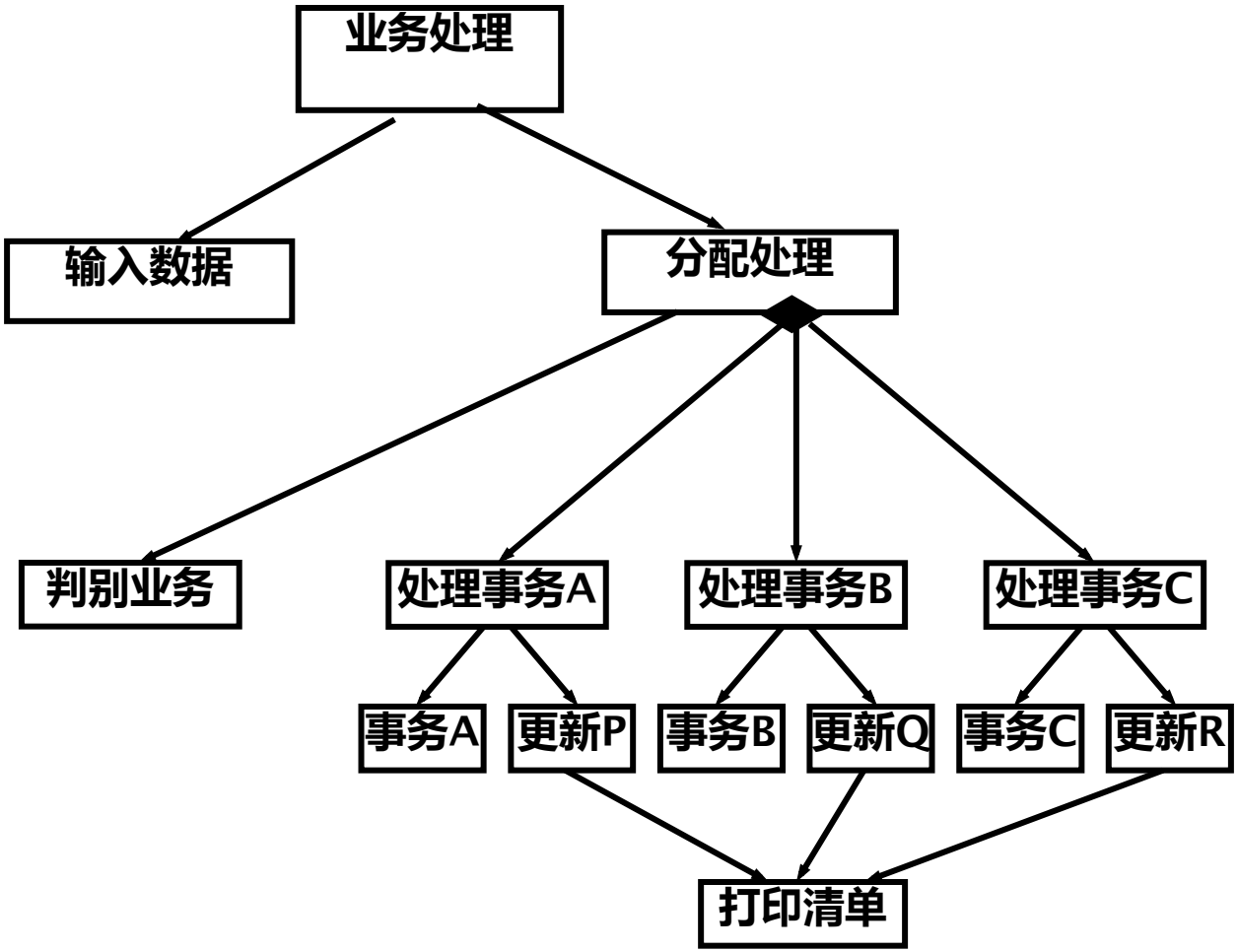


事务分析结果


业务处理
分析类型
分配处理
输入数据
判别业务
处理事务A
处理事务B
处理事务C
事务A
更新P
事务B
更新Q
事务C
更新R
打印清单

事务分析结果

上图存在控制耦合，消除后：



初始结构图的改进

- 对结构图改进的依据是观察这种改进是否符合软件设计的准则和启发式设计的策略
- 设计改进往往伴随着折衷
  - 例：提取多个模块中的相同功能可以提高模块独立性和复用性，但会增加模块间的联系
- 改进往往不是一次完成的，需要进行多次的反复， 有时还需要在多个改进方案进行选择

启发式原则

- 消除重复功能，改善软件结构
- 深度、宽度、扇出和扇入应适中
- 模块的影响范围应限制在该模块的控制范围内
- 减少模块间的耦合度
- 模块功能应该可以预测
- 模块大小适中

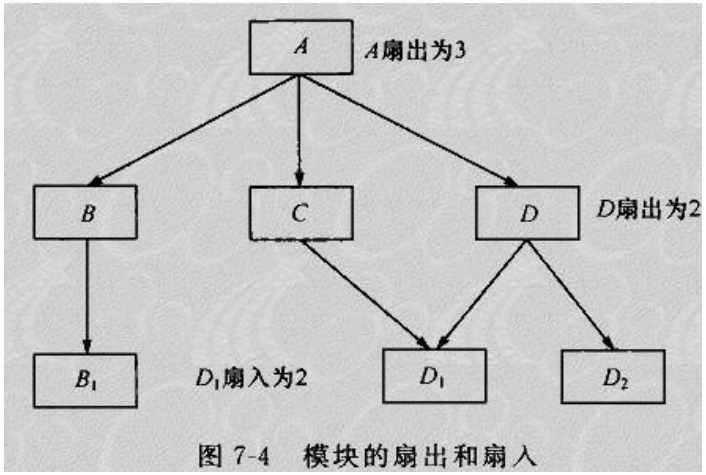
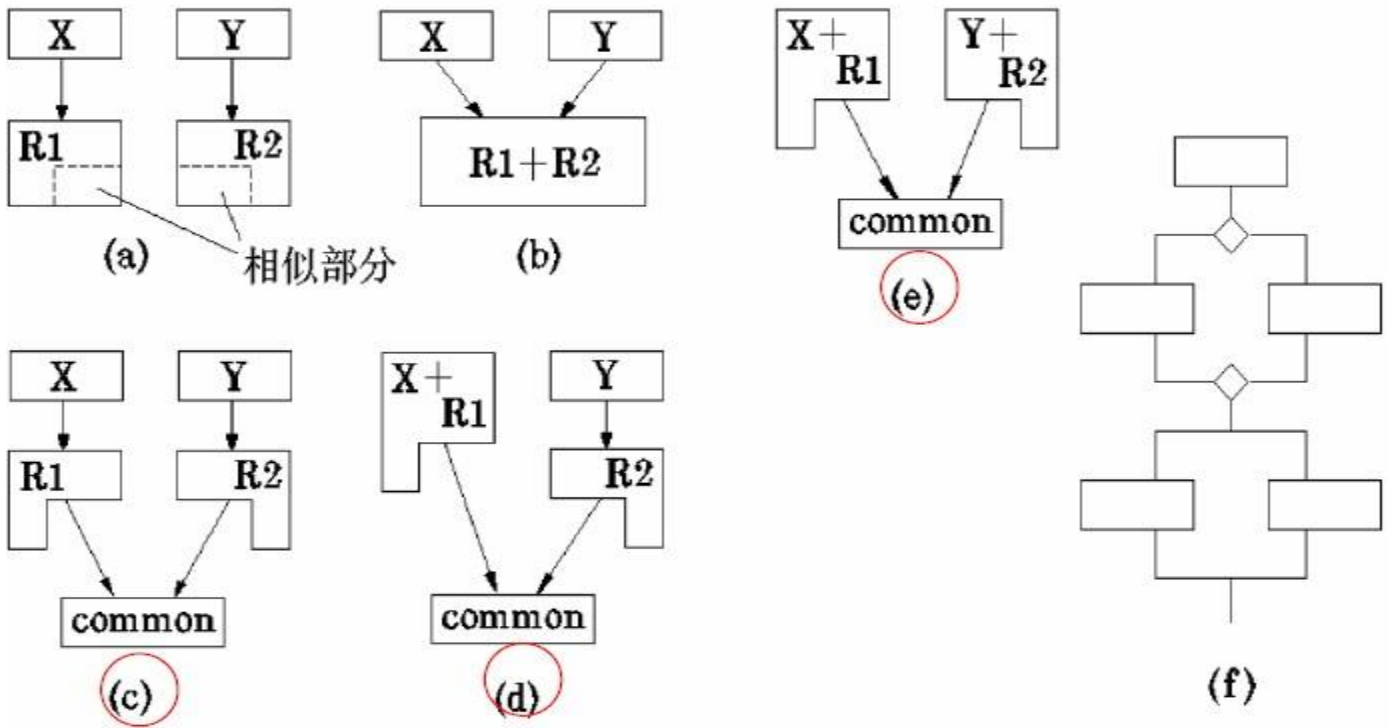


图 7-4　模块的扇出和扇入

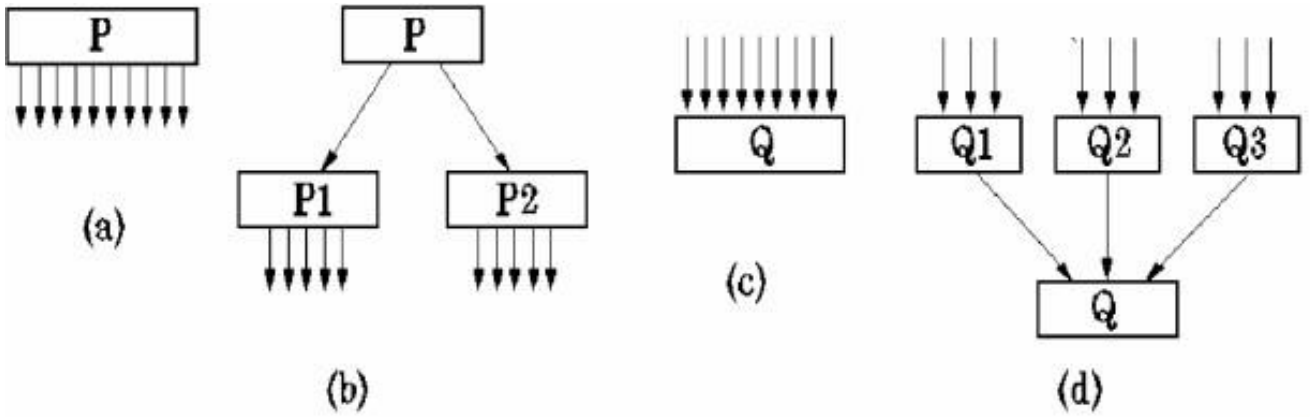
消除重复功能，改善软件结构

- 完全相似
  - 结构上完全相似，数据类型不一致，可采用完全合并的方法
- 局部相似
  - 找出相同部分分离出去，定义成一个独立的下一层模块
  - 也可与上级模块合并



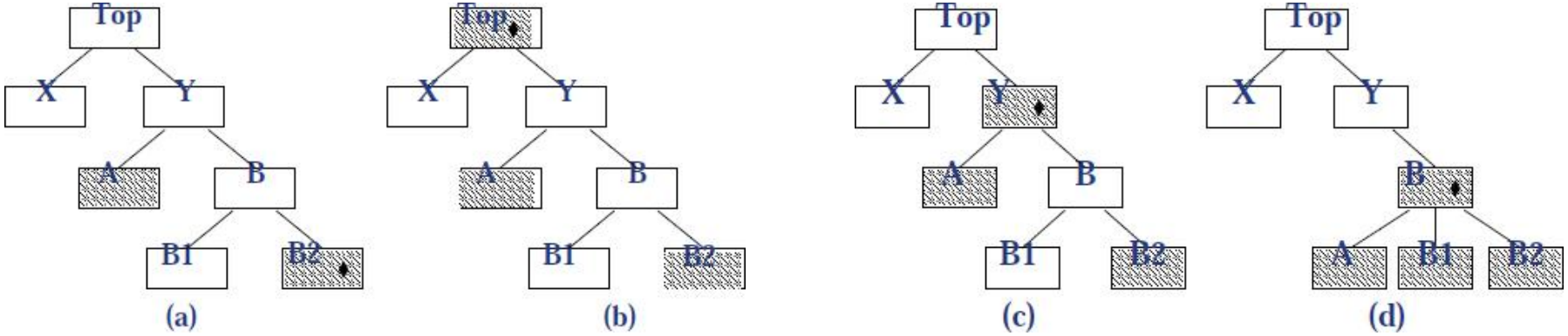
深度、宽度、扇出和扇入应适中

- 若一个模块扇出数过大，则该模块过分复杂，需要协调和控制过多的下属模块，适当增加中间层次的控制模块
- 若一个模块扇入数过大，而它又不是公用模块，则该模块可能具有多个功能，需进一步功能分解

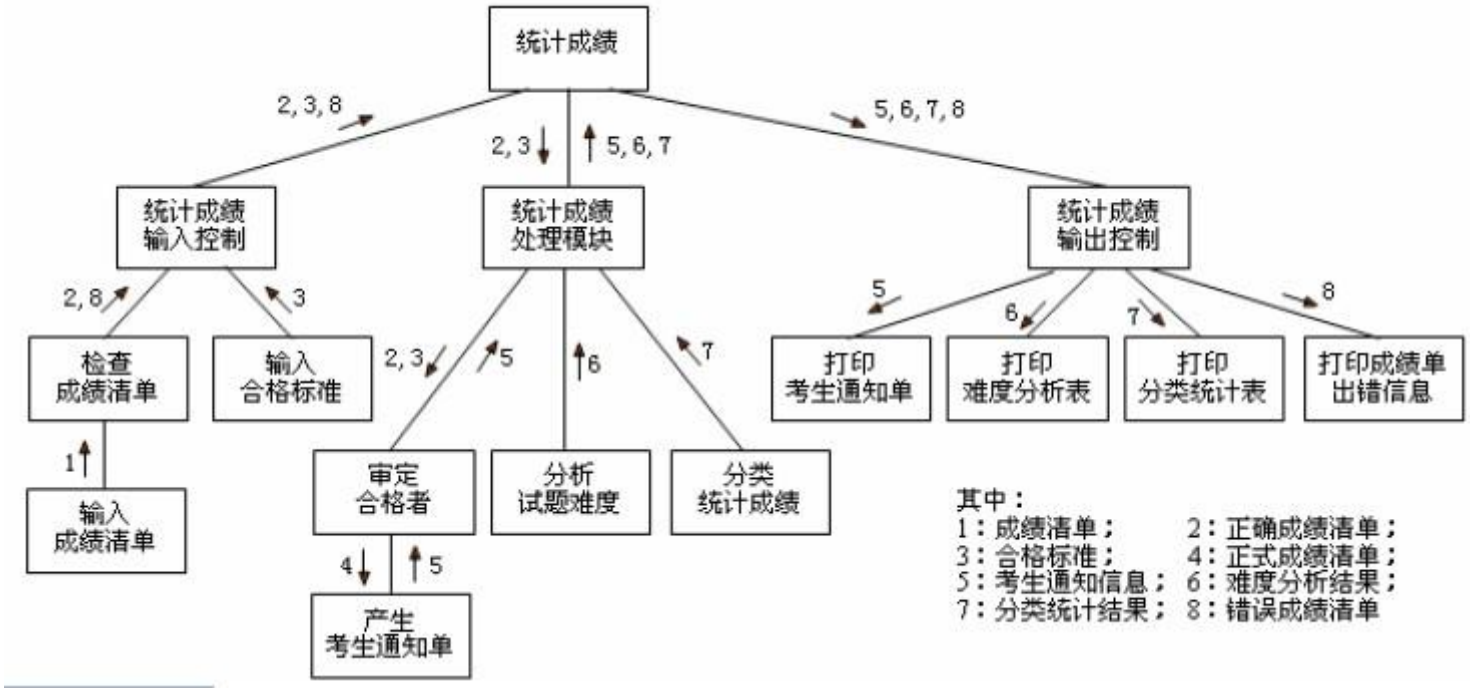


模块的影响范围应限制在模块的控制范围内

- 图a中，模块B2的影响范围（模块A）不在其控制范围（模块B2）内，增加数据传输和耦合
- 图b中，决策控制在顶层模块，其影响范围（A、B2）在控制范围内，但从控制模块到被控模块之间相差多个层次
- 图c和图d较合适，图d最好

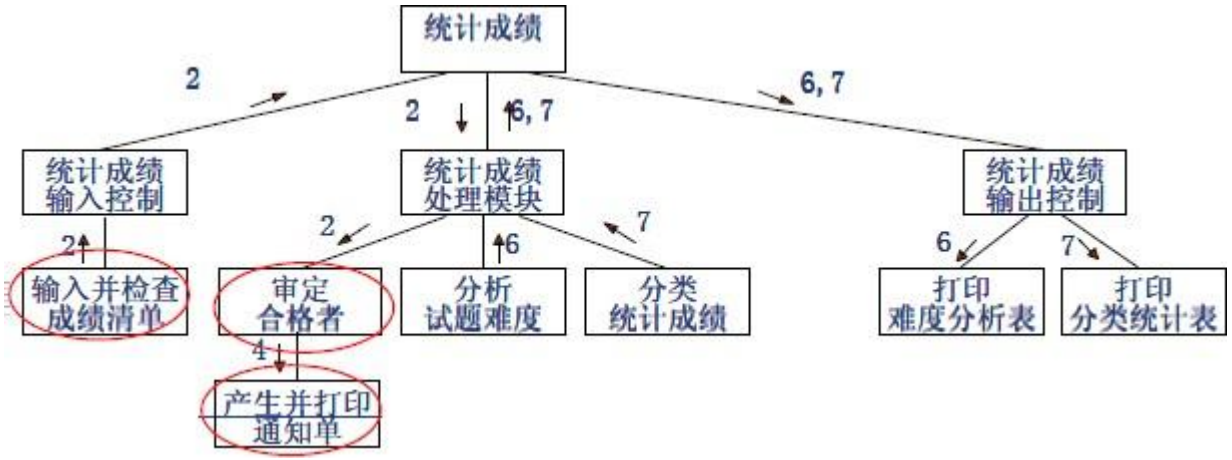


实例：统计成绩结构图


统计成绩
2, 3, 8
2, 3
5, 6, 7
5, 6, 7, 8
统计成绩
输入控制
统计成绩
处理模块
统计成绩
输出控制
2, 8
3
5
6
7
8
检查
成绩清单
输入
合格标准
2, 3
5
6
7
打印
考生通知单
打印
难度分析表
打印
分类统计表
打印成绩单
出错信息
1
输入
成绩清单
审定
合格者
分析
试题难度
分类
统计成绩
4
5
产生
考生通知单
其中：
1：成绩清单；
2：正确成绩清单；
3：合格标准；
4：正式成绩清单；
5：考生通知信息；
6：难度分析结果；
7：分类统计结果；
8：错误成绩清单

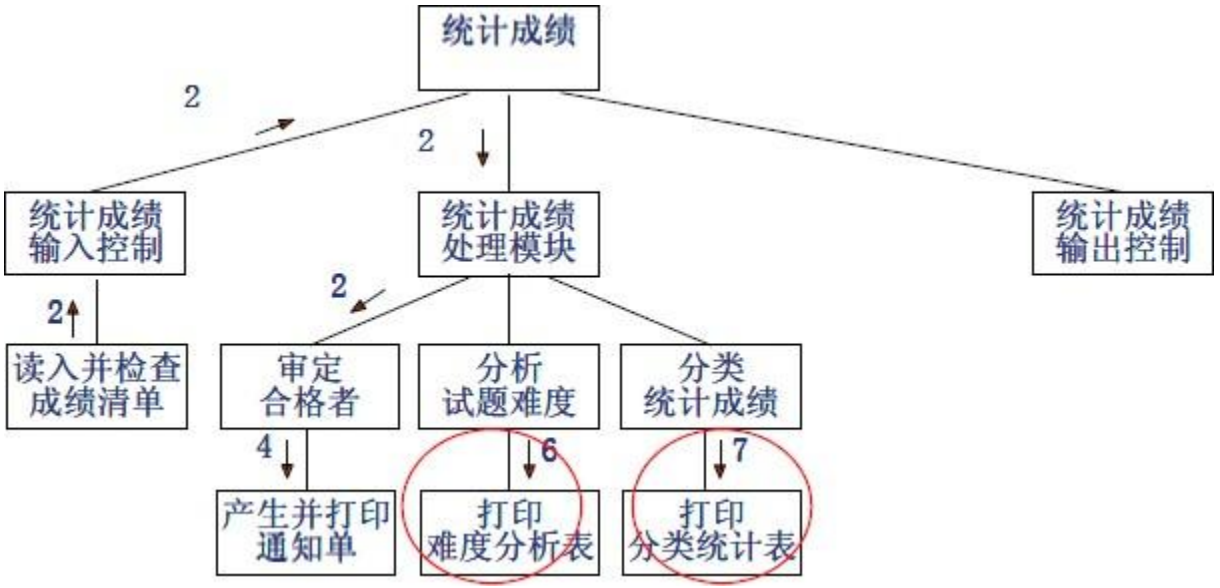
实例：统计成绩结构图的改进

- 先将一些比较简单的模块合并到与其功能相一致的模块中，以减少耦合度
  - 将“输入成绩清单”、“检查成绩清单”、“打印成绩单出错信息”合并成“输入并检查成绩清单”
  - 将“输入合格标准”与“审定合格者”合并，仍取名“审定合格者”，但它包含读入合格标准功能
  - 将“产生考生通知单”与“打印考生通知单”合并成“产生并打印考生通知单”



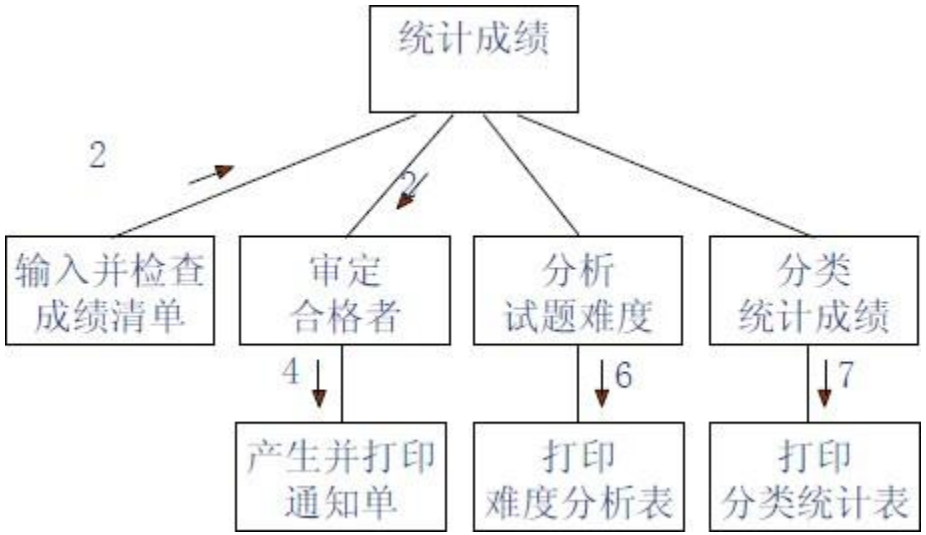
实例：统计成绩结构图的改进

- 降低模块间的耦合程度
  - 将“打印难度分析表”模块和“打印分类统计表”模块分别作为“分析试题难度”模块和“分类统计成绩”模块的下属模块

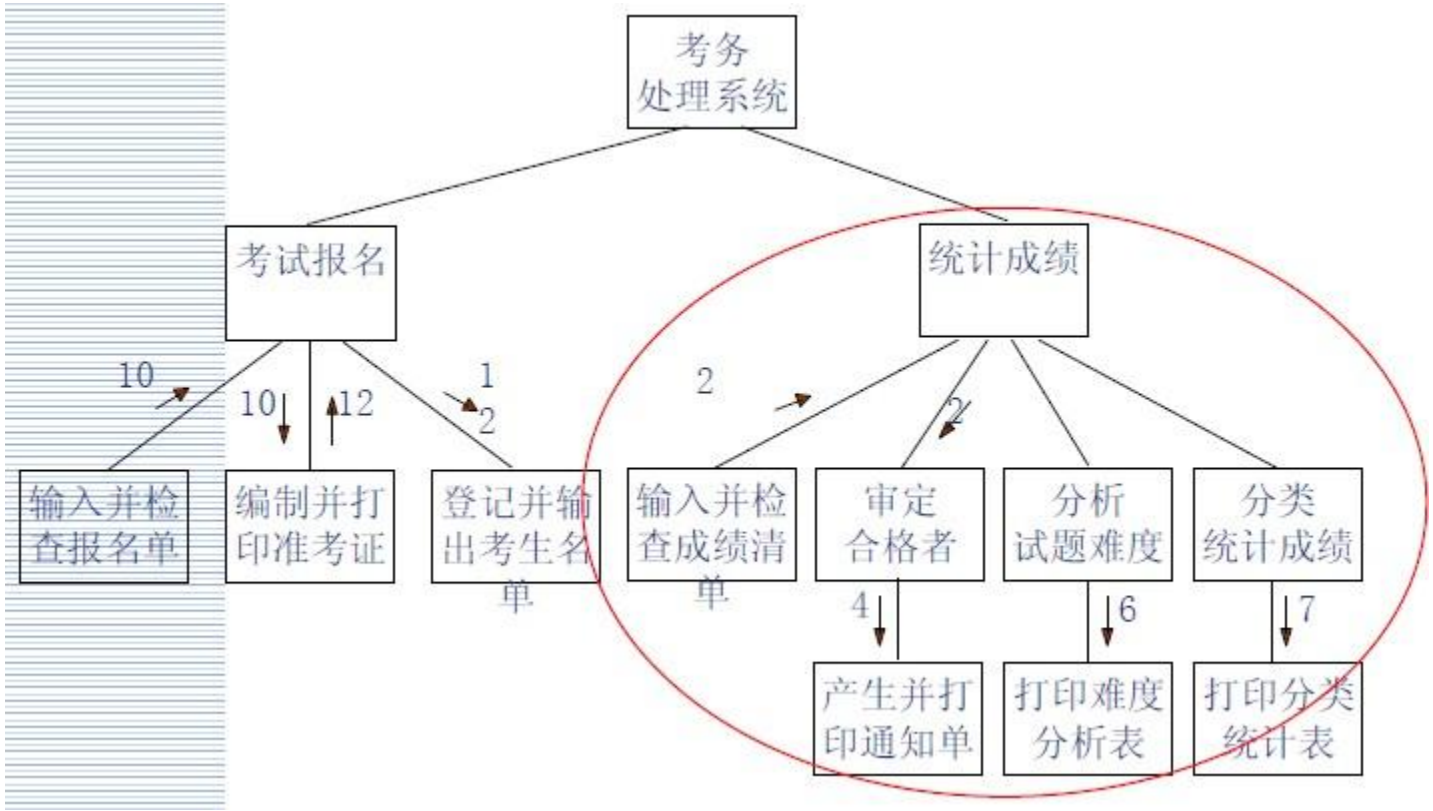


实例：统计成绩结构图的改进

- 删去“统计成绩输出控制”
- “统计成绩输入控制”模块和“统计成绩处理”模块均为管道模块，也可删去

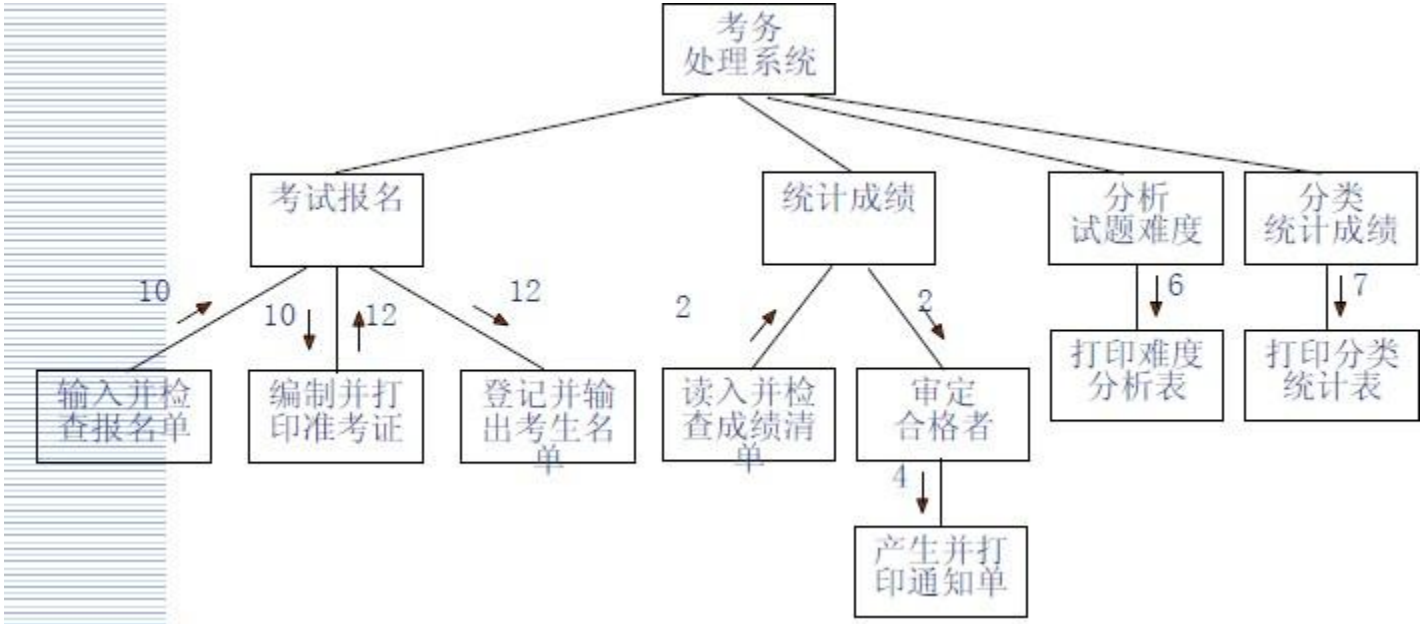


实例：改进后的考务处理系统结构图



实例：考务处理系统结构图的整体改进

- “分析试题难度”和“分类统计成绩”是相对独立的功能，因此将它们移到主控模块下
- “考试报名”模块和“统计成绩”模块似乎是管道模块，但删去后主控模块“考务处理系统”的扇出就比较大，因此可不删除

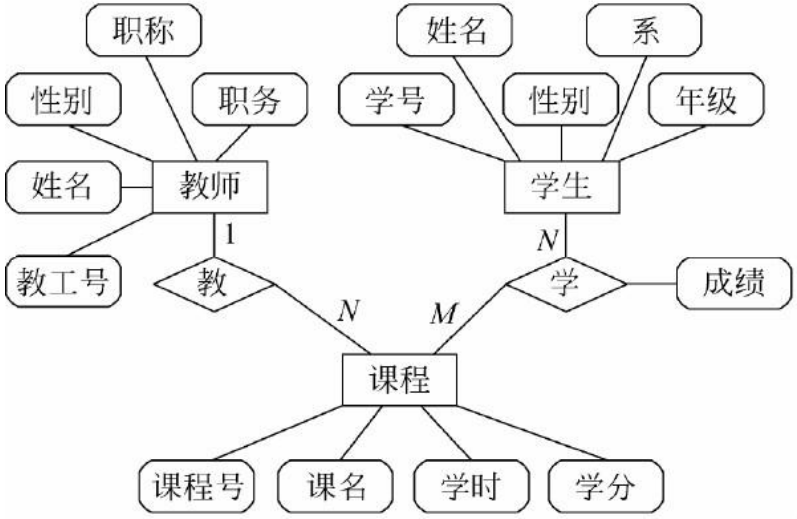


结构图改进技巧

- 减少模块间的耦合度
- 消除重复功能
- 消除管道模块
- 模块的大小适中
- 避免高扇出
- 应尽可能研究整张结构图，而不是只考虑其中的 一部分

数据库设计

- ER映射关系数据库
  - 为每个实体类型建立一张表
  - 为每个表选择一个主键
  - 对1:1关系，把对方主键作为自己的属性
  - 对1:N关系，把1的主键作为N的外键
  - 为每个M:N关系建立一张新表，字段由双方主键和关系本身的属性组成
  - 定义完整性约束
  - 评价并改进模式的质量
  - 选择字段数据类型和值域



关键概念

- 表：关系模型的逻辑结构，呈二维表
- 元组（记录）：表中的一行
- 字段（属性）：表中的一列
- 值域：属性的取值范围
- 主键：某个属性（组），它可以唯一确定一个元组
- 外键：依赖于别的表的数据，维护两个表数据的一致性，往往是另一个表的主键
- 完整性约束：对关系的某种约束条件，如主键不空

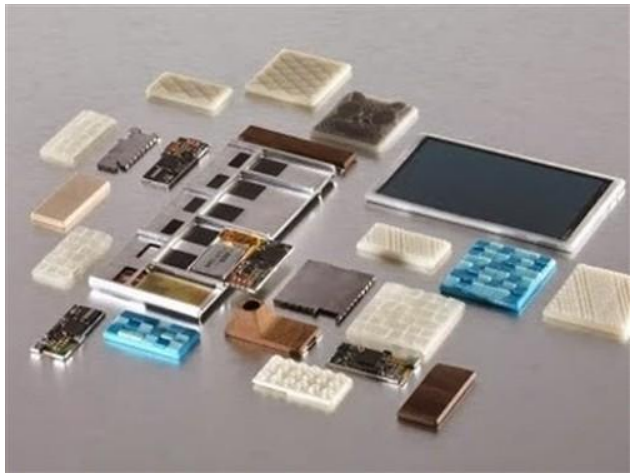
体系结构设计小结

- 结构化方法是一种传统的面向数据流开发方法
- 以数据流为中心构建软件的分析模型和设计模型
- 将分析的结果（DFD）映射成初始的程序结构图，包括变换分析和事务分析，并介绍对初始结构图的优化
- 初步了解ER图到数据库的映射

第16章：结构化程序设计

结构化设计技术

- 在结构化分析和设计方法时，部件往往被称为模块
- 在面向对象分析和设计时，部件被称为类
- 在基于构件的开发方法中，部件被称为构件



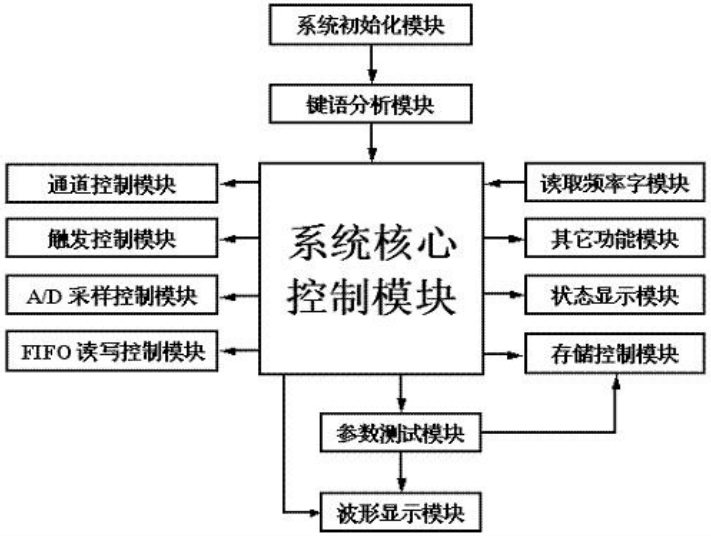


图 5.1 软件程序结构

结构化设计技术

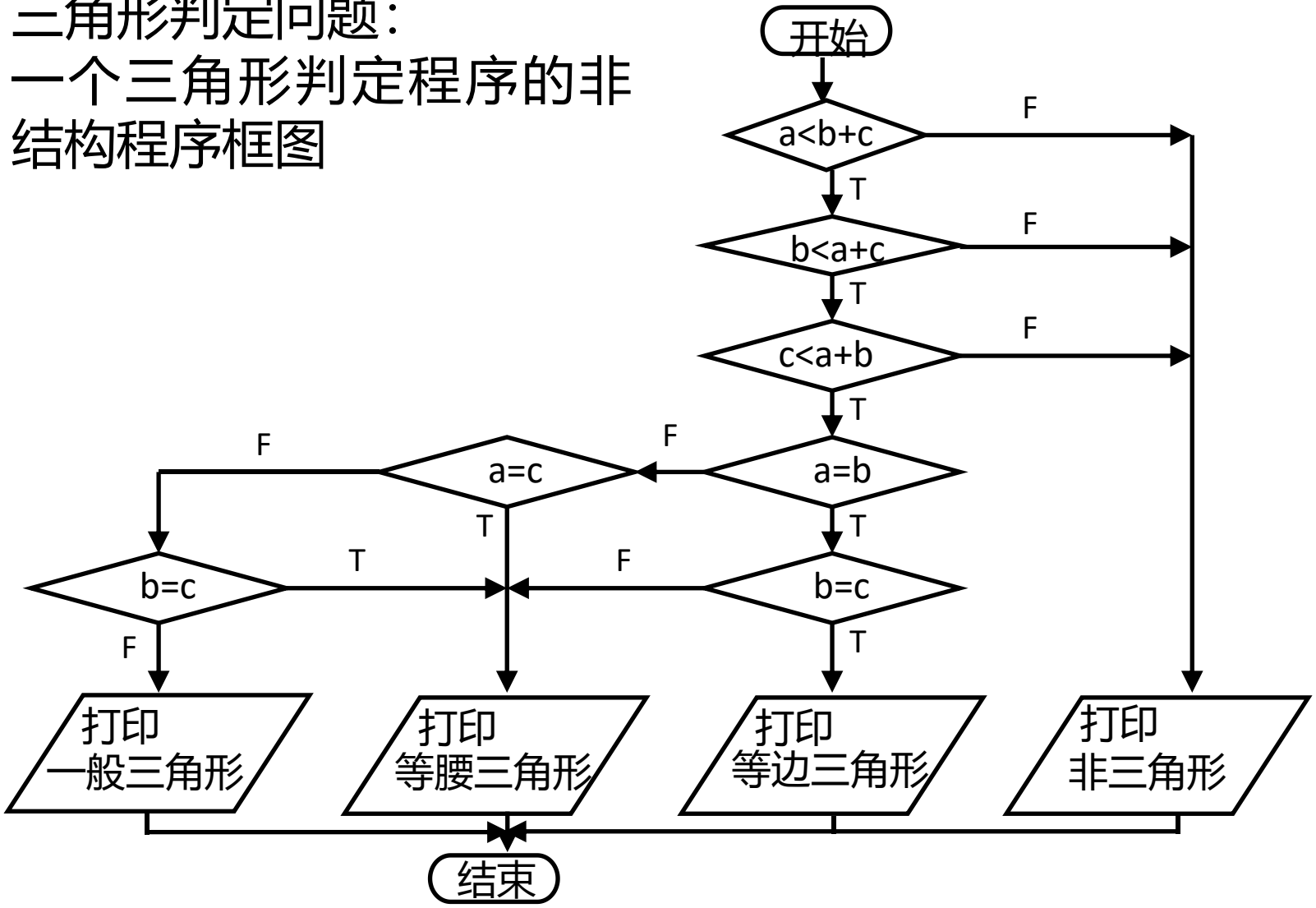
- 在结构化设计阶段，主要完成如下工作：
  1. 为每个部件确定采用的算法，选择某种适当的工具表达算法的过程，编写部件的详细过程性描述
  2. 确定每一部件内部使用的数据结构
  3. 在部件级设计结束时，应该把上述结果写入部件级设计说明书，并且通过复审形成正式文档，作为编码阶段的工作依据
- 结构化设计原则：
  1. 模块的逻辑描述正确可靠、清晰易读
  2. 采用结构化程序设计方法，改善控制结构，降低程序复杂度，提高程序的可读性、可测试性和可维护性

结构化程序设计方法

- 结构化程序设计采用**自顶向下、逐步求精**的设计方法和单入单出的控制结构
- 一种较为流行的定义是：“如果一个程序的代码块仅仅通过顺序、选择和循环这三种基本控制结构进行连结，并且每个代码块只有一个入口和一个出口，则称这个程序是**结构化**的”
- 随着面向对象和软件复用等新的软件开发方法和技术的发展，更现实、更有效的开发途径可能是**自顶向下和自底向上**两种方法有机的结合

非结构程序的例子

- 三角形判定问题:
- 一个三角形判定程序的非结构程序框图

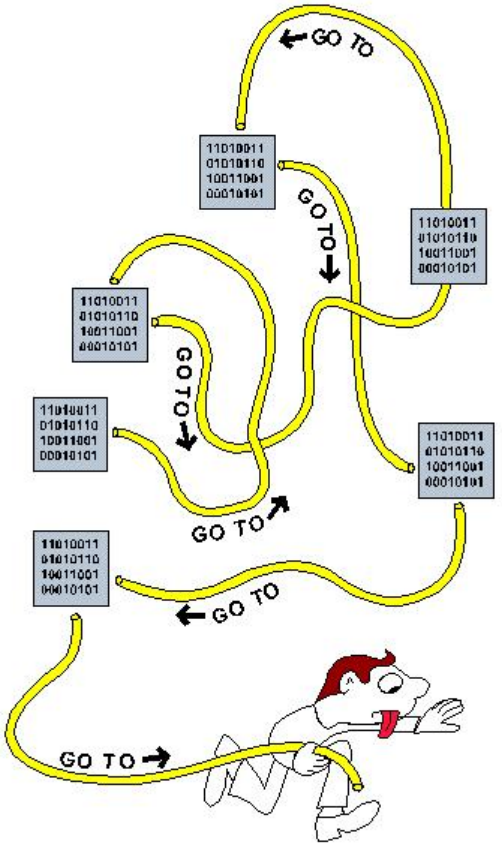


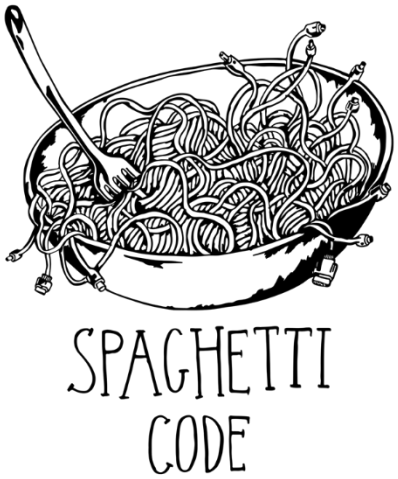
```
#include <stdio.h>
main( )
{
    int  a, b, c;
    scanf("%d%d%d", &a, &b, &c);
    if  (a>=b+c)       goto  notriangle;
    else if  (b>=a+c)     goto  notriangle;
    else if (c>=a+b)      goto  notriangle;
    if  (a==b)
        if  (b==c)     goto  triangle1;
        else        goto  triangle2;
    else
        if  (a==c)      goto  triangle2;
        else
            if  (b==c)     go triamgle2;
            else  {   prontf("一般三角形");   goto  complete; }
notriangle:   printf("非三角形");  goto  complete;
triangle1:    printf("等边三角形");  goto  complete;
triangle2:    printf("等腰三角形");
complete:
}
```

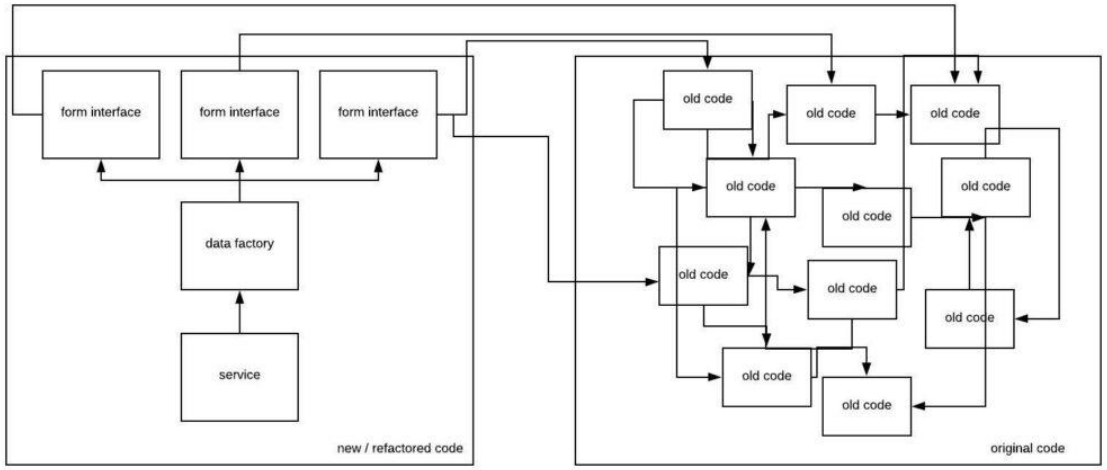
非结构程序的缺点

- 难以设计
- 难以阅读理解
- 难以验证其正确性
- 难以测试
- 难以维护

最终导致生产率低下

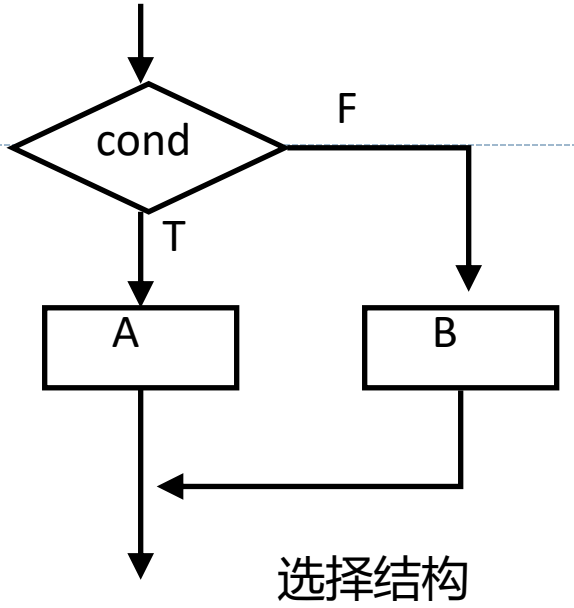




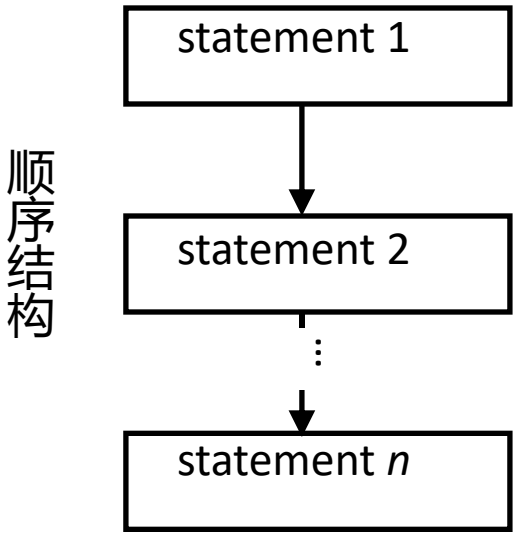


结构程序设计

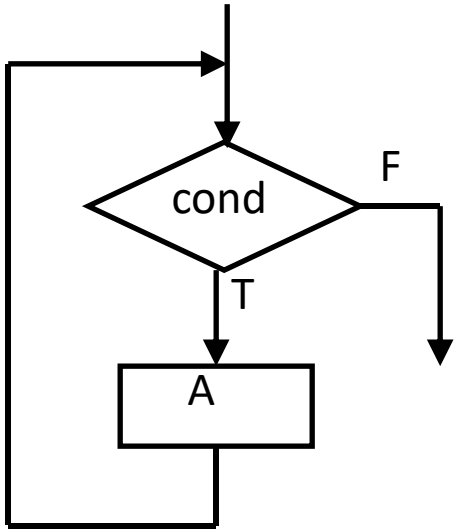
- 结构程序的概念
  - ① 顺序结构
  - ② 选择结构
  - ③ 循环结构



选择结构

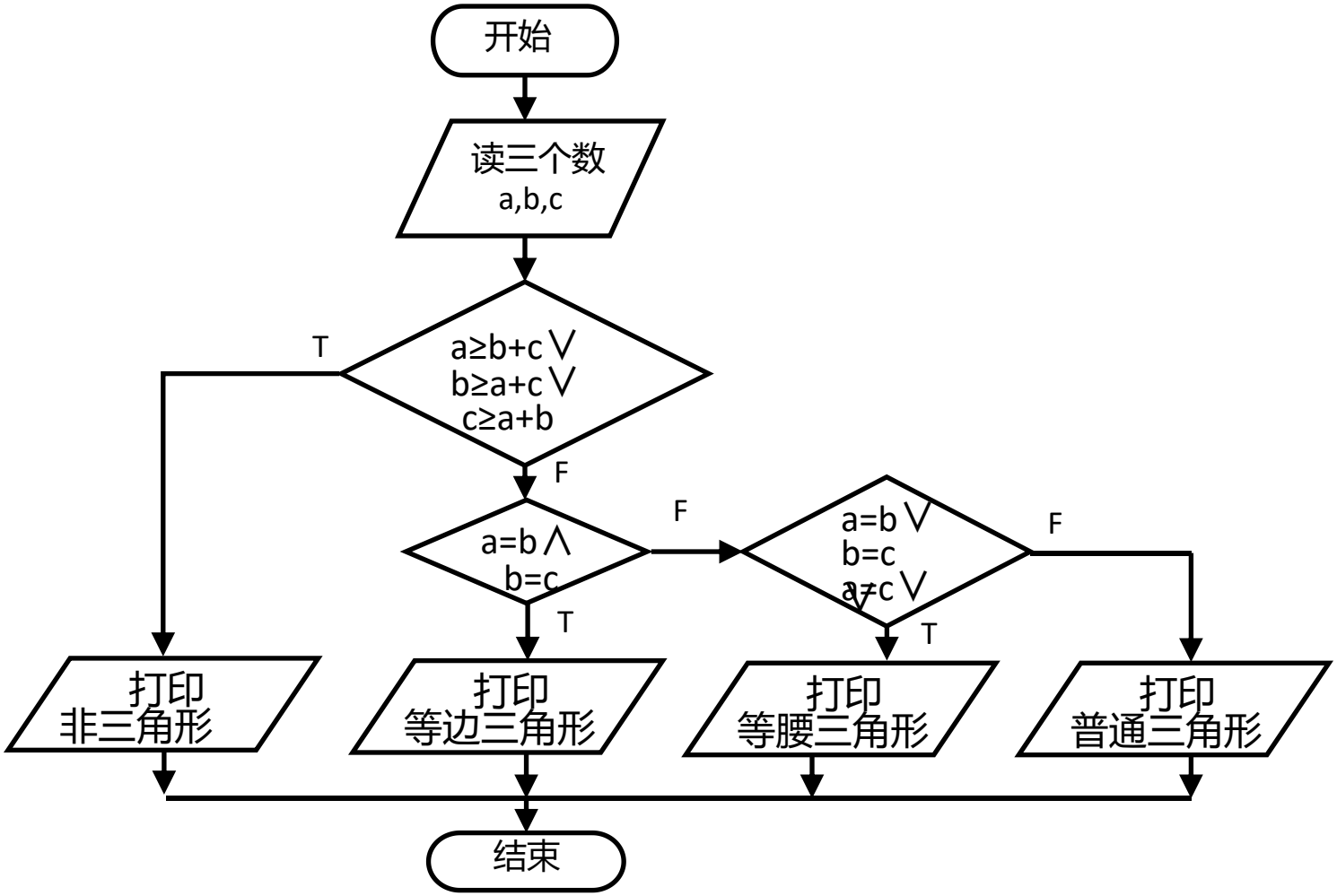


顺序结构



循环结构

三角形判定的结构程序框图



```
#include <stdio.h>
main( )
{
      int a,b,c;
      scanf("%d%d%d", &a, &b, &c);
      if  ((a>=(b+c))||(b>=(a+c))||(c>=(a+b))
           printf("非三角形");
      else if  ((a=b)&&(b=c))
                  printf("等边三角形");
      else if  ((a=b)||(b=c)||(a=c))
                  printf("等腰三角形");
      else
                  printf("普通三角形");
}
```

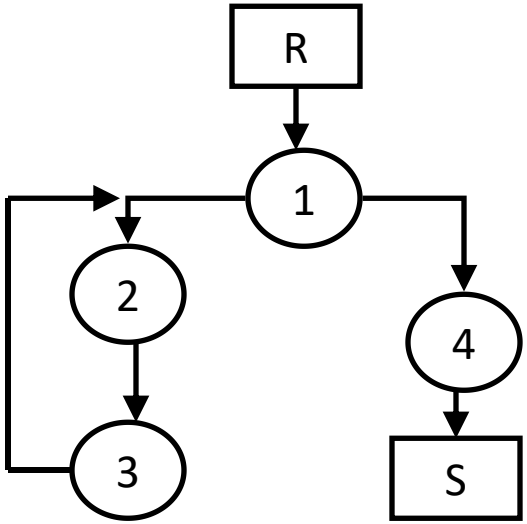
非结构程序转换为结构程序

➢ 结构程序定理

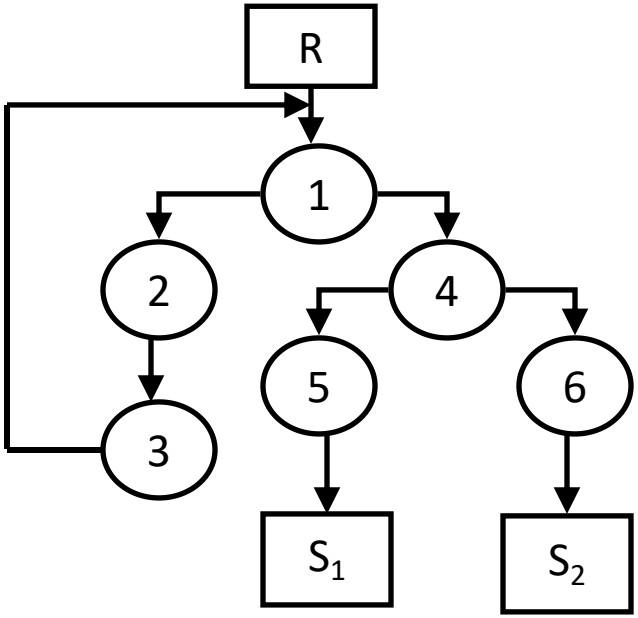
①好形程序: 有一个输入, 每个结点都有从输入经该结点到输出的途径, 可以有多个出口

②病形程序:

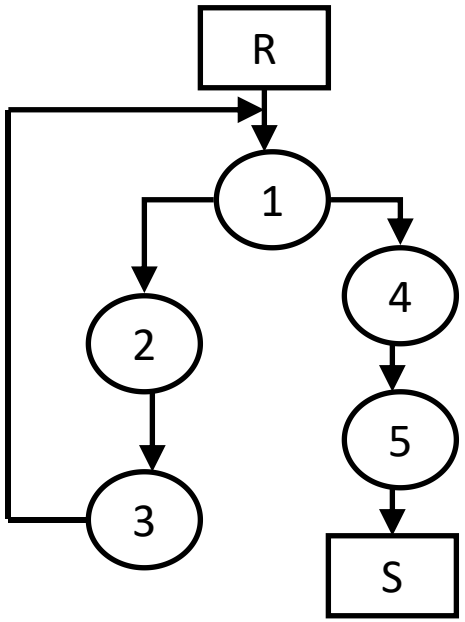
③适形程序: 只有一个出口的好形程序



病形程序



好形程序

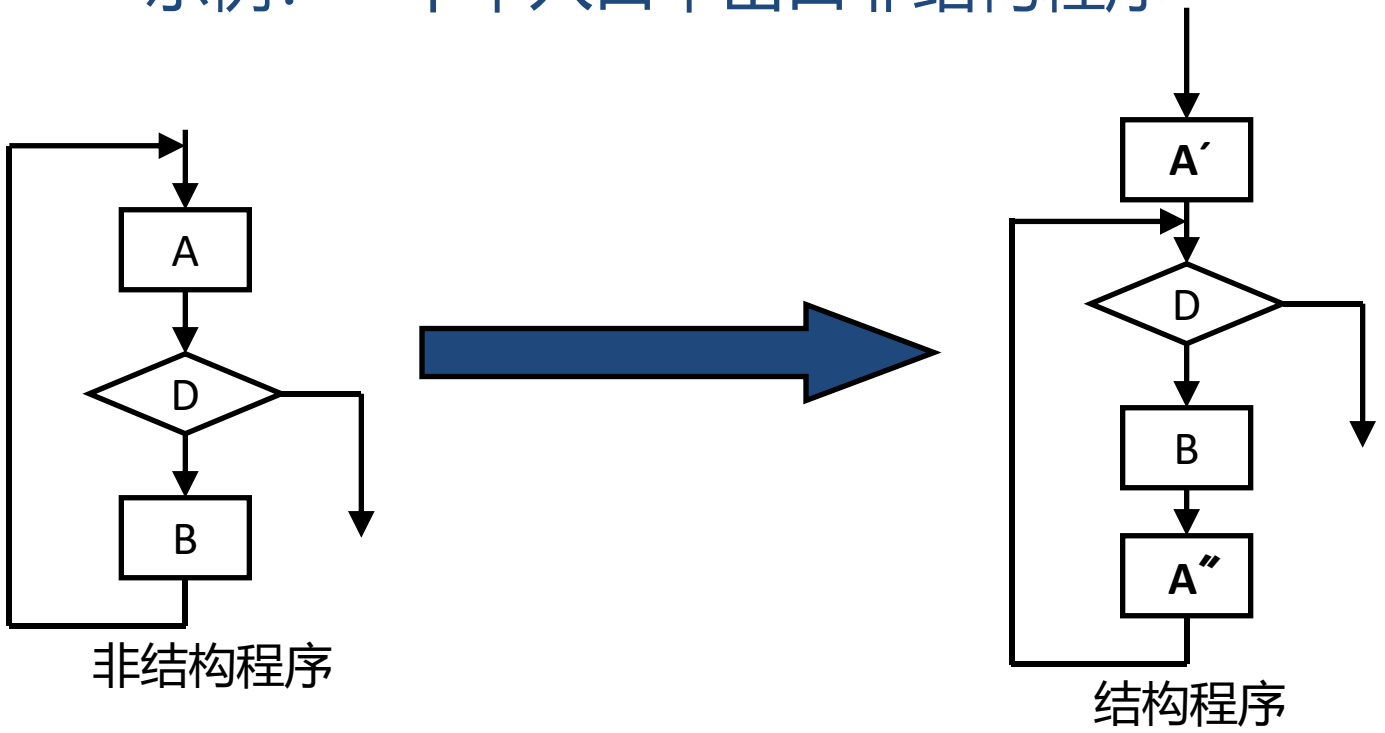


适形程序

非结构程序转换为结构程序

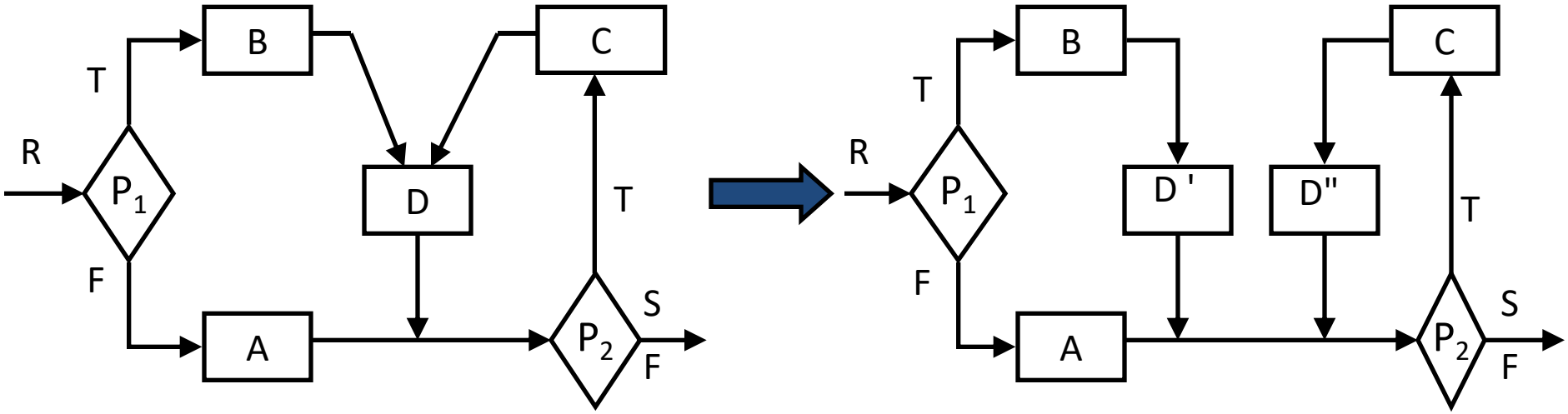
- 结构程序定理：任何一个适形程序都可以转换为一个等价的结构程序，使用原来功能框和判定框及赋值和测试一个附加的变量。

示例：一个单入口单出口非结构程序



非结构程序转换为结构程序

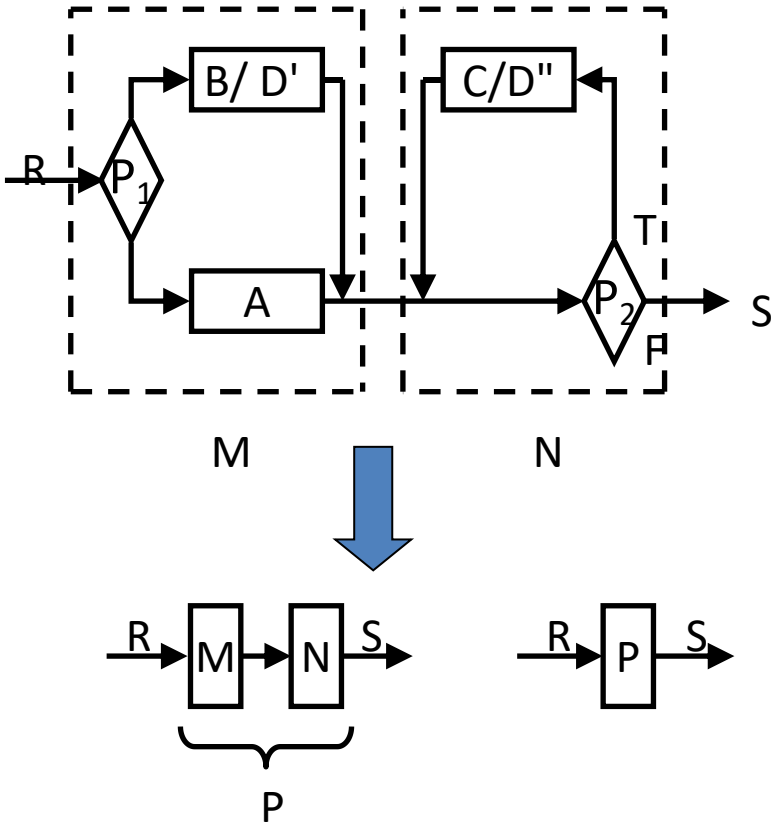
- “面条碗”程序：是单入口单出口的适形程序，但不是结构程序。



“面条碗”程序　　结构程序

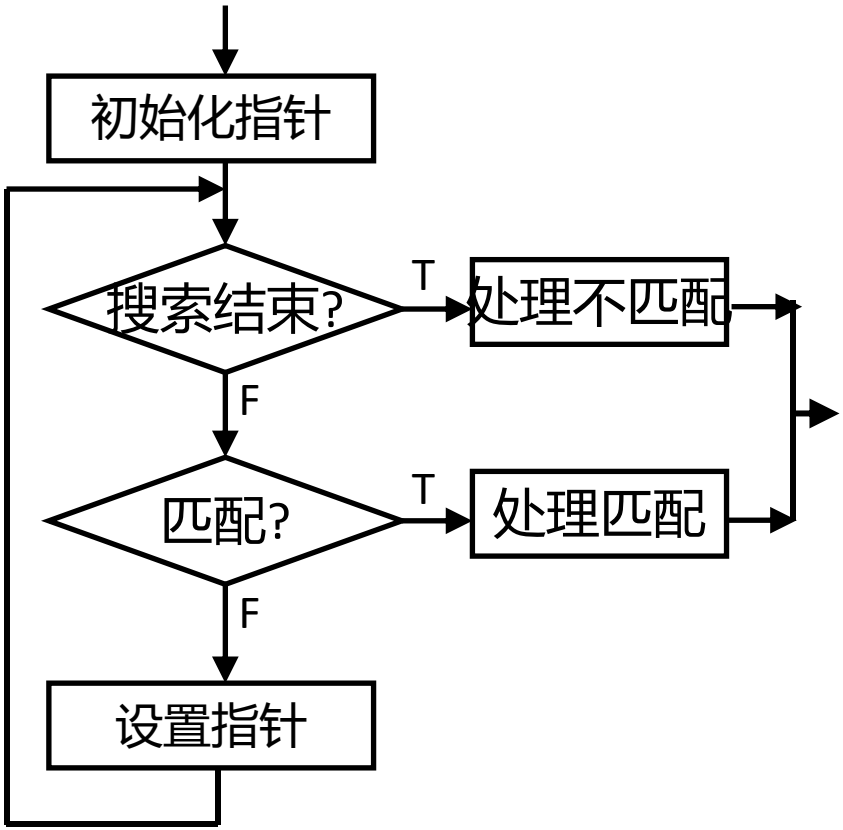
非结构程序转换为结构程序

- “面条碗”程序：是单入口单出口的适形程序，但不是结构程序。



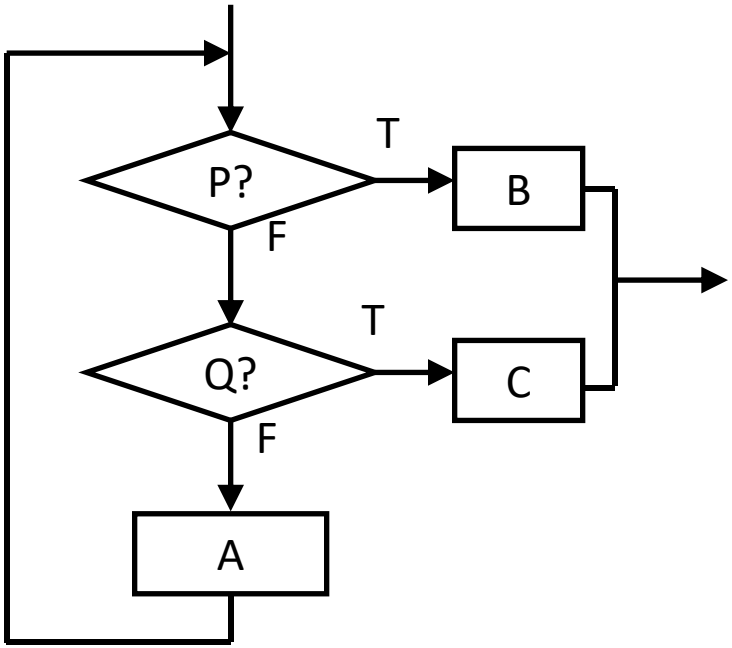
具有两个出口的循环结构

某搜索元素的表或文件的流程图描述如下:

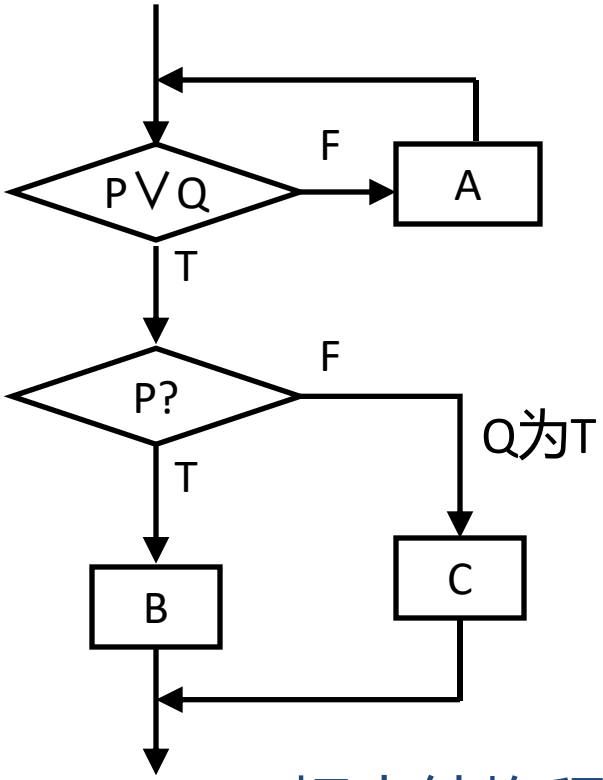


搜索算法的程序结构

无标志转换方式（布尔标志技术）

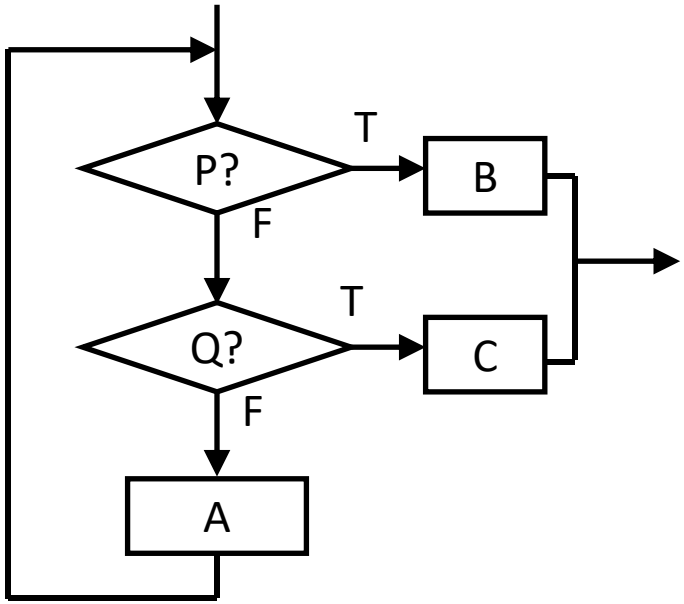


非结构程序

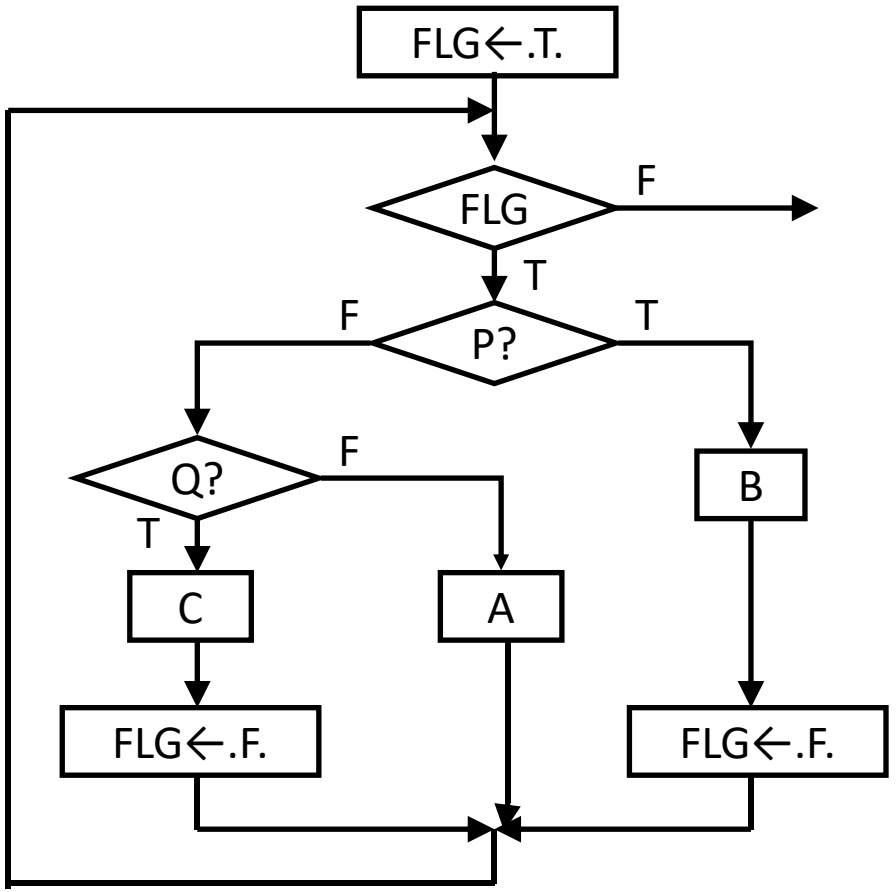


无标志结构程序

有标志转换方式

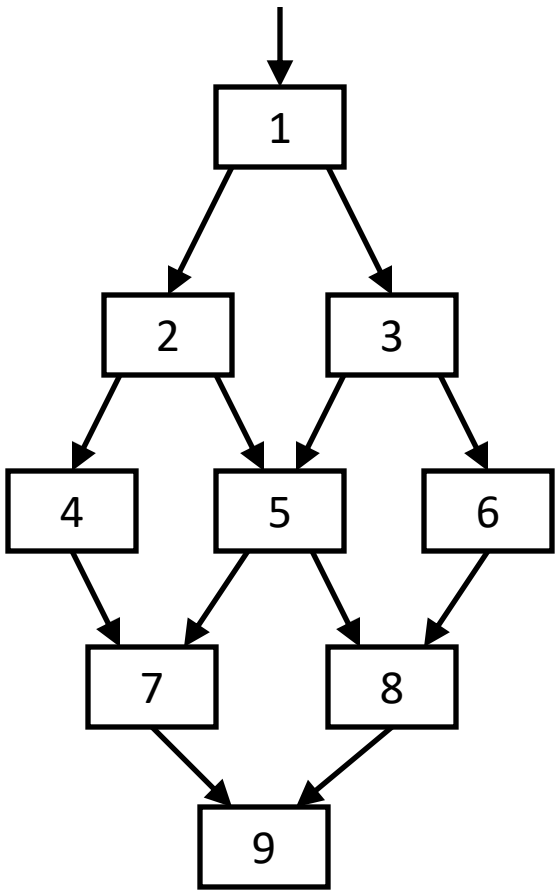


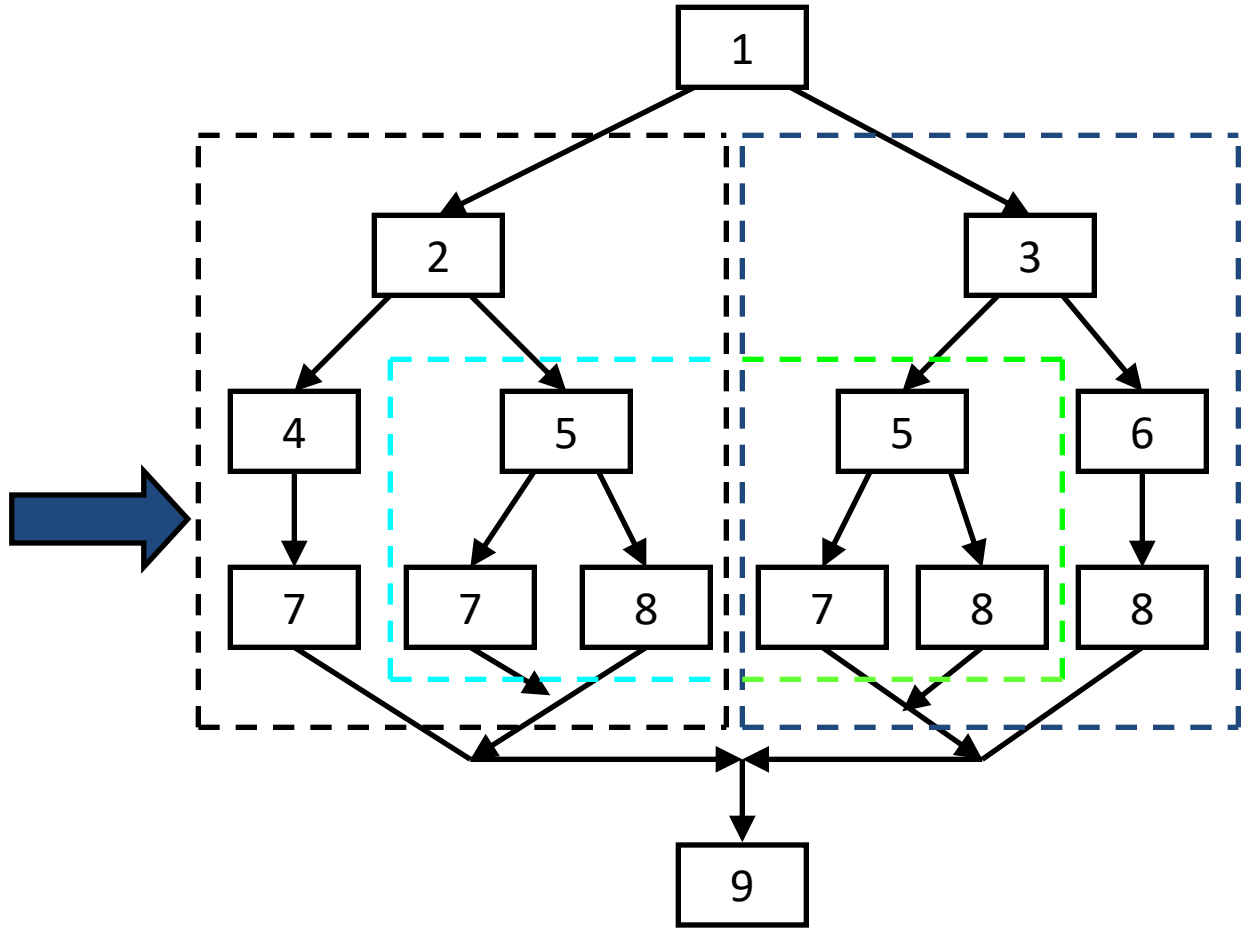
非结构程序



有标志结构程序

编码复制技术

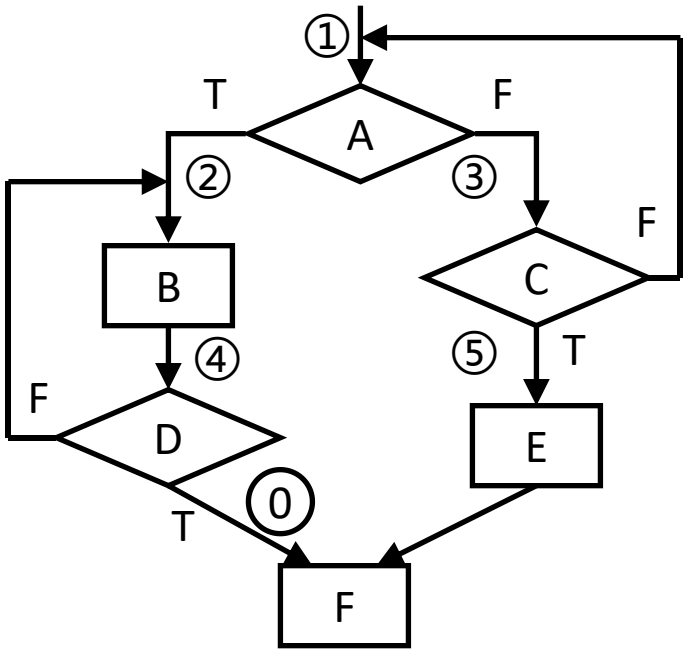


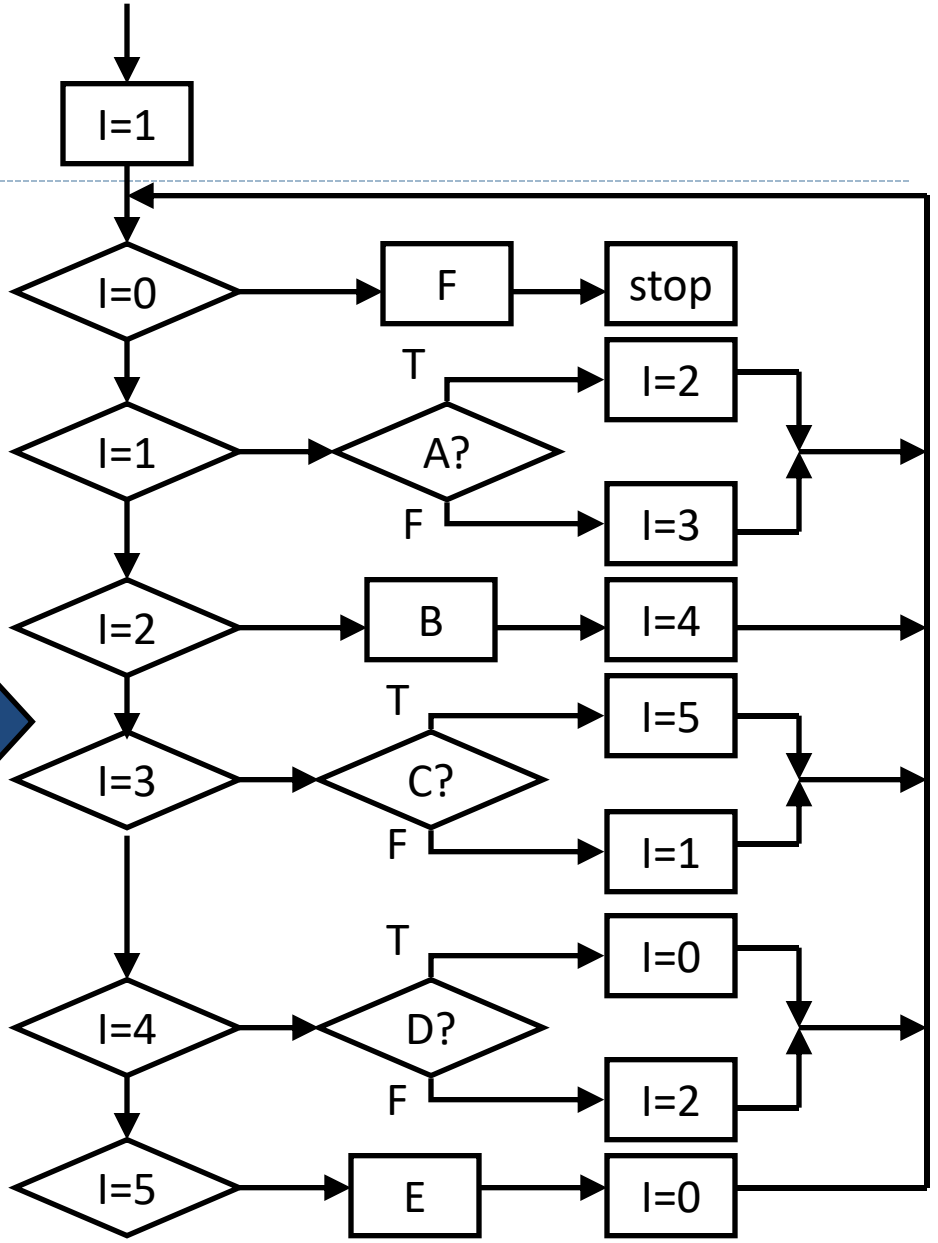


非结构程序

结构程序

状态变量方法



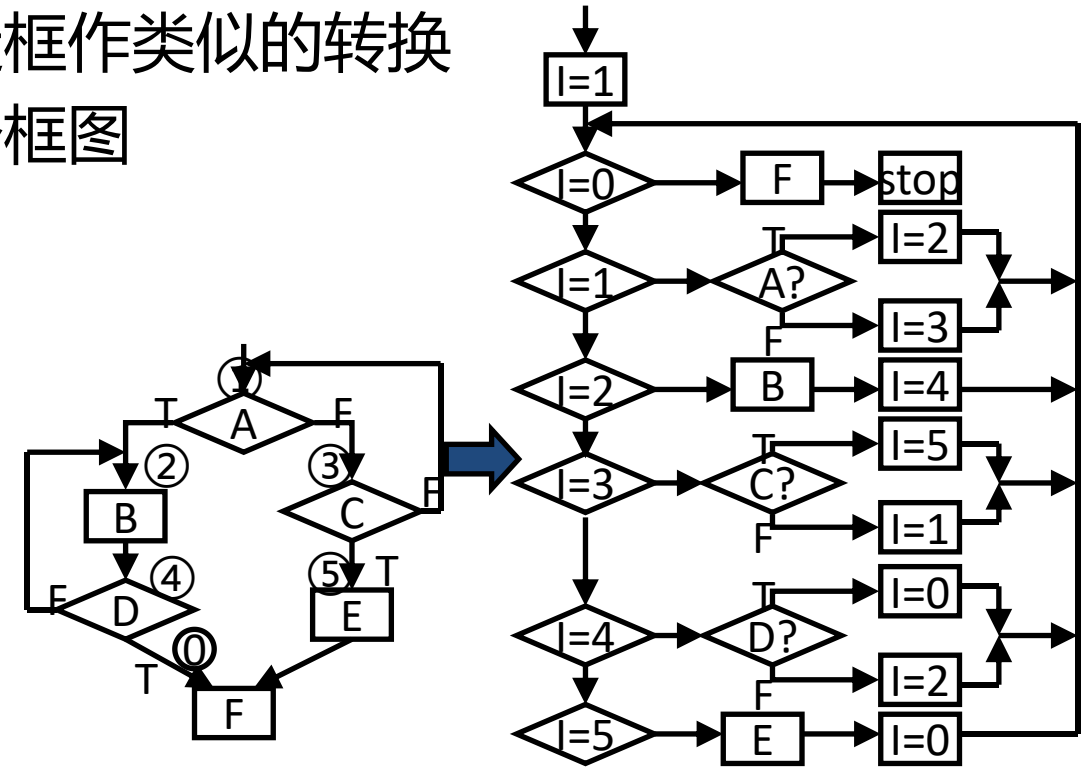


复杂的非结构程序

对应的结构程序

状态变量方法变换步骤

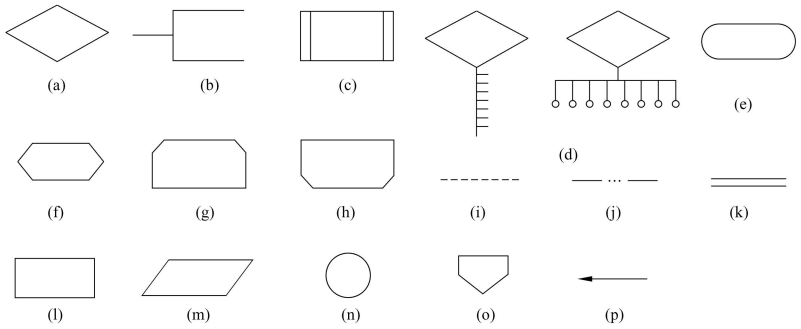
- 第一步：为每个箱编号码
- 第二步：引入新的变量I
- 第三步：用新过程箱代替旧过程箱
- 第四步：对于判定框作类似的转换
- 第五步：重绘整个框图

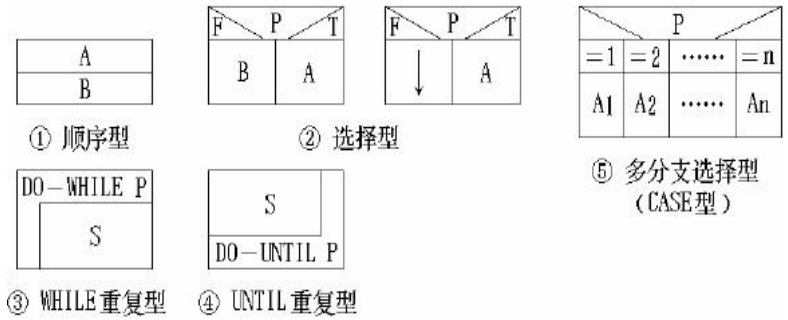


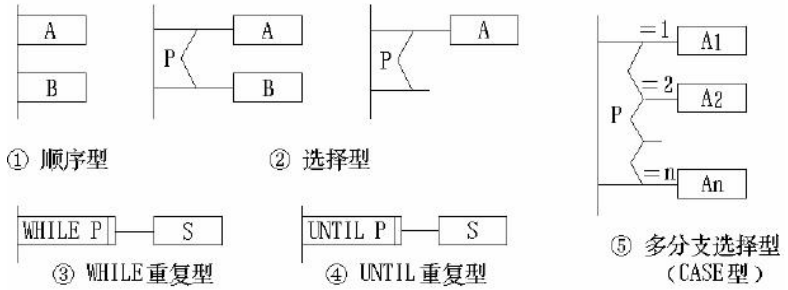
复杂的非结构程序　　对应的结构程序

结构程序的图形表示法

- 程序流程图
- N-S图
- PAD

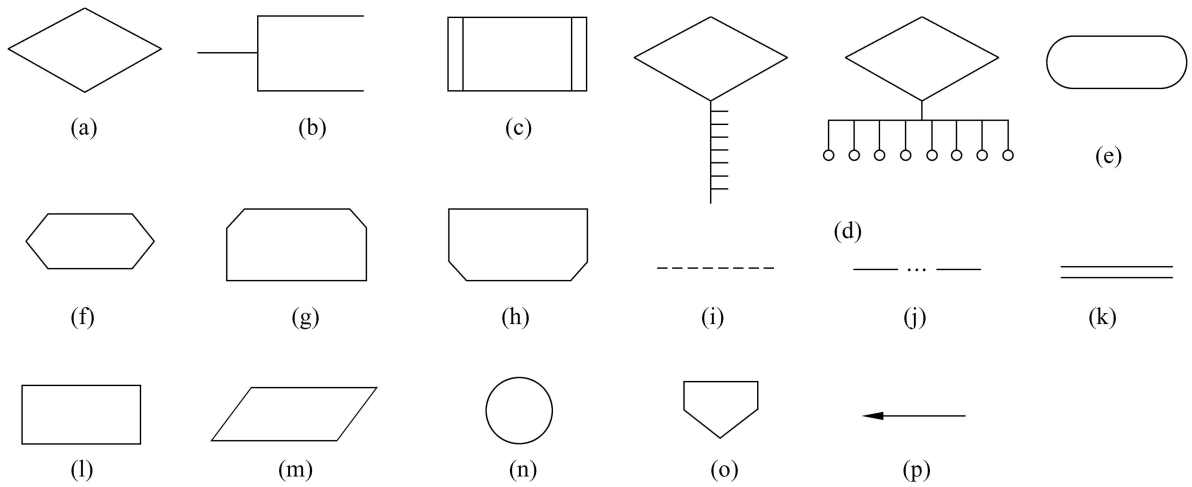






程序流程图

- 程序流程图又称为程序框图，它是历史最悠久、使用最广泛的描述过程设计的方法
- 它的优点是对控制流程的描绘很直观，便于初学者掌握
- 程序流程图历史悠久，至今仍在广泛使用

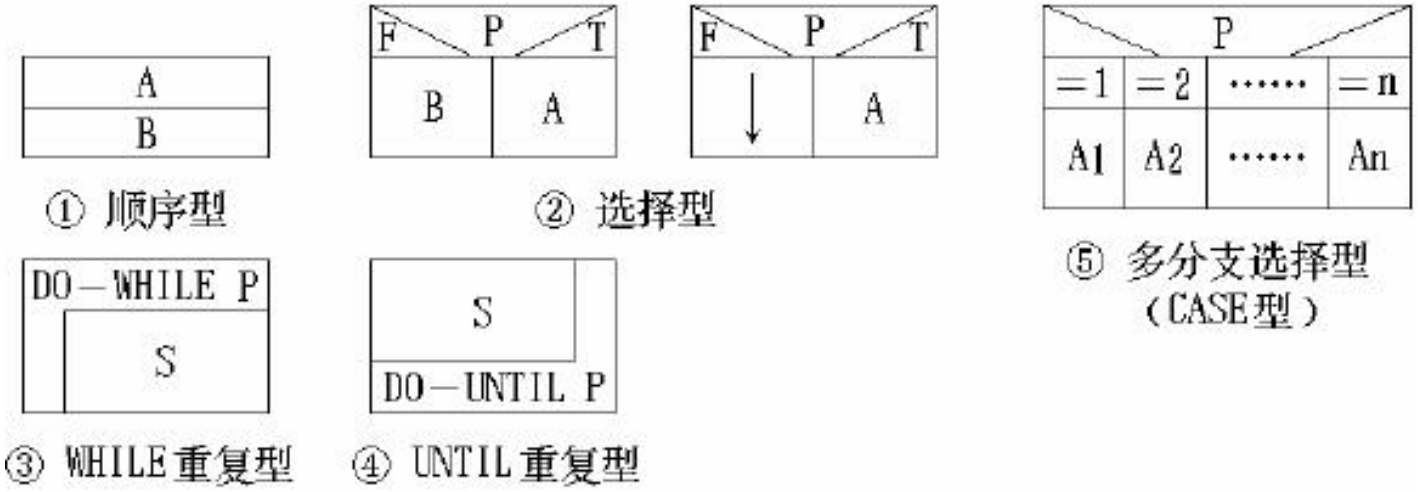


程序流程图的主要缺点

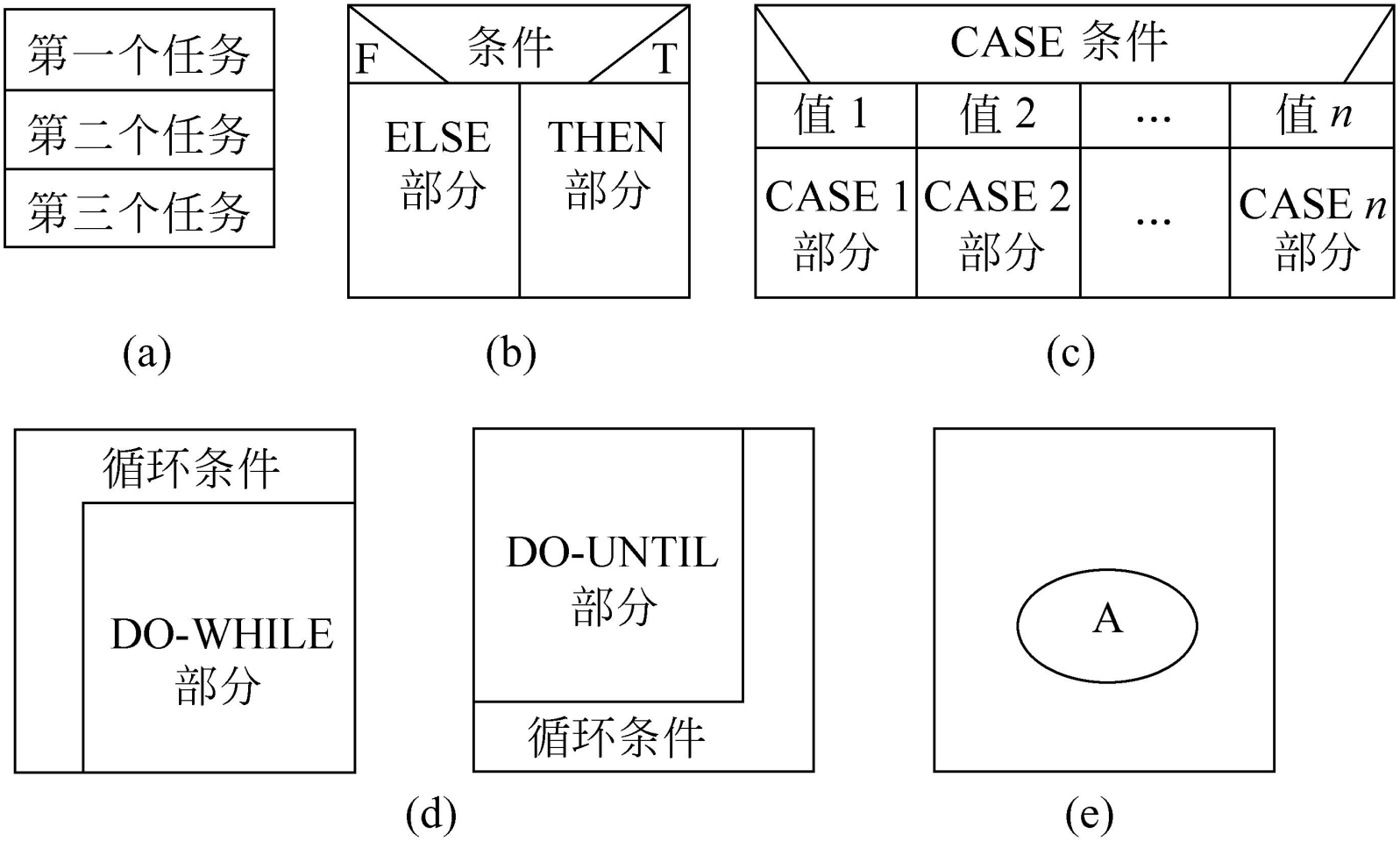
- 程序流程图本质上不是逐步求精的好工具，它诱使程序员过早地考虑程序的控制流程，而不去考虑程序的全局结构。
- 程序流程图中用箭头代表控制流，因此程序员不受任何约束，可以完全不顾结构程序设计的精神，随意转移控制。
- 程序流程图不易表示数据结构。

盒图/N-S图

- Nassi和Shneiderman提出了一种符合结构化程序设计原则的图形描述工具，叫做**盒图**，也叫做N-S图
- 盒图没有箭头，因此不允许随意转移控制
- 坚持以盒图为设计工具，可以使程序员逐步养成用结构化方式思考问题和解决问题的习惯
- N-S图的五种基本控制结构如下所示



盒图的基本符号


第一个任务
第二个任务
第三个任务
F
条件
T
ELSE
部分
THEN
部分
CASE 条件
值 1
值 2
…
值 n
CASE 1
部分
CASE 2
部分
…
CASE n
部分
(a)
(b)
(c)
循环条件
DO-WHILE
部分
DO-UNTIL
部分
循环条件
A
(d)
(e)

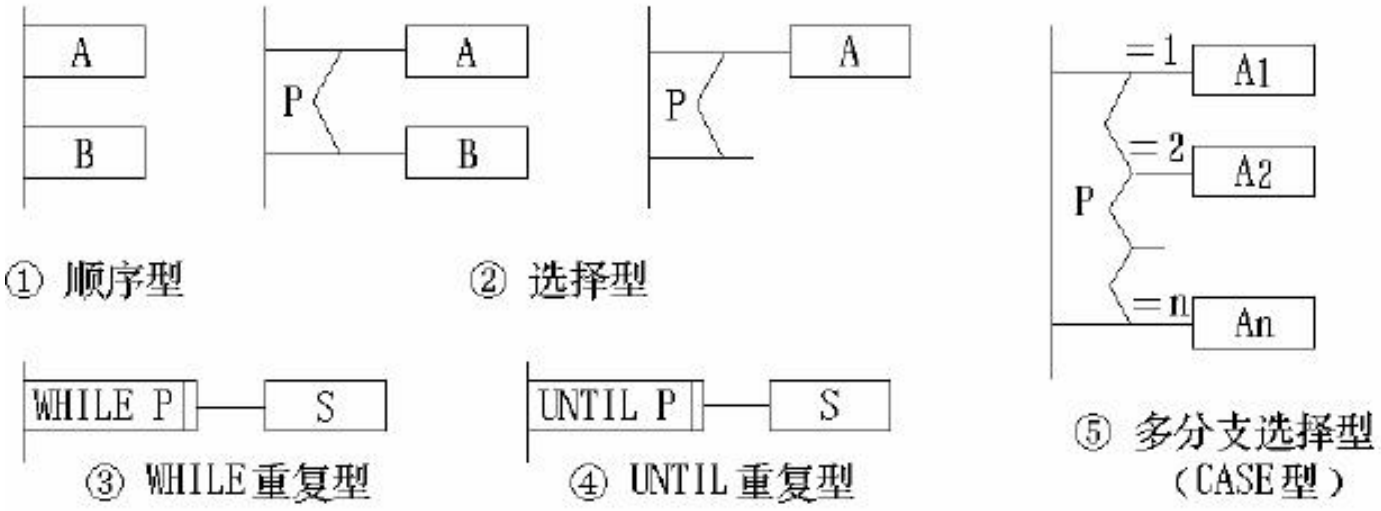
盒图具有下述特点

- 功能域明确
- 不可能任意转移控制
- 很容易确定局部和全程数据的作用域
- 很容易表现嵌套关系，也可以表示模块的层次结构

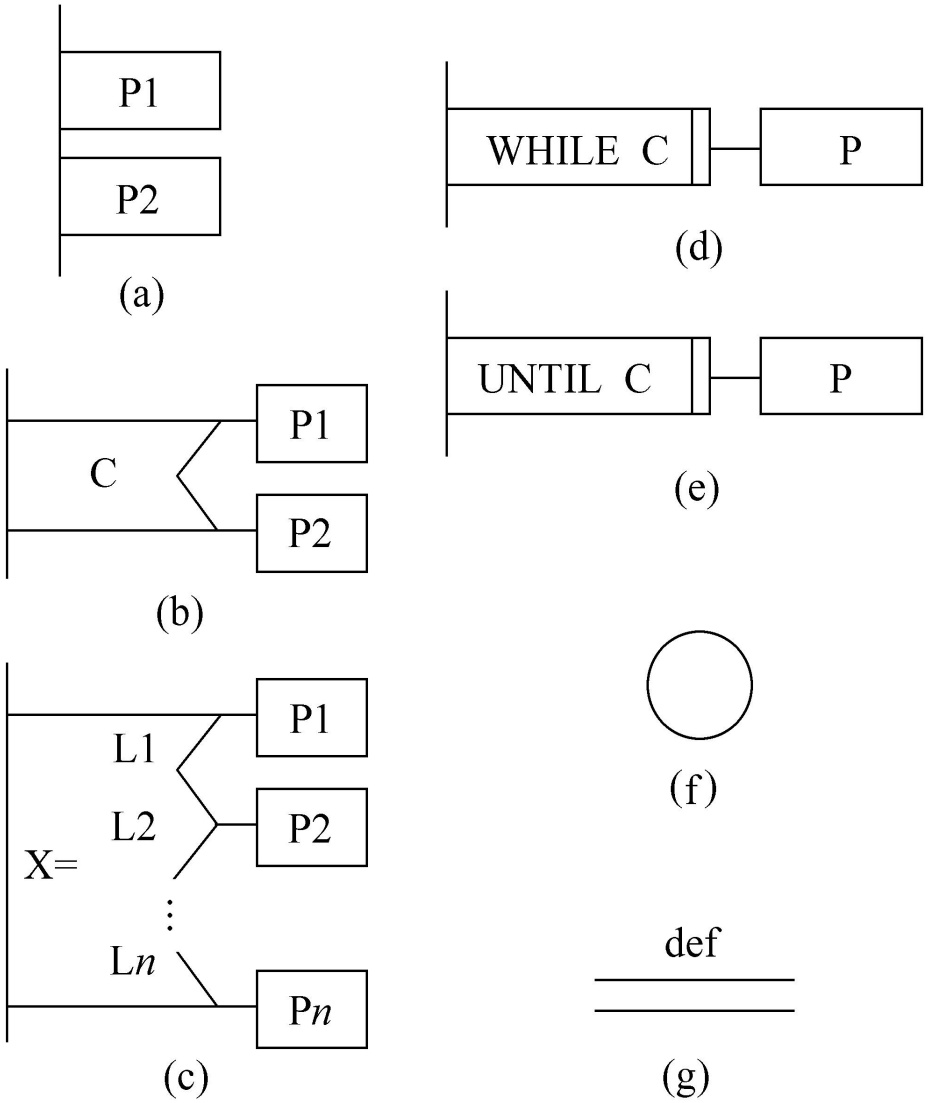
PAD图

- PAD是问题分析图(problem analysis diagram)的英文缩写，由程序流程图演化而来
- 自1973年由日立公司发明以后，已得到一定程度的推广
- 它用二维树形结构的图来表示程序的控制流，将这种图翻译成程序代码比较容易

PAD图的五种基本控制结构如下所示



PAD图的基本符号

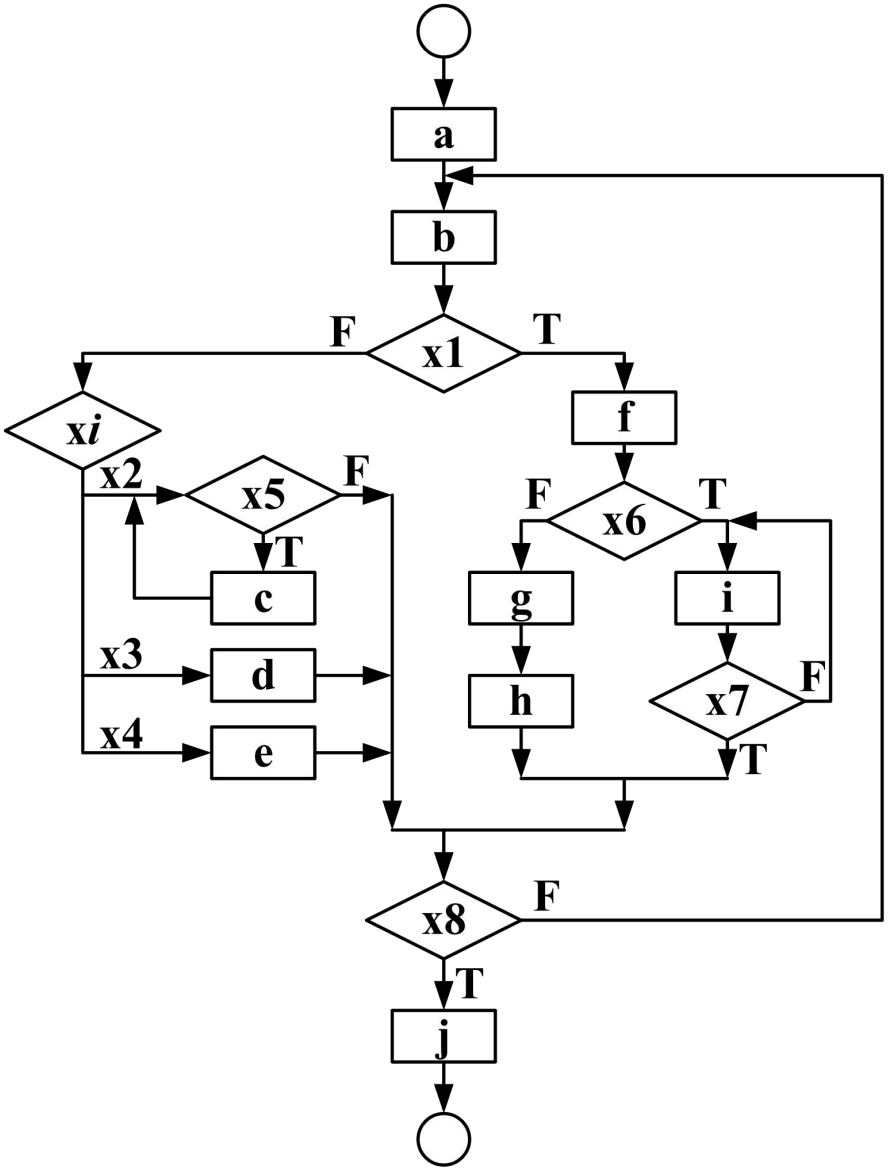


PAD图的主要优点如下

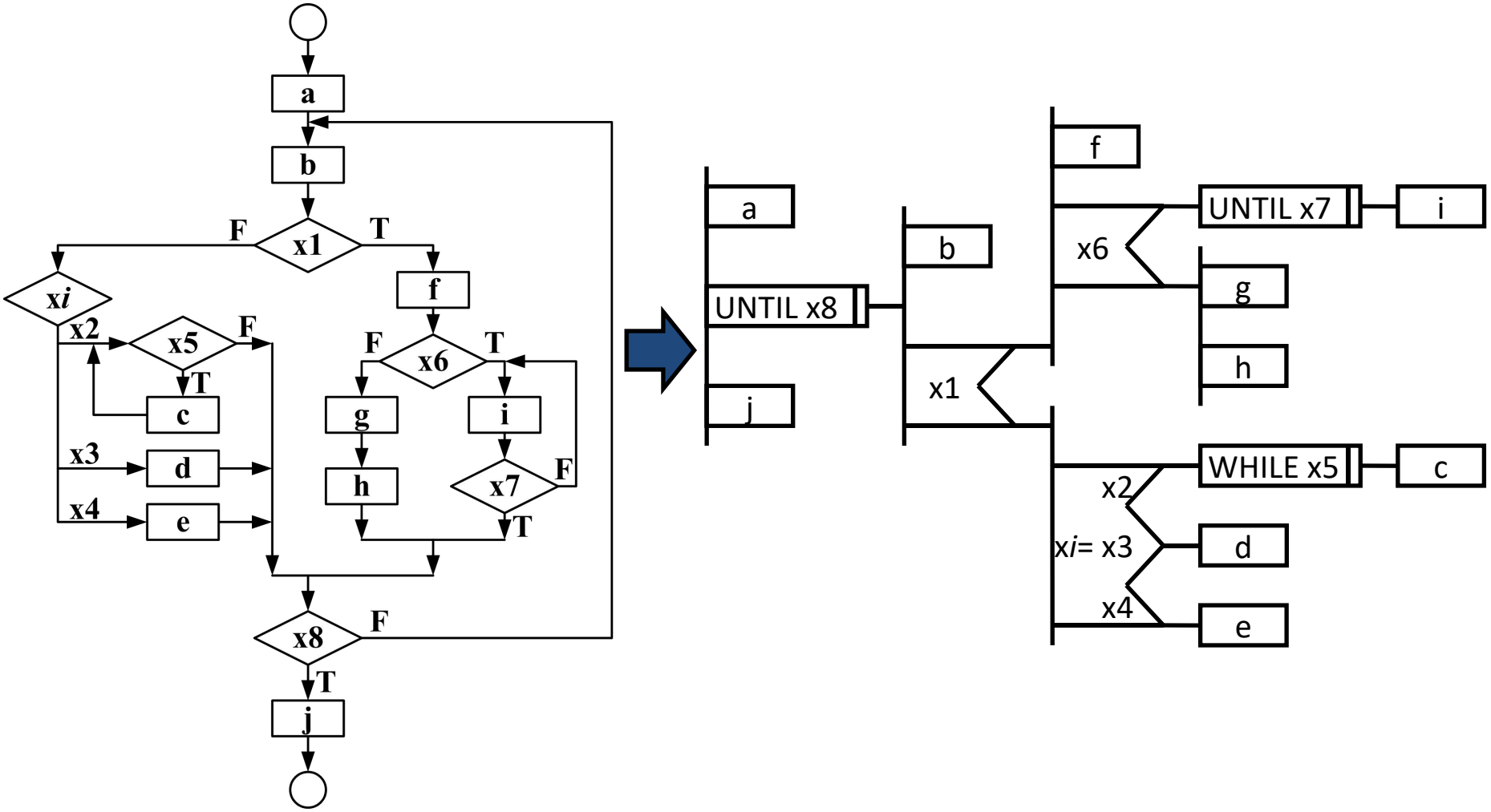
- 使用表示结构化控制结构的PAD符号设计出来的程序必然是结构化程序
- PAD图所描绘的程序结构十分清晰
- PAD图表现程序逻辑易读、易懂、易记
- 容易将PAD图转换成高级语言源程序，这种转换可用软件工具自动完成
- 即可表示程序逻辑，也可描绘数据结构
- PAD图的符号支持自顶向下、逐步求精方法的使用

例题

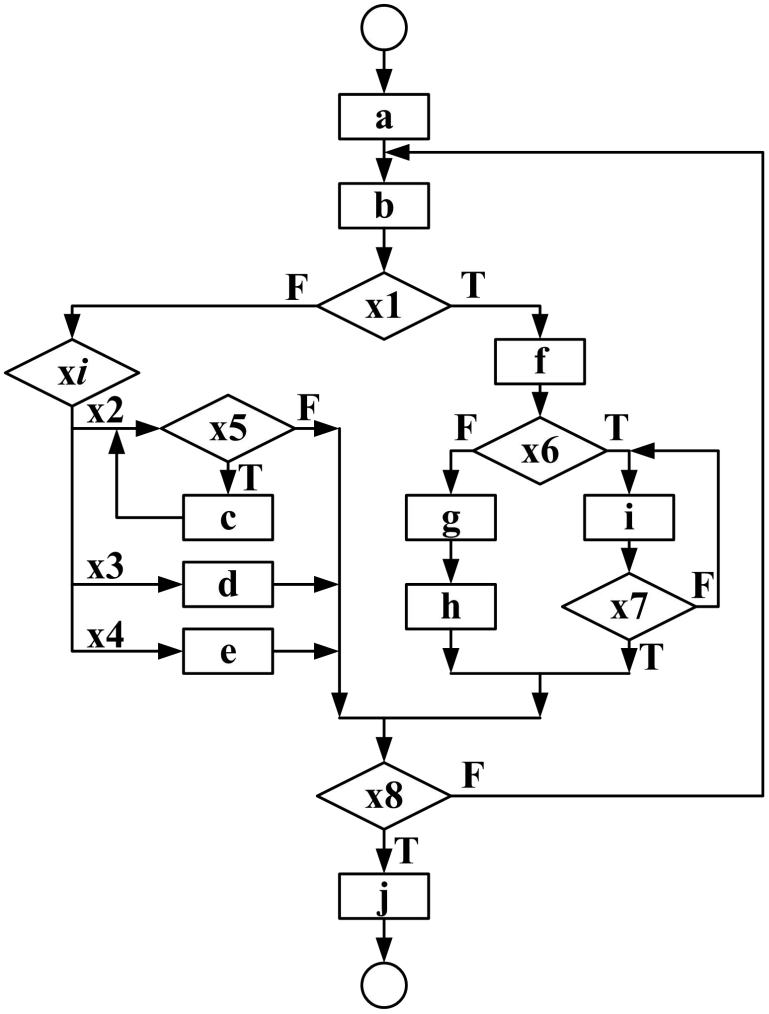
- 某程序流程图如右图所示，请分别用N-S图和PAD图表示

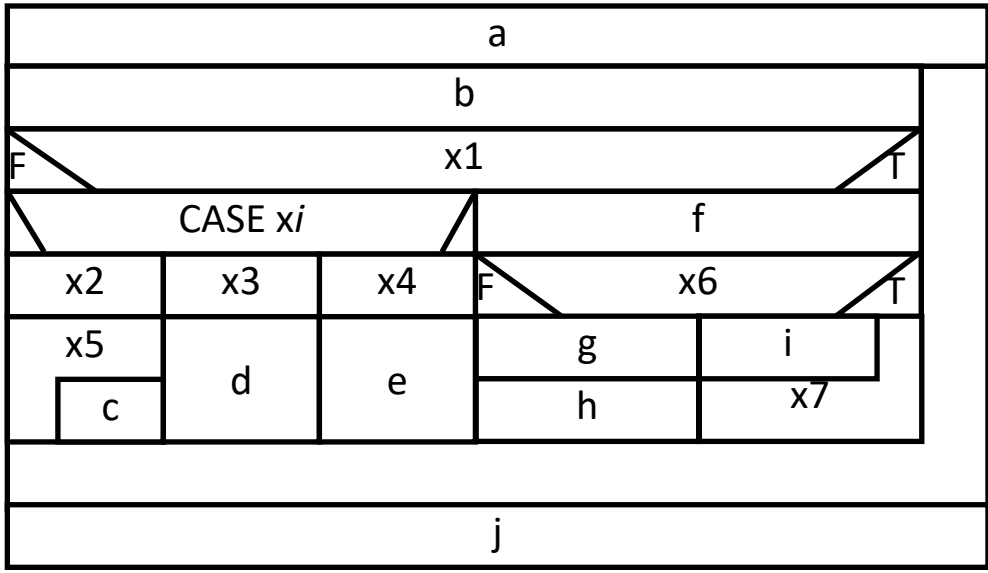


PAD图


a
b
F
x1
T
xi
x2
x5
F
T
c
x3
d
x4
e
f
F
x6
T
g
i
h
x7
F
T
x8
F
T
j
a
UNTIL x8
b
x1
j
f
x6
UNTIL x7
i
g
h
x2
xi= x3
x4
WHILE x5
c
d
e

N-S图


a
b
F
x1
T
xi
f
x2
x5
F
F
x6
T
T
c
g
i
x3
d
h
x7
F
x4
e
T
x8
F
T
j


a
b
F
x1
T
CASE xi
f
x2
x3
x4
F
x6
T
x5
d
e
g
i
c
h
x7
j

判定表

- 当算法中包含多重嵌套的条件选择时，用程序流程图、盒图、PAD图或后面即将介绍的过程设计语言（PDL）都不易清楚地描述
- 判定表却能够清晰地表示复杂的条件组合与应做的动作之间的对应关系

一张判定表由4部分组成

- 左上部：列出所有条件C
- 左下部：是所有可能做的动作A
- 右上部：是表示各种条件组合的一个矩阵
- 右下部：是和每种条件组合相对应的动作

所有条件	条件组合矩阵
所有动作	条件组合 对应的动作

功率>50马力	Y	Y	Y	Y	N	N	N	N
维修记录不全	Y	Y	N	N	Y	Y	N	N
已运行>10年	Y	N	Y	N	Y	N	Y	N
优先维修	√	√	√					
一般处理				√	√	√	√	√

例题

- 假设某航空公司规定，乘客可以免费托运重量不超过30kg的行李
  - 当行李重量超过30kg时，对头等舱的国内乘客超重部分每公斤收费4元，对其他舱的国内乘客超重部分每公斤收费6元
  - 对外国乘客超重部分每公斤收费比国内乘客多一倍；对残障乘客超重部分每公斤收费比正常乘客少一半

用判定表表示计算行李费的算法

	1	2	3	4	5	6	7	8	9
国内乘客		T	T	T	T	F	F	F	F
头等舱		T	F	T	F	T	F	T	F
残障乘客		F	F	T	T	F	F	T	T
行李重量 $W \leq 30$	T	F	F	F	F	F	F	F	F
免费	√								
$(W-30) \times 2$				√					
$(W-30) \times 3$					√				
$(W-30) \times 4$		√						√	
$(W-30) \times 6$			√						√
$(W-30) \times 8$						√			
$(W-30) \times 12$							√		

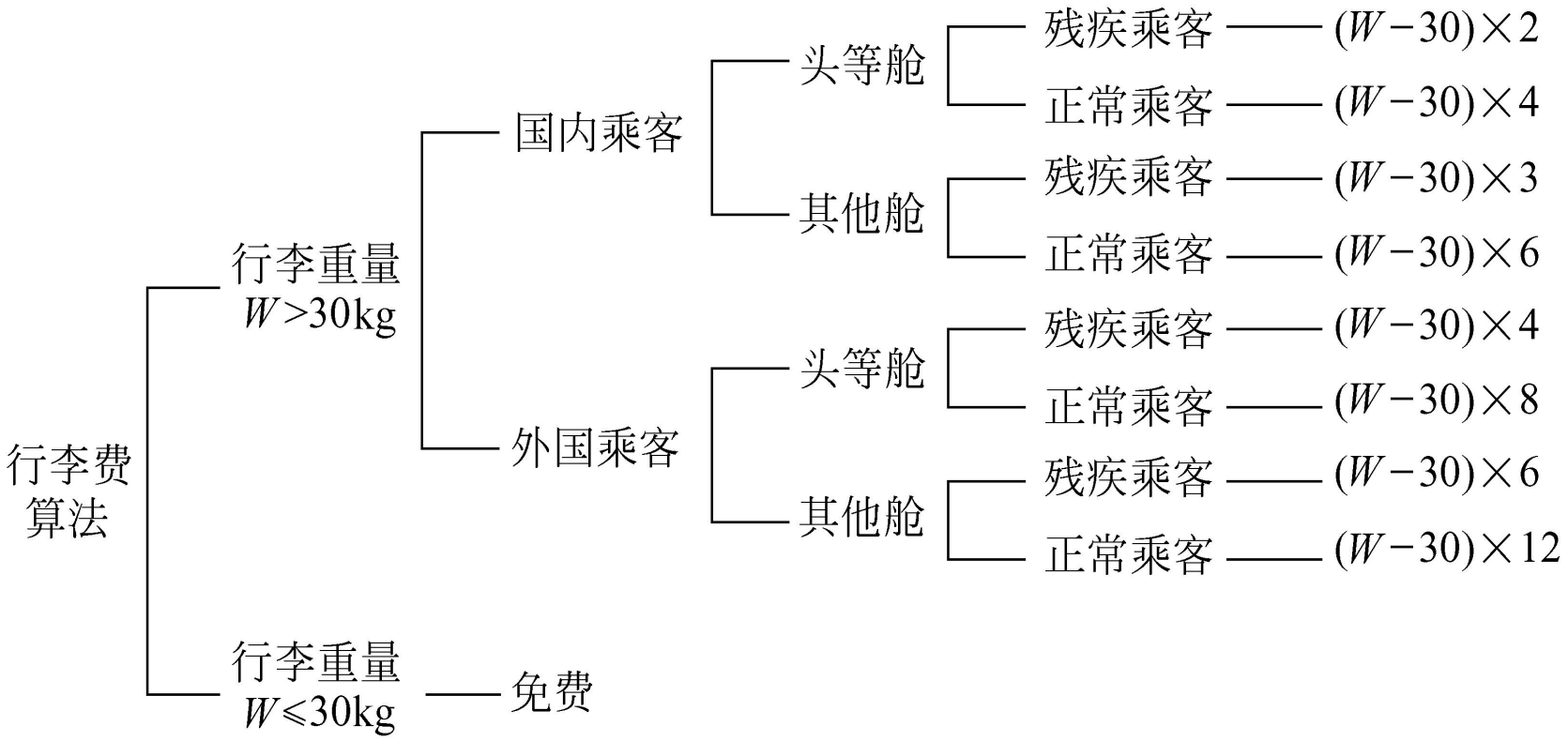
判定表的优缺点

- 判定表的优点：
  - 能清晰地表示复杂的条件组合与应做的动作之间的对应关系
- 判定表的缺点：
  - 判定表的含义不是一眼就能看出来的，初次接触这种工具的人理解它需要有一个学习过程
  - 当数据元素的值多于两个时，判定表的简洁程度也将下降

判定树

- 判定树是判定表的变种，也能清晰地表示复杂的条件组合与应做的动作之间的对应关系
- 多年来判定树一直受到人们的重视，是一种比较常用的系统分析和设计的工具

判定树



用判定树表示计算行李费的算法

判定树的优缺点

- 判定树的优点：
  - 形式简单，一眼就可以看出其含义，因此易于掌握和使用
- 判定树的缺点：
  - 简洁性不如判定表，数据元素的同一个值往往要重复写多遍，而且越接近树的叶端重复次数越多
  - 画判定树时分枝的次序可能对最终画出的判定树的简洁程度有较大影响

过程设计语言（PDL）

- 过程设计语言(PDL)也称为伪代码，它是用正文形式表示数据和处理过程的设计工具
- PDL具有严格的关键字外部语法，用于定义控制结构和数据结构；另一方面，PDL表示实际操作和条件的内部语法通常又是灵活自由的，可以适应各种工程项目的需要
- PDL是一种“混杂”语言，它使用一种语言的词汇，同时却使用另一种语言的语法

PDL的特点

- 有固定的关键字外语法，提供全部结构化控制结构、数据说明和部件特征。属于外语法的关键字是有限的词汇集，它们能对PDL正文进行结构分割，使之变得易于理解。为了区别关键字，规定关键字一律大写，其它单词一律小写
- 内语法使用自然语言来描述处理特性。内语法比较灵活，只要写清楚就可以，不必考虑语法错，以利于人们可把主要精力放在描述算法的逻辑上
- 有数据说明机制，包括简单的（如标量和数组）与复杂的（如链表和层次结构）的数据结构
- 有子程序定义与调用机制，用以表达各种方式的接口说明

PDL的基本控制结构

- 简单陈述句结构：避免复合语句
- 判定结构：IF_THEN_ELSE或CASE_OF结构
- 选择结构：WHILE_DO或REPEAT_UNTIL结构

例：检查订货单

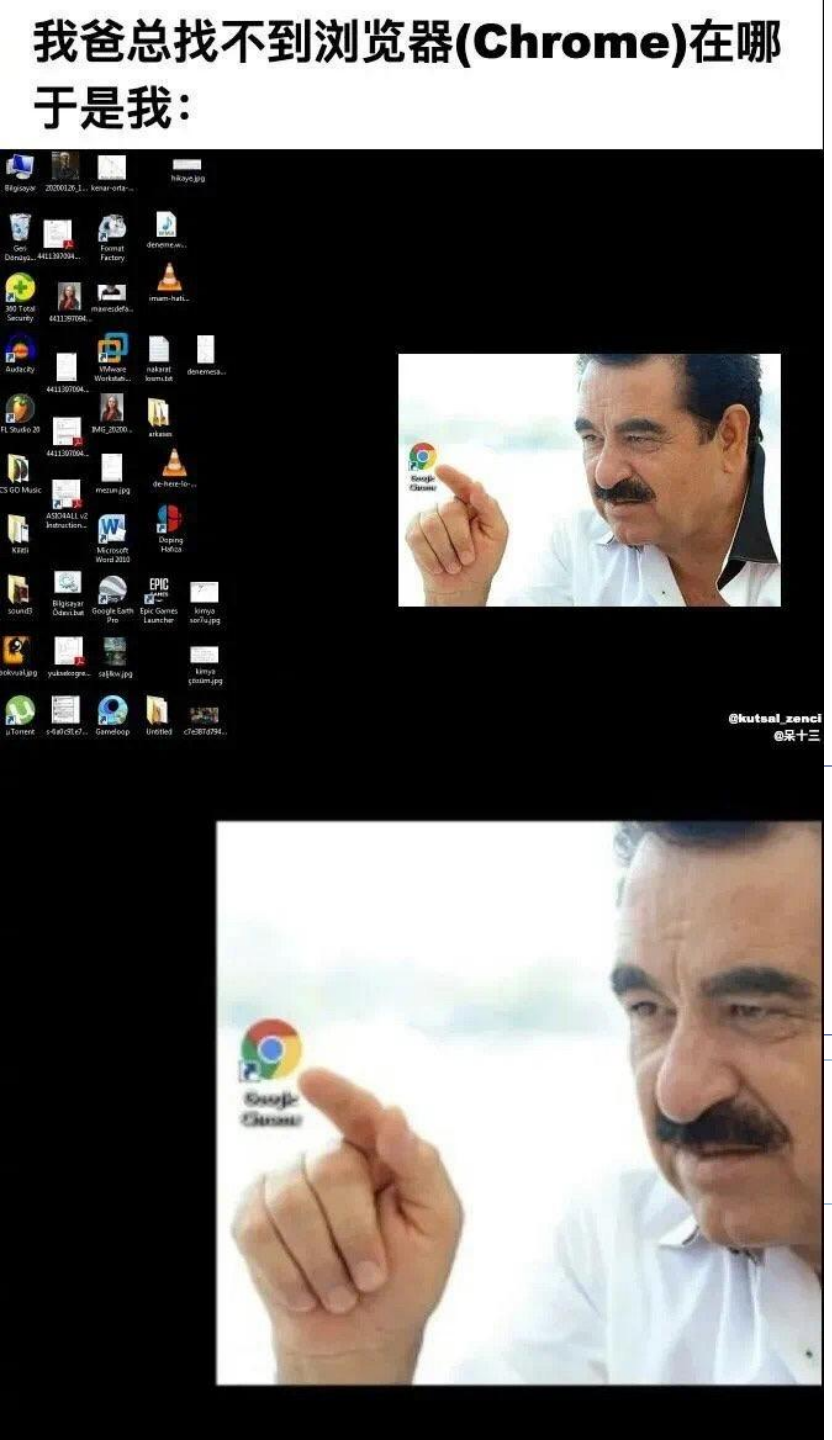
```
IF   客户订货金额超过5000元   THEN
     IF   客户拖延未还赊欠钱款超过60天   THEN
          在偿还欠款前不予批准
     ELSE   (拖延未还赊欠钱款不超过60天)
          发批准书，发货单
     ENDIF
ELSE   (客户订货金额未超过5000元)
     IF   客户拖延未还赊欠钱款超过60天   THEN
          发批准书，发货单，并发催款通知书
     ELSE   (拖延未还赊欠钱款不超过60天)
          发批准书，发货单
     ENDIF
ENDIF
```

PDL的特点

- 关键字的固定语法：它提供了结构化控制结构、数据说明和模块化的特点
- 自然语言的自由语法：它描述了处理过程
- 数据说明的手段：应该既包括简单的数据结构，又包括复杂的数据结构
- 模块定义和调用的技术：应该提供各种接口描述模式

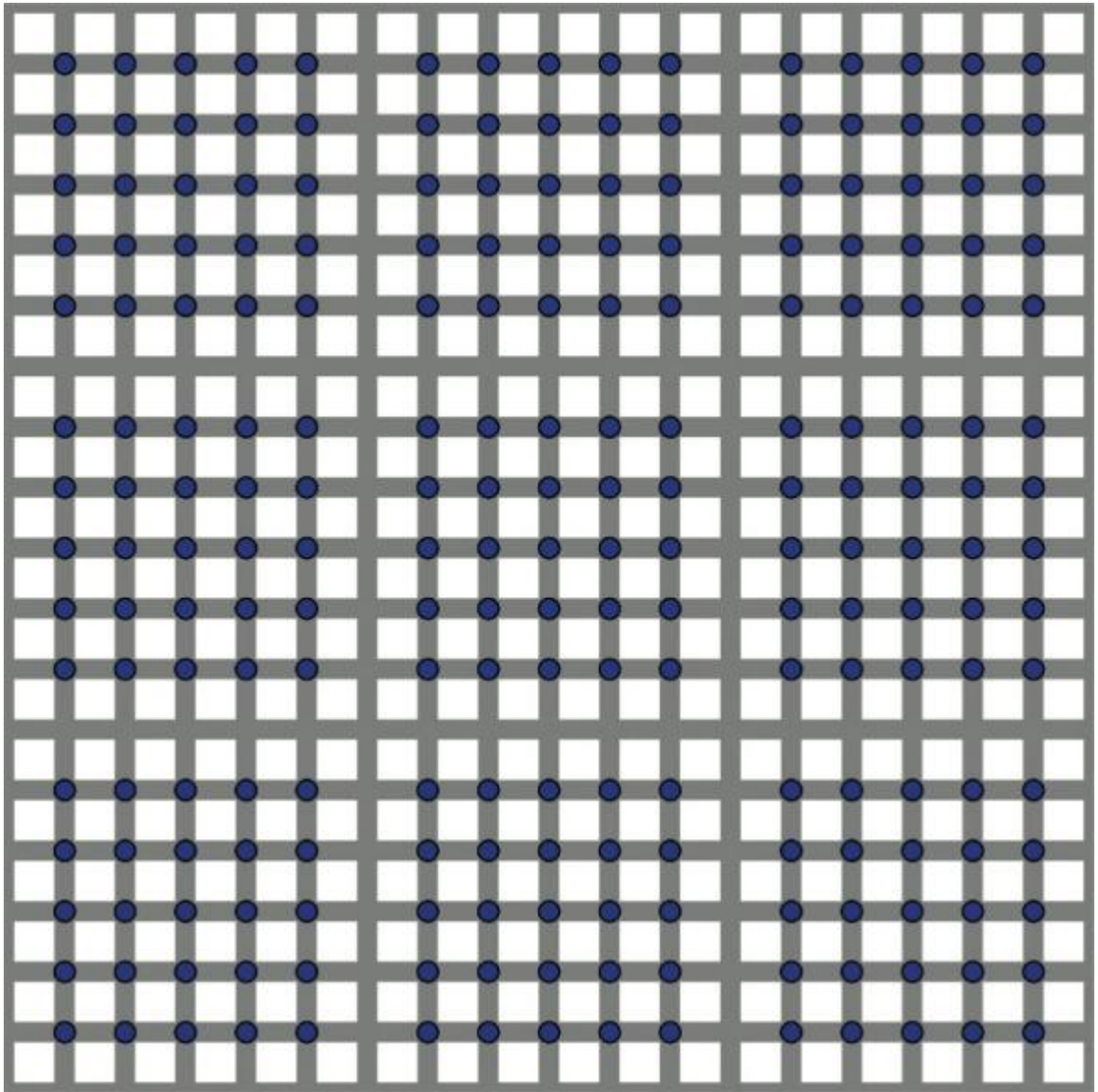
PDL的优缺点

- PDL的优点：
  - 可以作为注释直接插在源程序中间。有助于保持文档和程序的一致性，提高了文档的质量。
  - 可以使用普通的正文编辑程序或文字处理系统，很方便地完成PDL的书写和编辑工作。
  - 已经有自动处理程序存在，而且可以自动由PDL生成程序代码。
- PDL的缺点：
  - 不如图形工具形象直观，描述复杂的条件组合与动作间的对应关系时，不如判定表清晰简单。



第15章
用户界面设计

蓝宝石之光



斯特鲁普效应（Stroop Effect）



用户界面应具有的特性

- 可使用性
- 灵活性
- 复杂性和可靠性



可使用性

- **简单性**
  - 易于理解、附加信息量少、自动化程度高、 操作简便等
- **一致性**
  - 术语符合工程规范、术语没有歧义
- **有Help功能**
- **快速响应、系统开销小**
- **界面容错**
  - 具有错误诊断、修正错误及出错保护的功能

灵活性

- 针对用户：考虑用户特点、能力、知识水平等，满足要求
- 可定制界面：用户可根据需要制定和修改界面
- 系统能按照用户期望提供不同详细程度的响应信息

复杂性和可靠性

- 在完成预定功能的前提下，界面尽量简单
- 减少用户的短期记忆
- 用户界面应能保证用户正确、可靠地使用系统，保证有关程序和数据的安全性

用户因素

- 人对感知过程的认识
- 用户的技能和行为方式
- 用户所要求完成的整个任务以及用户对人机界面部分 的特殊要求

人对感知过程的认识

- **人通过感觉器官认识客观世界，因此设计用户界面时要充分考虑人的视觉、触觉、听觉的作用**
- **人机界面是在可视介质上实现的，如正文、图形、图表等**
- **人们根据显示内容的体积、形状、颜色等种种表征来解释所获取的可视信息**
  - 字体、大小、位置、颜色、形状等都会直接影响信息提取的难易程度
- **很好地表示可视信息是设计友好界面的关键**

人对感知过程的认识

- 用户从界面提取的信息需要存入人的记忆中，供以后回忆和使用。**在设计人机界面时不能要求用户记住复杂的操作顺序**
- 大多数人遇到问题时不进行形式的演绎和归纳推理，而是使用一组启发式策略，这组策略是以往对类似问题的处理中逐渐获得的
- 因此，设计人机界面时应便于用户积累有关**交互的经验**，同时要注意启发式策略的一致性，不宜受特殊交互的影响。如undo/exit等有统一的含义、位置和表示

用户的技能和行为方式

- 用户本身的技能、个性上的差异、行为方式的不同，都可能对人机界面造成影响
- **用户的技能**直接影响他们从人机界面上获取信息的能力，影响交互过程中对系统作出反应的能力，以及使用启发式策略与系统和谐地交互的能力
- 应根据用户的特点设计人机界面

用户分类

- **外行型**：不熟悉计算机操作，对系统很少或毫无认识
- **初学型**：对计算机有一些经验，对新系统不熟悉，需要相当多的支持
- **熟练型**：对系统有丰富的使用经验，能熟练操作，但不了解系统内部结构，不能纠正意外错误，不能扩充系统能力
- **专家型**：了解系统内部的结构，有系统工作机制的专门知识，具有维护和修改系统的能力，希望为他们提供具备修改和扩充系统能力的复杂界面

用户分类示例

- Win XP控制面板的向导功能适合不太熟练的用户



用户特性度量

- 用户使用频度：频度低则需要更多帮助
- 用户需要自由选用界面的情况
- 用户对计算机熟悉程度
- 用户知识
- 用户思维能力
- 用户生理能力和技能：特殊用户群

人机界面风格

- **第一代：命令和询问方式的界面**
  - 正文形式的通信，通过用户命令和用户对系统询问的响应来完成。由于使用正文通信，因此用户容易出错，界面不友善，难以学习
- **第二代：简单的菜单式界面**
  - 与第一代界面相比不易出错，但使用起来乏味，逐层进行不能一步到位
- **第三代：窗口、图标、菜单、指示器四位一体的界面**
  - 能同时显示不同种类的信息，可在多个工作环境（窗口）中切换，窗口使用户能自如地执行许多通信型和认知型任 务
  - 通过下拉式菜单可方便地执行控制型和对话型任务
  - 引入图标、下拉式菜单、按钮和滚动杆技术，大大减少键盘输入，提高交互效率
- **第四代：第三界面与超文本、多任务概念相结合的界面，用户可同时执行多个任务**

语言界面

- **根据语言的特点命令语言界面可分为：**
  - 形式语言。这是一种人工语言，特点是简洁、严密、高效，不仅是操纵计算机的语言，而且是处理语言的语言
  - 自然语言。特点是具有多义性、微妙、丰富
  - 类自然语言。这是计算机语言的一种特例
- **命令语言要求惊人的记忆和大量的训练，并且容易出错，使入门者望而生畏，但比较灵活和高效，适合于专业人员使用**

图形用户界面

- 图形用户界面（GUI-Graphics User Interface）是当前用户界面的主流，广泛应用于各档台式微机和图形工作站
- 当前各类图形用户界面的共同特点是以**窗口管理系统**为核心，使用键盘和鼠标器作为输入设备。窗口管理系统除基于可重叠多窗口管理技术外，广泛采用的另一核心技术是**事件驱动**（Event-Driven）技术
- 图形用户界面和人机交互过程极大地依赖视觉和手动控制的参与，因此具有强烈的直接操作特点

图形用户界面优缺点

- 图形用户界面中菜单的表现形式比字符用户界面更为丰富，在菜单项中可以显示不同的字体、图标甚至产生三维效果
- 菜单界面与命令语言界面相比，用户只需确认而不需回忆系统命令，从而大大降低记忆负荷。但**菜单的缺点是灵活性和效率较差**，可能不十分适合于专家用户
- 基于图形用户界面的优点是具有一定的文化和语言独立性，并可提高视觉目标搜索的效率。**图形用户界面的主要缺点是需要占用较多的屏幕空间，并且难以表达和支持非空间性的抽象信息的交互**

直接操纵用户界面

- **直接操纵（Direct Manipulation）用户界面更多地借助物理的、空间的或形象的表示，而不是单纯的文字或数字的表示**
- **从用户界面设计者角度看：**
  - 设计图形比较因难，需大量的测试和实验
  - 复杂语义、抽象语义表示比较困难
  - 不容易使用户界面与应用程序分开独立设计
- **总之，直接操纵用户界面不具备命令语言界面的某些优点**

多媒体用户界面

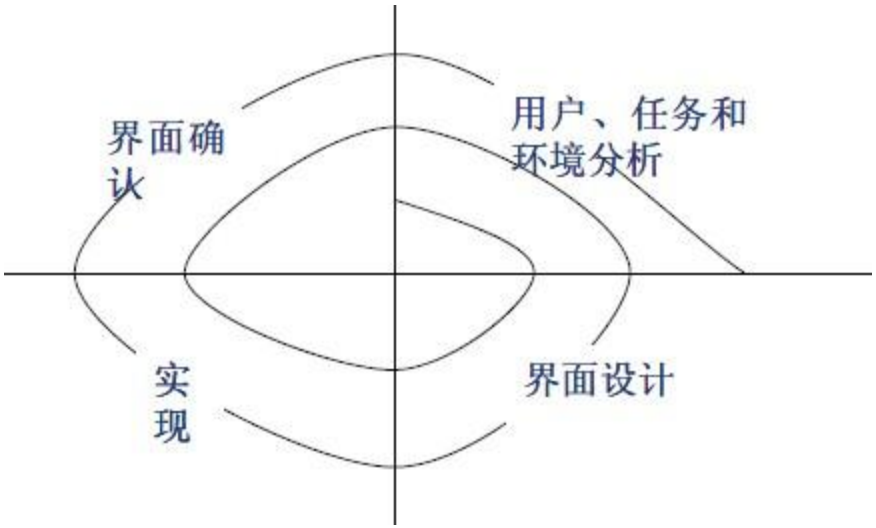
- 多媒体用户界面被认为是在智能用户界面和自然交互技术取得突破之前的一种过渡技术
- 多媒体技术引入了动画、音频、视频等动态媒体，特别是引入了音频媒体，从而丰富了计算机表现信息的形式，拓宽了计算机输出的带宽，提高了用户接受信息的效率
- 典型的界面如：多媒体电子地图，多媒体电子书
- 多媒体用户界面丰富了信息的表现形式，但基本上限于信息的存储和传输方面，并没有理解媒体信息的含义，这是其不足之处，从而也限制了它的应用场合

多通道用户界面

- 80年代后期以来，多通道用户界面（Multimodal User Interface）成为人机交互技术研究的崭新领域，在国际上受到高度重视
- 多通道用户界面综合采用视线、语音、手势等新的交互通道、设备和交互技术，使用户利用多个通道以自然、并行、协作的方式进行人机对话，通过整合来自多个通道的精确的和不精确的输入来捕捉用户的交互意图，提高人机交互的自然性和高效性

人机界面设计过程

- **人机界面的设计过程是迭代的，包括四个不同的框架活动**
  - 用户、任务和环境分析及建模
  - 界面设计
  - 界面构造
  - 界面确认



人机界面设计过程

- 设计人员首先分析将与系统交互的用户的特点。记录下技能级别、业务理解以及对新系统的一般感悟，并定义不同的用户类别。对每一个用户类别，进行需求诱导。软件工程师应试图去理解每类用户的系统感觉
- 一旦定义好一般需求，将进行更详细的任务分析。标识、描述和精化那些用户为了达到系统目标而执行的任务

人机界面设计过程

- **用户环境分析关注系统物理工作环境。通常问这些问题：**
  - 界面将物理地位于何处？
  - 用户是否坐着、站着或完成其他和该界面无关的任务？
  - 界面硬件是否适应空间、光线或噪音的约束？
  - 是否需要考虑特殊的由环境因素驱动的人的因素？

人机界面设计过程

- **界面设计的目标是定义一组界面对象和动作（以及它们的屏幕表示）。设计完成后，软件工程师根据设计方案，使用实现工具完成界面的构造**
- **界面确认关注：**
  - 界面正确地实现每个用户任务的程度、适应所有任务变更的能力以及达到所有一般用户需求的能力
  - 界面容易使用和学习的程度
  - 用户接受界面作为它们工作中有用工具的程度

人机界面设计中涉及的模型

- **软件工程师创建的设计模型（Design Model）：**
  - 整个系统设计模型包括对软件的数据结构、体系结构、界面和过程的表示。界面设计往往是设计模型附带结果
- **人机工程师创建的用户模型（User Model）：**
  - 用户模型描述系统终端用户的特点。设计前，应对用户分类，了解用户的特点，包括年龄、性别、实际能力（physical abilities）、教育、文化和种族背景、动机、目的以及个性

人机界面设计中涉及的模型

- **终端用户在脑海里对界面产生的映象，称为系统感觉（System Perception）：**
  - 系统感觉是终端用户主观想象的系统映象，它描述了期望的系统能提供的操作，其描述的精确程度依赖于终端用户对软件的熟悉程度
- **系统实现者创建的系统映象（System Image）：**
  - 系统映象包括基于计算机的系统的外在表示（界面的观感） 和用来描述系统语法和语义的支撑信息（书、手册、录像带、帮助文件）。如果系统映象和系统感觉是一致的，用户就会对软件感到舒服，使用起来就很有效

设计问题

- 系统响应时间
- 用户求助设施（user help facilities)
- 错误信息处理
- 命令标记（command labeling)

系统响应时间

- 系统响应时间指从用户执行某个控制动作（如按回车键或点鼠标）到软件作出响应（期望的输出或动作）的时间
- 系统**响应时间长**会使用户感到不安和沮丧。**稳定的响应时间**（如1秒）比不定的响应时间（如0.1秒到2.5秒） 要好

用户求助设施（User help facilities)

- **关于求助设施，在设计时须考虑如下问题：**
  1. 在系统交互时，是否总能得到各种系统功能的帮助？是提供部分功能的帮助还是提供全部功能的帮助
  2. 用户怎样请求帮助？使用帮助菜单、特殊功能键还是HELP命令
  3. 怎样表示帮助？在另一个窗口中、指出参考某个文档（不是理想的方法）还是在屏幕特定位路的简单提示
  4. 用户怎样回到正常的交互方式？可做的选择有：屏幕上显示返回键、功能键或控制序列
  5. 怎样构造帮助信息？是平面式（所有信息均通过关键字来访问）、分层式（用户可以进一步查询得到更详细的信息）还是超文本式

错误信息处理

- **交互系统给出的出错消息和警告应具备以下特征：**
  1. 消息以用户可以理解的术语描述问题
  2. 消息应提供如何从错误中恢复的建议性意见
  3. 消息应指出错误可能导致哪些不良后果（比如破坏数据），以便用户检查是否出现了这些情况或帮助用户进行改正
  4. 消息应伴随着视觉或听觉上的提示，也就是说，显示消息时应该伴随警告声或者消息用闪耀方式，或明显表示错误的颜色显示
  5. 消息应是“非批评性的”（Nonjudgmental），即不能指责用户

命令标记（Command Labeling）

- **在提供命令交互方式时，必须考虑以下问题：**
  1. 每一个菜单选项是否都有对应的命令？
  2. 以何种方式提供命令？控制序列、功能键还是键入命令？
  3. 学习和记忆命令的难度有多大？命令忘了怎么办？
  4. 用户是否可以定制和缩写命令？

界面设计的黄金原则

- 让用户拥有控制权
- 减少用户的记忆负担
- 保持界面一致

1. 让用户拥有控制权

- 交互模式的定义不能强迫用户进入不必要的或不希望的动作的方式
- 提供灵活的交互（键盘，鼠标，语音等）
- 允许用户交互可以被中断和撤销
- 当技能级别增长时可以使交互流水化并允许定制（宏）
- 使用户隔离内部技术细节（如：不应要求用户在应用软件中输入操作系统命令）

2. 减少用户的记忆负担

- 减少对短期记忆的要求
- 建立有意义的缺省
- 定义直觉性的捷径
- 界面的视觉布局应该基于真实世界的隐喻
- 以不断进展的方式揭示信息（界面以层次化方式进行组织，由抽象到具体）

3. 保持界面一致

- 允许用户将当前任务放在有意义的语境中，用户应该能够确定他来自何处以及存在什么途径转换到新任务
- 在应用系列内保持一致性（如：Office)
- 不要改变用户已经熟悉的用户交互模型（如：Ctrl + C)

设计评估

- 一旦建立好操作性用户界面原型，必须对其进行评估，以确定是否满足用户的需求
- 对任何一个应用系统， 评估计划必须包含长期持续测试的方法，以便对界面在整个生命周期里出现的各种问题进行不断的评估和修正
- 对于关键系统的界面设计，需要开发特别的评估计划，例如核反应堆等系统的人机界面
- 有效的设计评估包括专家评审和可用性测试

专家评审

- 正式的专家评审需要依托专家作为支柱或者顾问，这些专家往往具有丰富的应用领域或者用户界面领域的专业知识
- 专家评审可以在设计阶段的前期或者后期进行
- 对于评审结果，可由进行评审的专家们出一份正式报告，其中包含评审中所发现的问题以及对其修改的建议；或者由这些专家与设计人员或者管理人员直接进行面对面讨论

专家评审

- 专家评审的方法包括启发式评审、指导文档评审、一致性检查、认知尝试和正式的可用性评审
- **启发式评审**：评审人员对界面进行评判，以便使其与一系列的设计启发规则相符合，如果评审人员熟悉这些规则并能够理解应用，那将对评审非常有利
- **指导文档评审**：检查所涉及的界面与组织内的指导文档或者其他的一些指导文档是否相符

专家评审

- **一致性检查**：检查所有同类界面的一致性，检查内容包括实际界面中的术语、颜色、布局、输入输出格式等与培训材料或者在线帮助是否一致
- **认知尝试**：专家模仿用户使用界面执行典型的任务。 以执行频率高的任务作为起点进行尝试，但执行较少的关键性任务，如错误恢复等也都要尝试到
- **可用性评审**：专家们组织一场讨论，整个设计小组的成员也参与其中，仲裁设计的利弊

- **专家评审可能出现以下问题：专家对任务或用户缺乏足够的理解，且对项目目标有不同的意见，所以必须选择熟悉项目，经验丰富的专家组成专家小组**

可用性测试

- 可用性指的是产品的**使用效率、易学性和舒适程度**
- 对界面进行可用性测试和评价是确保产品可用性的重要手段
- 通过各种可用性测试及早发现界面存在的可用性问题，不仅可以节约开发成本，提高产品的品质， 还可以降低用户使用产品的心理负荷，减少操作错误， 提高工作效率以及对产品的认可度和满意度

可用性测试

- 在进行可用性测试前，设计者需要**制订详细的测试计划**，包括任务列表、主观满意标准以及所要询问的相关问题。同时必须确定参与测试的用户数目、类型和来源
- 可用性测试可以要求用户完成一系列任务，对用户的完成过程进行记录，再对记录进行评审。这可以给设计人员很大的启发，及时发现缺陷并改正

可用性测试

- 虽然可用性测试有很多好处，但也至少存在局限性：它强调的是首次使用的情况，也只能涉及到部分的界面；可用性测试不能延续太长时间，很难确定长时间使用后的情况
  - 例如Microsoft公司的MSN Messenger的“用户帮助改进计划”，就是相当庞大的一个可用性测试计划。当然虽然问题可能会不断地出现，但在适当的时候，必须果断地完成原型测试并交付产品

第17章： 编码设计

概述

- 编码阶段的任务是根据详细设计说明书编写程序
- 程序设计语言的特性和程序设计风格会深刻地影响软件的质量和可维护性
- 为了保证程序编码的质量，程序员必须深刻理解、熟练掌握并正确地运用程序设计语言
- 此外，还要求源程序具有良好的结构性和良好的程序设计风格



程序设计语言的发展史

- **第一代语言**：机器语言和汇编语言
- **第二代语言**：早期的高级语言，如BASIC，FORTRAN，COBOL等
- **第三代语言**：具有很强的数据结构和过程描述能力，支持结构化编程，如Pascal，Modula，C，Ada等
- **第四代语言**：这类语言出现于七十年代，其目的是为了提高程序开发速度，以及让非专业用户能直接编制计算机程序，如：数据库查询语言、报表生成程序、应用生成程序、电子表格、图形语言等。多数是面向领域的，很少是通用的

基本概念

- 程序设计语言是指用于书写计算机程序的语言， 它是一种实现性的软件语言
- 语法（syntax）用来表示构成语言的各个记号之间的组合规则，它是构成语言结构正确成分所需遵循的规则集合
  - 如C语言中for语句的构成规则是：
    - for（表达式1；表达式2；表达式3）语句
  - 语法中不涉及到这些记号的含义，也不涉及使用者

基本概念

- 语义（semantic）用来表示按照各种表示方式所表示的各个记号的特定含义，但它不涉及到使用者
  - 如上述for语句中：表达式1表示循环初值；表达式2表示循环条件；表达式3表示循环的增量；语句为循环体。整 个语句的语义是：
    1. 计算表达式1
    2. 计算表达式2，若计算结果为0，则终止循环；否则转（3）
    3. 执行循环体
    4. 计算表达式3
    5. 转向（2）

基本概念

- 语用（pragmatic）表示构成语言的各个记号和使用者的关系
  - 如：语言是否允许递归？是否要规定递归层数的上界？ 这种上界如何确定？这些都属于语用上的问题

程序设计语言的基本成分

- 程序设计语言基本成份可归纳为四种：数据成分、 运算成分、控制成分、传输成分
  - **数据成分**：它指明该语言能接受的数据，用来描述程序中的数据，如各种类型的变量、数组、指针、记录等。作为程序操作的对象，具有名称、类型和作用域等特征。使用前要对数据的这些特征加以说明。数据名称由用户通过标识符命名，类型说明数据需占用存储单元的多少和存放形式，作用域说明数据可以使用的范围
  - **运算成分**：它指明该语言允许执行的运算，用来描述程序中所需进行的运算。如+ 、 - 、 * 、 / 等
  - **控制成分**：它指明该语言允许的控制结构，人们可利用这些控制成分来构造程序中的控制逻辑。基本的控制成分包括：顺序结构、条件选择结构和循环结构
  - **传输成分**：它指明该语言允许的数据传输方式， 在程序中可用它进行数据传输。例如：控制台格式化输入、 输出函数printf ( )和scanf( )，可以在标准输入输出设备上以各种不同的格式读写数据

程序设计语言的选择

- 为一个特定项目选择编程语言时，通常要考虑如下因素：
  - 应用领域
  - 算法和计算复杂性
  - 软件运行环境
  - 用户需求，特别是性能需求
  - 数据结构的复杂性
  - 软件开发人员的知识水平
  - 可用的编译器与交叉编译器

程序设计语言的选择

- 项目所属的应用领域常常是首要的标准
  - COBOL适用于商业领域
  - FORTRAN适用于工程和科学计算领域
  - Prolog、Lisp适用于人工智能领域
  - Smalltalk、C++适用于OO系统的开发
  - 有些语言适用于多个应用领域，如C
- 若有多种语言都适合于某项目的开发时，也可考虑选择开发人员比较熟悉的语言

程序设计语言的选择

- 选择高级语言还是低级语言
  - 优先选择高级语言
    - 开发和维护高级语言程序比开发和维护低级语言程序容易得多
  - 必要时使用低级语言
    - 对运行时间和存储空间有过高要求的项目
    - 在某些不能提供高级语言编译程序的计算机上开发程序， 如单片机上的软件
    - 大型系统中对系统执行时间起关键作用的模块

程序设计风格

- 编程的依据是详细设计的结果，因此程序的质量主要取决于设计，但编程的质量也在很大程度上影响着程序的质量
- 编程风格主要包括：
  - 源程序文档化
  - 数据说明
  - 语句构造
  - 输入/输出

源程序文档化

- 源程序文档化主要包括：
  - 标识符的命名
  - 注解
  - 程序的视觉组织

标识符的命名

- 选择含义明确的名字，使其能正确提示标识符所代表实体
  - 例如，　　，　　等
- 名字不要太长，太长会增加打字量，且易出错。必要时可使用缩写
- 不用相似的名字，相似的名字容易混淆，不易发现错误
  - 如
- 不用关键字作标识符
- 同一个名字不要有多个含义
- 名字中避免使用易混淆的字符
  - 如数字0与字母　；数字1与字母 或；数字2与字母　等

程序的注释

- 程序中的注解用来帮助人们理解程序，决不是可有可无的
- 一些正规的程序文本中，注解行的数量约占整个源程序的1/3到1/2，甚至更多
- 注解分为序言性注解和功能性注解

序言性注释

- 通常置于每个程序模块的开头部分，主要描述：
  - 模块的功能
  - 模块的接口：包括调用格式、参数的解释、该模块需要调用的其它子模块名
  - 重要的局部变量：包括用途、约束和限制条件
  - 开发历史：包括模块的设计者、评审者、评审日期、修 改日期以及对修改的描述

功能性注释

- 通常嵌在源程序体内，主要描述程序段的功能
- 书写功能性注解时应注意的问题：
  - 注解要正确，错误的注解比没有注解更坏
  - 为程序段作注解，而不是为每一个语句作注解
  - 用缩进和空行，使程序与注释容易区分
  - 注解应提供一些从程序本身难以得到的信息，而不是语 句的重复

视觉组织

- 通过在程序中添加一些空格、空行和缩进等技巧，帮助人们从视觉上看清程序的结构
  - 例如，通过缩进技巧可清晰地观察到程序的嵌套层次，同时还容易发现诸如“遗漏end”那样的错误

```
IF（…） THEN
      IF（…） THEN
          ……
      ELSE
          ……
      ENDIF
      ……
  ELSE
      ……
  ENDIF
```

数据说明

- 为了使程序中数据说明更易于理解和维护，可采用以下风格
  - 数据说明的次序应当规范化
  - 说明语句中变量安排有序化
  - 使用注解说明复杂数据结构

数据说明

- 数据说明次序规范化，使数据属性容易查找，也有利于测试，排错和维护
  - 原则上，数据说明的次序与语法无关，其次序是任意的。但出于阅读、理解和维护的需要，最好使其规范化，使说明的先后次序固定
  - 如：常量说明、简单变量说明、数组说明、公用数据块说明等
- 说明语句中变量安排有序化：当多个变量名在一个说明语句中说明时，可以将这些变量按字母的顺序排列，以便于查找

数据说明

- 使用注解说明复杂数据结构：如果设计了一个复杂的数据结构，应当使用注释来说明在程序实现时这个数据结构的固有特点
  - 例如用户自定义的数据类型，应当在注释中做必要的补充说明

语句构造

- 编码阶段的主要任务就是书写程序语句。有关书写语句的原则有几十种，总的来说，**每条语句尽可能简单明了，能直截了当地反映程序员的意图，不能为了片面追求效率而使语句复杂化**

1. 在一行内只写一条语句

- 在一行内只写一条语句，并且采取适当添加空格的办法，使程序的逻辑和功能变得更加明确
- 许多程序设计语言允许在一行内写多个语句。但这种方式会使程序可读性变差。因而不可取

2. 程序编写首先应当考虑清晰性

- 程序编写首先应当考虑清晰性，不要刻意追求技巧性，使程序编写得过于紧凑
  - 例如，有一个用C语句写出的程序段
    - A[I] = A[I] + A[T];
    - A[T] = A[I] - A[T];
    - A[I] = A[I] - A[T];
  - 此段程序可能不易看懂，有时还需用实际数据试验一下。 实际上，这段程序的功能就是交换A[I]和A[T]中的内容。目的是为了节省一个工作单元。如果改一下：
    - WORK = A[T];
    - A[T] = A[I];
    - A[I] = WORK;
  - 就能让读者一目了然了

3. 程序要能直截了当地说明程序员的用意

- 程序编写得要简单，写清楚，直截了当地说明程 序员的用意

```
for ( i = 1; i <= n; i++ )
  for ( j = 1; j <= n; j++ )
    V[i][j] = ( i / j ) * ( j / i )
```



```
for ( i=1; i <= n; i++ )
   for ( j=1; j <= n; j++ )
      if ( i == j )
         V[i][j] = 1.0;
      ELSE
         V[i][j] = 0.0;
```

4. 其他常用规则

- 先保证正确，再考虑速度
- 尽可能使用库函数
- 避免不必要的转移（goto）
- 尽量只采用三种基本的控制结构来编写程序。除顺序结构外，使用if-then-else来实现选择结构；使用do-until或do-while来实现循环结构

输入和输出

- 输入和输出信息是与用户的使用直接相关的。输入和输出的方式和格式应当尽可能方便用户的使用。要避免因设计不当给用户带来的麻烦
- 因此，在软件需求分析阶段和设计阶段，就应基本确定输入和输出的风格。系统能否被用户接受， 有时就取决于输入和输出的风格

输入和输出

- 不论是批处理的输入/输出方式，还是交互式的输入/输出方式，在设计和编码时都应考虑下列原则：
  1. 对所有的输入数据进行检验，识别错误输入，保证数据有效性
  2. 检查输入项的各种重要组合的合理性，必要时报告输入状态信息
  3. 输入的步骤和操作尽可能简单，并保持简单的输入格式
  4. 输入数据时，应允许使用自由格式输入
  5. 应允许缺省值
  6. 输入一批数据时，使用输入结束标志，而不要由用户输入数据数目
  7. 在交互式输入输入时，要在屏幕上使用提示符明确提示交互输入的请求，指明可使用选择项的种类和取值范围。同时，在数据输入的过程中和输入结束时，也要在屏幕上给出状态信息
  8. 当程序设计语言对输入/输出格式有严格要求时，应保持输入格式与输入语句的要求的一致性；
  9. 给所有的输出加注解，并设计良好的输出报表